
Chapter 3

Beating the S&P 500

The Virtue of Transaction Costs

PATRICK BROCK

Abstract. Machine learning (ML) return forecasts often generate striking results in back-tests, yet their performance frequently collapses once realistic trading frictions are imposed. This has led to scepticism that ML primarily identifies hard-to-arbitrage stocks rather than genuine mispricing (alpha), particularly because ML performance deteriorates in liquid large-cap universes with low limits-to-arbitrage. This paper argues that the problem lies in how return forecasts are translated into trades. Return prediction is an extremely low signal-to-noise environment, and the forecasts exhibit small explanatory power. The dominant implementation in the literature, long-short decile portfolios, maps these noisy forecasts into extreme portfolio reallocations, mechanically amplifying forecast noise, turnover, and transaction costs. I propose a modular predict-then-optimize framework that maps given return forecasts into portfolio weights through a transaction-cost-aware portfolio optimisation. The linear-quadratic optimiser trades off predicted returns against trading costs under realistic investment constraints (long-only, fully invested, and benchmarked risk). Transaction costs enter the optimisation as an economically grounded stock- and period-specific ridge regularisation on rebalancing. The resulting optimiser is a filter that extracts signal from noisy ML return predictions by shrinking portfolio reallocations. Using established ML return forecasts from the literature, I evaluate portfolios in a demanding institutional setting: long-only portfolios within the S&P 500, monthly trading from 2004–2024, and benchmark-relative risk control. In this environment, these ML forecasts fail under long-short decile portfolios, but generate substantial and statistically significant outperformance when passed through my optimiser. The central finding is the Virtue of Transaction Costs: optimising as if transaction costs were large improves trade selection by preventing overtrading from noisy return forecasts. ML forecasts generate a 25% higher Sharpe ratio than that of the S&P 500 (net of costs) with low turnover (5%). Consequently, the key challenge is not prediction itself, but the mapping from forecasts to trades that extract the small amount of signal contained in noisy forecasts.

3.1 Introduction

A large and influential literature documents that modern machine learning (ML) forecasts of stock returns generate striking *gross* (before transaction costs) backtest performance in cross-sectional strategies (e.g., [Gu et al. \(2020\)](#), [Kelly et al. \(2019\)](#), [Freyberger et al. \(2020\)](#), [Cakici et al. \(2023\)](#), [Jiang et al. \(2023\)](#), [Chen et al. \(2024\)](#)). Yet, a parallel body of evidence shows that these gains attenuate sharply once realistic implementation frictions are imposed. In particular, performance weakens in liquid large-cap universes, after transaction costs, or performance often depends on frequent rebalancing or exposure to hard-to-trade securities (e.g., [Chen & Velikov \(2022\)](#), [Avramov et al. \(2023\)](#), [Detzel et al. \(2023\)](#)). This pattern is commonly interpreted through the lens of limits to arbitrage: mispricing is more likely to persist where arbitrage is *constrained*, namely among small, illiquid stocks. Conversely, since the performance of ML predictions is significantly weaker in highly liquid large-cap stocks, where arbitrage is comparatively *less* constrained, the criticism is that ML doesn't find any signal (alpha), it merely finds hard to arbitrage stocks.

Table 3.1 illustrates the empirical foundation for that scepticism. Using one-month-ahead return forecasts from widely used ML predictors, long-short decile portfolios in the broad CRSP universe exhibit high *gross* Sharpe ratios. Yet, when the same construction is restricted to stocks in the highly liquid active S&P 500 universe, where mispricing is quickly arbitrated away, long-short decile performance collapses and falls well below the benchmark.

Table 3.1: *Gross Performance of Long-Short Decile Portfolios.*

ML Predictor	LS Decile CRSP			LS Decile S&P 500		
	μ	σ	Sharpe ratio	μ	σ	Sharpe ratio
XGB	0.344	0.142	2.314	0.011	0.107	-0.037
TF	0.179	0.173	0.945	0.019	0.143	0.030
IPCA	0.184	0.119	1.417	0.038	0.155	0.147
RFF	0.137	0.137	0.886	0.001	0.107	-0.128

Notes: Trading is monthly from 2004-2024. All portfolios are equal-weighted, rebalanced monthly and dollar neutral. The performance of the value-weighted portfolios is uniformly worse. All quantities are annualised. ML portfolios are formed using predicted one-month-ahead stock returns. Predictors used are XGBoost, Transformer, Instrumented Principal Component Analysis ([Kelly et al. 2019](#)) and Random Fourier Features ([Kelly et al. 2023](#)).

I argue that this conclusion is not implied by the evidence. The key problem of Table 3.1 is that the dominant practice for turning return forecasts into portfolios uses no explicit regularisation (shrinkage) whatsoever¹. A rank-based decile rule translates small differences in predicted returns into large reallocations. As a result, the portfolio rule overreacts to forecast noise and induces excessive trading exactly where trading is perhaps not warranted. This is conceptually

¹Shrinkage is implicit because the decile portfolios do filter out 80% of predicted returns, but this filter is ad-hoc and binary.

inconsistent: ML forecasts employ regularisation throughout the return prediction step (cross-validation, shrinkage, early stopping, complexity control), but the second step, mapping forecasts into implementable trades, is essentially treated in the literature as if it were noiseless. The drop in performance in Table 3.1 shows that for large-cap and liquid universes this step is not noiseless. In short, the relevant question is not only whether ML predicts returns, but also whether these predicted returns can be translated into *economically disciplined* trades in a realistic mandate. The literature has tackled the first, but not the second component.

Hence, the key challenge in machine learning return prediction is not the forecasting itself, but translating noisy forecasts into disciplined trading decisions. Under noisy return forecasts, aggressive rebalancing, such as in long-short decile portfolios, translates small differences in predicted returns into large trades. My key idea is to build a filter that uses transaction costs as an economically grounded stock- and period-specific regularisation device that extracts the (little) signal present in noisy ML return predictions. Accordingly, the object of interest is not a new (ML) return prediction algorithm, but an *implementation filter*: a portfolio construction rule that determines which forecast differences are strong enough to justify active rebalancing of portfolio weights once realistic frictions and constraints are imposed.

This filter relies on a modular *predict-then-optimize* framework. In the first stage, I obtain one-month-ahead stock return forecasts in exactly the same manner as is highly common in the literature (Gu et al. 2020). Hence, I do not modify or re-engineer the return prediction in any shape or form. This design choice ensures that any improvement in performance can be attributed to how forecasts are translated into portfolio weights rather than to changes in the forecasting model itself. The four return predictors I use are XGBoost, Transformers, Instrumented Principal Component Analysis, and Random Fourier Feature Regression (trees, neural nets, factor models, regression).

The key contribution of this paper is the second stage. Here, the return forecasts are mapped into portfolio weights by solving a transaction-cost-aware portfolio optimisation problem². Transaction costs are given by quadratic price-impact³ model as in Jensen et al. (2022) who build on Kyle (1985). The optimiser trades off predicted returns against the costs of rebalancing while respecting realistic investment constraints. Exactly this trade-off is responsible for dramatically improving the performance of portfolios based on ML return forecasts. Moreover, the portfolio optimiser is highly flexible: it can immediately incorporate a wide range of inequality and equality constraints or additional transaction costs and trading frictions.

The central finding of this paper is what I call the *Virtue of Transaction Costs* (VoT). The key insight is that treating transaction costs as large within the portfolio optimisation problem (second stage), regardless of the costs actually paid at execution, is the driver behind superior performance. When the portfolio optimiser faces large transaction costs, it shrinks forecast-induced

²This portfolio choice problem nests rank-based portfolios as a special case.

³Price impact entails that trading larger dollar amounts moves prices against the trader, so the marginal cost of trading rises with trade size, especially in less liquid stocks. Managers with larger AUM trade larger sizes and thus incur higher transaction costs.

trades and avoids reacting to small and potentially noisy stock return predictions. As a result, portfolio turnover falls and the trades that are executed are more likely to reflect genuine information rather than forecast noise. For every return predictor considered, portfolios constructed under strong cost discipline deliver higher Sharpe and information ratios, more stable benchmark-relative performance, and materially improved drawdown behaviour at very low and stable turnover for every return predictor. Conversely, when the portfolio optimiser is allowed to behave as if transaction costs are small, turnover rises, realised risk deteriorates, and the portfolios perform worse than the benchmark which is a signature of forecast-noise-induced overtrading. Hence, the central mechanism is that the transaction costs act as a stock-specific and period-specific penalty on *active rebalancing*. This additional regularisation extracts the small amount of signal from the return predictions that remains economically actionable.

To identify this mechanism, I exploit the fact that price-impact transaction costs scale with the wealth level used inside the portfolio optimisation problem. I therefore separate the wealth level that determines the transaction cost penalty inside the portfolio optimiser (*Act as*) from the wealth level used to compute the transaction costs actually deducted from realised returns (*Pay as*). This decomposition isolates two conceptually distinct roles of transaction costs: the mechanical drag from costs paid ex post and the ex-ante regularisation effect that transaction costs impose on portfolio rebalancing decisions. Holding the portfolio allocation fixed by optimising as if costs were large while varying only the costs paid shows that the performance improvement arises from the regularisation of trading behaviour rather than from differences in realised execution costs. In this sense, transaction costs act as a stock-specific and time-varying penalty on active rebalancing that extracts the small amount of economically actionable signal contained in noisy return forecasts.

I deliberately subject my predict-then-optimize framework to a setting under which ML return forecast strategies most often fail. As identified by the literature, these are as follows: the investable universe is restricted to S&P 500 constituents (low limits-to-arbitrage), portfolios are long-only (all limits to short-selling and their costs can be ignored (Muravyev et al. 2025)) and fully invested (the portfolio cannot lever the market factor), trading is monthly from 2004–2024 (reduce concerns about post-publication decay (McLean & Pontiff 2016)), volatility is benchmarked against the S&P 500 (with mean–variance as a robustness check) and concentration limits constrain single-name exposures. Moreover, I use the data from Jensen et al. (2023) which is also used in Jensen et al. (2022). Hence, there are no “easy” channels through which forecast strategies can look good but remain untradeable (microcaps, extreme illiquidity, and leverage/shorting frictions). The paper is also aligned with the institutional reality that much capital is deployed in benchmarked, long-only mandates.

In this environment, naive portfolios formed on the ML return predictions, which include the standard rank-based portfolios, do not outperform the index net of even tiny costs (Table 3.2), establishing that *prediction alone is insufficient* and motivating a distinct portfolio construction step.

In contrast, when the exact same return forecasts are passed through my predict-then-optimize framework, for every return predictor the performance improves substantially (Table 3.5). Importantly, the improvement necessarily arises from the portfolio construction stage as the return forecasts themselves are unchanged so that this channel is perfectly identified. The best portfolio, which uses IPCA return forecasts, earns an annualised mean return of 14.2% versus 11.0% for the S&P 500, with essentially benchmark-level volatility (15.8% versus 14.7%), a Sharpe ratio that is 25% higher than that of the S&P 500, an information ratio of 0.512, and a statistically significant positive annualised abnormal return (alpha) of 2.2% even when accounting for the most common factors such as FF5, Pastor-Stambaugh Liquidity and various Momentum factors, all while maintaining a turnover of only 5.4% per month (Table 3.4). For comparison, the turnovers in Gu et al. (2020) are at least 10 times and many times 20 times higher.

The VoT also holds under alternative risk objectives (such as mean-variance) and strict concentration limits of 10%. Moreover, performance is not subsumed by standard factors such as Fama & French (2015), Momentum and Liquidity (Pastor & Stambaugh 2003). Placebo tests that destroy the information content of forecasts while holding the entire predict-then-optimize pipeline fixed show that performance collapses toward benchmark-like outcomes when return forecasts are randomly permuted, indicating that the portfolio optimiser does not mechanically manufacture alpha and the machine learning return forecasts indeed contain (a little) signal.

On top of that, the calibration of *large* transaction costs is straightforward, economically grounded and works completely out-of-sample. No cross-validation or other such method is required. Since large transaction costs are incurred when the wealth level in the portfolio optimisation is large, I calibrate this large wealth level as the midpoint of the 97th and 98th percentile of the AUM taken from SEC's 13F filings⁴ from December 2003. From then onwards, the wealth level evolves exogenously according to the market return and requires no further attention whatsoever. The VoT is not sensitive to this calibration. For robustness, I use a wealth level half the size and achieve completely analogous results.

The documented performance of the best portfolio in this paper also reflects what is economically viable. Dello Preite et al. (2025) show within an Arbitrage Pricing Theory framework that the wedge between expected returns and compensation for systematic risk ("alpha") may actually represent compensation for unsystematic (diversifiable) risk when investors hold underdiversified portfolios. This applies to the portfolios constructed in this paper. Importantly, however, asymptotic no-arbitrage restricts the Sharpe ratio attainable from portfolios exposed purely to such unsystematic shocks, implying that any resulting return differentials must be economically small. Whether or not this outperformance reflects mispricing on behalf of the market is a different research question. The key question answered in this paper is how to extract the limited signal in return predictions and turn it into economically viable portfolio allocations.

Altogether, this paper makes three main contributions. First, it reframes the ML implementability debate by locating the primary bottleneck not in the existence of transaction costs or limits-

⁴These filings contain the AUM of institutional investors in the US. This data is widely used in demand-based asset pricing (Koijen & Yogo 2019).

to-arbitrage, but in the absence of an economically disciplined forecast-to-trade mapping in low signal-to-noise environments. Second, it provides an implementation filter in which the strength of trade regularisation is economically identified by observable quantities and can be calibrated fully out of sample without re-tuning, cross-validation, or any ad-hoc measures. This implementation filter is easily adaptable and can be applied to a wide range of settings and accommodate any machine learning predictor. Third, it introduces an empirical identification strategy: the Act-as versus Pay-as decomposition. This distinguishes the regularisation role of transaction costs from their mechanical drag on realised return, allowing one to test whether cost-aware optimisation improves the *quality* of executed active risk rather than merely reducing accounting costs.

This paper is related to [Jensen et al. \(2022\)](#) and [Gao et al. \(2026\)](#). Both papers emphasise that return prediction and portfolio construction should be aligned. Their solution is to integrate the two stages into a single estimation problem in which machine learning directly learns a portfolio policy. Hence, their argument is that the return prediction must be re-designed. In [Jensen et al. \(2022\)](#), the Portfolio-ML approach estimates portfolio weights through a Random Fourier Feature expansion to maximise a mean–variance objective net of trading costs. Similarly, [Gao et al. \(2026\)](#) develop an end-to-end learning framework in which the return prediction model is trained jointly with the portfolio optimisation problem. In both cases, the prediction model becomes *decision-aware*: return forecasts are shaped by the portfolio objective rather than estimated independently. In short, both papers greatly modify the return prediction step and their return predictions are not at all comparable to those in the literature such as in [Gu et al. \(2020\)](#).

In contrast, this paper uses a modular *predict–then–optimise* architecture. This modularity (i) allows the optimiser to ingest return forecasts from *any* predictive architecture without any re-engineering, (ii) accommodate any institutional constraints, and (iii) yields a clean attribution of value creation to the implementation filter. In other words, my argument is that the return prediction need not be modified. Instead, a mapping of forecasts to trades (filter) is required. Therefore, my paper is directly comparable to the large literature documenting the predictive performance of ML return predictions which which is not the case for [Jensen et al. \(2022\)](#) and [Gao et al. \(2026\)](#) since they both re-design the ML return prediction stage. As a result, my contribution is *not* a new type of return predictor as in [Jensen et al. \(2022\)](#) and [Gao et al. \(2026\)](#). Instead, my contribution is the forecast-to-trade mapping (implementation filter). This enables the finding that the same ML return forecasts from the literature, which fail under stringent portfolio rules and transaction costs, generate substantial outperformance once passed through my transaction-cost-aware implementation filter. Lastly, my *predict–then–optimise* architecture is highly flexible and easy to implement. [Jensen et al. \(2022\)](#) require Random Fourier Features for their ML return predictions. The end-to-end learning framework in [Gao et al. \(2026\)](#) is computational demanding, uses roughly six million stock-day observations which is much higher than my roughly 720,000 stock-month observations and evaluated on the Chinese stock market.

The rest of the paper is organised as follows. Section 3.2 formalises the portfolio choice problem that maps return forecasts into benchmarked, long-only portfolio weights in the presence

of quadratic price-impact transaction costs and inequality constraints, and derives properties that clarify how forecast dispersion and liquidity govern trading aggressiveness. Section 3.3 describes how stock returns are forecasted. Section 3.4 describes the data, the investable universe, and the feature engineering and screening steps. Section 3.5 presents the empirical results. It first establishes that naive forecast-to-trade rules fail in the S&P 500 universe net of even tiny costs, and then evaluates the predict-then-optimize strategies under matched *Act as* and *Pay as* transaction-cost regimes. It then isolates the regularisation channel via the *Act-as versus Pay-as* decomposition that identifies the Virtue of Transaction Costs (VoT), and subjects the mechanism to robustness checks (alternative risk objectives, concentration limits) and placebo tests based on forecast permutations. Additional results discuss factor exposures and abnormal performance relative to standard asset-pricing factors. Section 3.6 concludes and outlines directions for future research.

3.2 Portfolio Choice Problem

This section lays out the portfolio choice problem that links return predictions to implementable trading strategies. I formalise the manager's decision problem in a long-only S&P 500 universe with realistic transaction costs and risk constraints. The section is structured to (i) define the economic environment and trading frictions, (ii) state the optimisation problem, (iii) derive key properties that govern how forecasts translate into trades, and (iv) explain how transaction costs act as an economically grounded regularisation device. These elements jointly underpin the paper's central mechanism and empirical results. The basic building blocks are taken from Jensen et al. (2022).

3.2.1 Basics

Here, I introduce the economic and institutional primitives of the portfolio choice problem. I define the information structure, the evolution of assets under management (AUM), portfolio weights, trading volume, transaction costs, and net returns. The goal is to make explicit how forecasts, portfolio rebalancing, and wealth-dependent trading frictions interact mechanically.

Filtration. Time is discrete. Let $\{\mathcal{F}_t\}_{t \in \mathbb{N}}$ denote the underlying filtration which captures the known information at the *end* of each month. Information arrives only at the end of the month⁵ as the dataset from Jensen et al. (2023) consists of end-of-month observations. In each month, the investable universe consists of $N_t \in \mathbb{N}$ assets⁶. Decisions are made at $t-$, which is the first (trading day) of month t , based on the information contained in $\mathcal{F}_{t-1}$.

⁵Technically, information arrives at the end of the last trading day of each month. For simplicity, the dataset always reports the last calendar date of the month.

⁶Therefore, if a stock ceases to be in the S&P 500, the manager must sell the entire holding at once. This can cause a non-negligible amount of transaction costs and could be smoothed out which I chose not to do in order to avoid postulating a respective smoothing algorithm.

AUM. Let $w_{t-} \in \mathbb{R}_+$ denote the manager's wealth (AUM) at the *beginning* of month t when trading occurs. For tractability and comparability, as in [Jensen et al. \(2022\)](#) the manager's AUM is assumed to grow exogenously at rate $g_t^w \in \mathbb{R}$, which is equal to the market return from the beginning to the end of month t . Thus, AUM increases (decreases) following positive (negative) market returns. No such assumption is necessary and, for instance, the manager could be self-financing. However, the exogenous growth rate aides in comparing different algorithms since the impact of trading on the AUM, which affects the transaction costs, can be neglected.

Portfolio Decision. At the *beginning* of each month t , the manager chooses a vector of portfolio weights π_{t-} . Moreover, he is forced to allocate the entire AUM among the active S&P 500 members (investable universe) and is not allowed to engage in short-selling. Although short-selling is immediately implementable, this assumption reflects typical behaviour of large institutional investors ([Koijen & Yogo 2019](#)). Moreover, it eliminates the need to be aware of any intricacies involved in short-selling such as limits to supply or market-dependent fee spikes. On top of that, [Section 3.5.1](#) shows that the ML algorithms I consider find the gains in the long side of the S&P 500.

The dollar amount allocated to each asset at the beginning of period t is given by $\pi_{t-}^{\$} \in \mathbb{R}_+^{N_t}$. The manager's vector of portfolio weights is characterised by the unit simplex

$$\pi_{t-} = \frac{\pi_{t-}^{\$}}{w_{t-}} \quad \text{and} \quad \pi_{t-} \geq 0 \quad \text{and} \quad \mathbf{1}^\top \pi_{t-} = 1.$$

Trading Volume. Let $\tau_{t-} \in \mathbb{R}^{N_t}$ denote the manager's trading volume in dollars, defined as the difference between the dollar position at the beginning of month t and the drifted portfolio carried over from month $t-1$. Moreover, the realised return of asset n over month t is denoted by $r_{n,t}$, and the vector $\mathbf{r}_t \in \mathbb{R}^{N_t}$ collects these returns across all assets. Trading volume τ_{t-} is then given by

$$\tau_{t-} = \pi_{t-}^{\$} - \text{diag}(1 + \mathbf{r}_{t-1})\pi_{t-1}^{\$} = w_{t-}\pi_{t-} - \text{diag}(1 + \mathbf{r}_{t-1})w_{t-1}\pi_{t-1}.$$

Using the exogenous evolution of the manager's AUM, i.e. $w_{t-} = (1 + g_{t-1}^w)w_{t-1}$, τ_{t-} can be written as

$$\tau_{t-} = w_{t-} (\pi_{t-} - \mathbf{G}_{t-1}\pi_{t-1}) \quad \text{with} \quad \mathbf{G}_{t-1} \equiv \text{diag}\left((1 + \mathbf{r}_{t-1})/(1 + g_{t-1}^w)\right).$$

Transaction Costs. Transaction costs arise from the price impact of trading dollar amount τ_{t-} , which is quantified by *Kyle's lambda* ([Kyle 1985](#)). Executing a large order typically requires walking up the supply curve when buying (or down when selling). For example, consider an investor who buys \$10 million of a \$1 stock. Suppose the first 5 million shares execute at \$1.00, the next 3 million at \$1.01, and the remaining 2 million at \$1.02. The volume-weighted average execution price is \$1.007, implying \$70,000 in implementation shortfall relative to the initial quote. Because deeper liquidity reduces how quickly prices move with quantity, price impact is

lower for more liquid stocks and higher for less liquid stocks.

Formally, the transaction cost matrix $\Lambda_{t-} \in \mathbb{R}^{N_t \times N_t}$ maps dollar trading volume τ_{t-} to dollar costs, capturing how asset prices respond to trades and reflecting market liquidity. Following Jensen et al. (2022), Λ_{t-} is assumed to be diagonal, implying that trades in one asset do not affect others⁷. Each diagonal element represents the price impact parameter of asset n , calibrated such that trading 1% of its daily dollar volume moves its price by 0.1% (Frazzini et al. 2018). Transaction costs TC_t are then given by⁸

$$TC_{t-} = \tau_{t-}^\top \Lambda_{t-} \tau_{t-}. \quad (3.2.1)$$

Very importantly, dollar transaction costs are quadratic in the wealth level so that large asset managers are disproportionately impacted. In other words, for higher wealth levels active trades are penalised more, so that the level of wealth acts as a regularisation parameter. For this reason, I consider four different wealth levels which each determine a transaction cost regime due to the direct dependence of the transaction costs on the manager's AUM.

As an example, in 2024 Nvidia roughly had an average daily dollar volume of \$20B. Thus, for an AUM of \$5B, increasing NVDA's position from, e.g. 10% to 11%, incurs the following transaction costs

$$\tau_n \lambda_n \tau_n = \$50M \cdot \frac{0.1}{\$20B} \cdot \$50M = \$12,500, \quad (3.2.2)$$

which is 0.025% of the invested amount.

Portfolio Return. Accounting for transaction costs, for a given portfolio vectors π_{t-} and π_{t-1} , the manager's net return, as a fraction of wealth, is given by

$$r_t^{\pi, net}(\pi_{t-}) = \mathbf{r}_t^\top \pi_{t-} - w_{t-}(\pi_{t-} - \mathbf{G}_{t-1} \pi_{t-1})^\top \Lambda_{t-}(\pi_{t-} - \mathbf{G}_{t-1} \pi_{t-1}). \quad (3.2.3)$$

Hence, $w_t r_t^{\pi, net}$ denotes the profit in dollars at the end of month t .

3.2.2 Optimisation Problem

Here I formulate the manager's portfolio optimisation problem. The objective emphasises *investability*: benchmark-like risk, controlled turnover, and realistic trading constraints. The manager chooses portfolio weights myopically, trading off expected returns against transaction costs while respecting long-only and full-investment constraints. This formulation provides the decision rule that maps machine learning return forecasts into actual trades.

Rather than imposing a mean–variance objective as the baseline⁹, I benchmark *risk* directly. Concretely, I impose a *volatility benchmark* constraint that restricts the portfolio's predicted volatility

⁷Given the high liquidity of the S&P 500, this assumption is reasonable.

⁸This is motivated by $Cost = \int_0^\tau \lambda x dx = 0.5 \lambda \tau^2$, since for each additional dollar spent (sold) a new seller (buyer) has to be found who will only sell (buy) at a higher (lower) price.

⁹Jensen et al. (2022) exclusively use mean-variance preferences.

not to exceed the volatility of the S&P 500. That is, the optimiser is free to choose the cross-section of holdings, but it must do so while keeping total risk at a market-like level. This design ensures that the resulting strategies are comparable to the S&P 500 not only in risk-adjusted terms (Sharpe/IR) but also in the *absolute* level of risk. For the benchmark I use the total return of the SPDR S&P 500 ETF Trust (dividends reinvested). As a robustness check, I also report results under a standard mean–variance formulation.

Volatility benchmarking has a further practical benefit for this paper: it allows the optimiser to shift composition across stocks while keeping total risk fixed, so that the primary remaining trade-off is between expected return and transaction costs. This makes it easier to isolate the role of transaction costs as a discipline on rebalancing rather than as a mechanical drag on risk-taking.

I further impose that the manager is myopic. Else-wise, the portfolio choice problem would constitute a dynamic programming problem for which the underlying dynamics of the state variables are entirely unknown¹⁰. It is possible to circumvent the imposition of state dynamics by using multi-period forecasts. While for stock returns this is straightforward from a purely mechanical viewpoint, forecasting future realisations of objects such as the covariance matrix of stock returns is more difficult. Moreover, multi-period forecasts in low frequency data can lead to substantial portfolio misallocation due to widening forecasting errors. Another alternative would be to use model-free reinforcement learning which is however computationally intensive and the underlying policy function (action) is more difficult to interpret. Due to the drawbacks of the alternatives, I adopt the *myopic* setting in which the manager optimises the portfolio solely based on the expected performance in the next period.

On top of that, in a further robustness check I impose a concentration limit $\pi_{\max}$. The reason is that investors may be adverse to a manager that may incur too high exposures with respect to a set of stock(s)¹¹. Hence, a concentration limit improves the investability of the portfolio.

Thus, at each beginning of the month $t-$, the manager solves the following problem

$$\begin{aligned} \max_{\pi_{t-}} \quad & \left\{ \mathbb{E}[\mathbf{r}_t \mid \mathcal{F}_{t-1}]^\top \pi_{t-} - w_{t-} (\pi_{t-} - \mathbf{G}_{t-1} \pi_{t-1})^\top \mathbb{E}[\boldsymbol{\Lambda}_{t-} \mid \mathcal{F}_{t-1}] (\pi_{t-} - \mathbf{G}_{t-1} \pi_{t-1}) \right\} \\ \text{s.t.} \quad & 0 \leq \pi_{t-} \leq \pi_{\max} \quad (\text{Long-Only \& Concentration Limit}), \\ & \mathbf{1}^\top \pi_{t-} = 1 \quad (\text{Fully invested}), \\ & \sqrt{\pi_{t-}^\top \mathbb{E}[\boldsymbol{\Sigma}_t \mid \mathcal{F}_{t-1}] \pi_{t-}} \leq \sigma_t^B \quad (\text{Volatility Benchmarking}). \end{aligned} \quad (3.2.4)$$

where σ_t^B is the volatility of the benchmark's returns estimated by their standard deviation using an exponentially weighted moving average. Fully invested means that there is no outside asset.¹²

¹⁰For example, this would necessitate a model for how the stock characteristics for each individual stock change over time.

¹¹Without a concentration limit, this indeed happens. For example, a portfolio weight may reach up to 30% which is five to six times higher than the largest weight in the S&P 500.

¹²It is, of course, immediate to implement an outside asset.

Notice that the optimiser will not blindly follow high liquidity stocks (low λ) since high liquidity stocks will be sold if their predicted returns are lower than that of other S&P 500 stocks.

Moreover, under zero transaction costs and absent the concentration limit and the volatility benchmark, the portfolio choice problem collapses into a rank-based long only portfolio as typically considered in the literature. For example, the manager could always buy the 10% stocks (decile) with the highest predicted returns in order to maintain an adequate variance of the portfolio. In other words, the portfolio optimisation problem (3.2.4) nests standard portfolio formations as done in the literature.

3.2.3 Properties

Here, I derive key analytical properties of the portfolio choice problem that clarify its economic mechanics. The results establish (i) level independence of predicted returns in the absence of an outside asset, (ii) scale dependence of forecasts as the driver of trading aggressiveness and turnover, and (iii) how these properties interact with transaction costs. These insights explain why different machine learning models generate different trading behaviour even when returns are similarly ranked across predictors.

Level Independence. Let $\hat{\mu} \in \mathbb{R}^N$ denote the vector of predicted (excess) returns available for some trading month.

It is convenient to decompose $\hat{\mu}$ into a *cross-sectional level* and a *demeaned component*:

$$\hat{\mu} = sz + l\mathbf{1} \quad \text{where} \quad l = \frac{\mathbf{1}^\top \hat{\mu}}{N} \quad \text{and} \quad s = \sqrt{\frac{(\hat{\mu} - l\mathbf{1})^\top (\hat{\mu} - l\mathbf{1})}{N}}.$$

Thus, l is the level (average) and s is the scale (standard deviation) of vector $\hat{\mu}$. Therefore, z is a z -score (mean zero, standard deviation one).

Focus on the revenue term in objective (3.2.4). For any feasible portfolio weight vector π satisfying $\mathbf{1}^\top \pi = 1$,

$$\hat{\mu}^\top \pi = (sz + l\mathbf{1})^\top \pi = sz^\top \pi + l \quad (3.2.5)$$

Therefore, the level l changes the objective function exactly by this constant level which does *not* affect the argmax π . Therefore, no machine learning algorithm has to get the level of the returns correct, which is nothing else than the market component. This makes sense since the portfolio is long-only and fully invested in the market, so that it is impossible to hedge this market component.

The irrelevance of the level l hinges critically on the absence of an outside asset and on the full-investment constraint $\mathbf{1}^\top \pi = 1$. Once an outside asset is introduced, this argument no longer applies.

Suppose the investor can additionally allocate wealth to an outside asset (e.g. cash or a risk-free

asset) with portfolio weight π_0 and predicted excess return μ_0 . It then holds that

$$\pi_0 + \mathbf{1}^\top \boldsymbol{\pi} = 1,$$

so that $\mathbf{1}^\top \boldsymbol{\pi}$ is no longer fixed at one and instead is a choice variable capturing the total exposure to risky assets.

The revenue term in the objective function now reads

$$\begin{aligned} \mu_0 \pi_0 + \hat{\boldsymbol{\mu}}^\top \boldsymbol{\pi} &= \mu_0 (1 - \mathbf{1}^\top \boldsymbol{\pi}) + (s\mathbf{z} + l\mathbf{1})^\top \boldsymbol{\pi} \\ &= \mu_0 + s\mathbf{z}^\top \boldsymbol{\pi} + (l - \mu_0) \mathbf{1}^\top \boldsymbol{\pi}. \end{aligned}$$

Since $\mathbf{1}^\top \boldsymbol{\pi} \neq 1$, the level l matters. This also makes sense because if the manager can invest into the riskless asset, he would be more inclined to do so if he predicts a recession in which potentially all assets in the S&P 500 have a negative return.

Scale Dependence. From (3.2.5) it is immediately clear that the scale s influences the argmax since it scales the part of the revenue that depends on $\boldsymbol{\pi}$. Therefore, scaling the forecast dispersion changes the relative weight of expected returns versus transaction costs in the objective. The higher the scale, the less the optimiser looks at the transaction costs and is more focused on revenue maximisation which will lead to more aggressive trades, i.e. $\Delta\boldsymbol{\pi}$ increases. In contrast, the lower the scale, the more focused the optimiser is on transaction costs minimisation which implies that the optimiser will change the portfolio weights less.

As an example, consider a simple two-asset case. Suppose there are three different ML algorithms (A, B and C) which forecast the following returns

$$\text{A: } (r_{1,t} = -50\%, r_{2,t} = +50\%)$$

$$\text{B: } (r_{1,t} = -1\%, r_{2,t} = +1\%)$$

$$\text{C: } (r_{1,t} = +99\%, r_{2,t} = +101\%).$$

In all three scenarios, the ranking of assets is identical and hence a rank-based long–short portfolio would generate the same portfolio allocation. In contrast, under portfolio choice problem (3.2.4), the prediction of ML algorithm A will entail aggressive selling and buying, whereas ML algorithm B and C will entail the same portfolio (due to being identical up to the level) and engage in doveish selling and buying due to the low variance of predicted returns. Hence, larger dispersion in $\mathbb{E}[r_t]$ weakens the relative importance of transaction costs and leads to more aggressive rebalancing, whereas tighter clustering of return forecasts induces smaller trades. This is one of the key mechanisms of the double regularisation channel.

3.2.4 Transaction Costs

This subsection explains the central role of transaction costs in the portfolio choice problem. Transaction costs enter both mechanically—by reducing realised returns—and behaviourally—by

disciplining active trades ex ante. I show that price-impact-based transaction costs act as a stock-specific, easy to calibrate out-of-sample regularisation term that shrinks noisy forecasts without introducing ad hoc tuning parameters. The subsection also introduces distinct transaction cost regimes and the separation between optimisation and execution wealth, which is crucial for identifying the *Virtue of Transaction Costs*.

Transaction Cost as Regularisation. Since it is assumed that Kyle's lambda is a diagonal matrix, the transaction costs can be written as

$$TC_{t-} = \sum_{i=1}^{N_t} (\lambda_{n,t-} w_{t-}) \times (\pi_{t-} - \mathbf{G}_{t-1} \pi_{t-1})^2, \quad (3.2.6)$$

where $(\lambda_{n,t-} w_{t-}) \equiv \gamma_{n,t-}$ is a stock-specific and period-specific penalty parameter on the active trades. The wealth level can be easily calibrated from the SEC's 13F data prior to when trading starts, so that penalty parameter $\gamma_{n,t-}$ can be easily set. See further below.

A key advantage of disciplining active trades through transaction costs rather than through an abstract turnover penalty is that the regularisation strength is economically identified and can be immediately calibrated fully out of sample. This calibration requires no cross-validation, no re-estimation, and no ad hoc re-tuning of a free parameter since the penalty on active trades in is pinned down by observable market liquidity.

Transaction costs on their own are not "virtue" because they mechanically reduce net returns (see further below). They are a virtue because they provide a natural and easy calibration. Moreover, if a manager faces more transaction costs than just the price impact, the joint set of transaction costs can act as a regularisation by directly penalising costly trades and therefore giving a natural definition of what an aggressive trade is. This channel is emphasised throughout the paper: transaction-cost-aware optimisation improves the quality of the trades that are actually executed, not merely the accounting of the costs that are ultimately incurred.

Transaction Costs Regimes. Recall that the transaction costs scale with the manager's wealth. Therefore, the manager's AUM determines the degree of transaction costs. While AUM's growth rate, and the thus its entire trajectory, is exogenously fixed, the level remains to be calibrated. The wealth levels are calibrated without a look-ahead bias. I use the midpoint of the 97% and 98% percentile of assets under management from the SEC's 13F filings from December 2003, which is roughly \$38B, as the largest wealth level. The next wealth levels are 50%, 10% and 1% of that. For ease of interpretation, I list these in the realised at the most recent date of the sample below

$$\begin{aligned} w_{Dec,2024} &= \$500B \text{ (Large TC Regime)} \\ w_{Dec,2024} &= \$250B \text{ (Medium TC Regime)} \\ w_{Dec,2024} &= \$ 50B \text{ (Small TC Regime)} \\ w_{Dec,2024} &= \$ 5B \text{ (Tiny TC Regime)}. \end{aligned} \quad (3.2.7)$$

As a comparison, the wealth level at the end of the sample used by Jensen et al. (2022) is \$10 Billion and the SPDR S&P 500 ETF Trust had approximately \$700 Billion AUM in 2024 which is publically disclosed by the provider. The tiny TC regime is a sufficient floor as the difference between gross returns and returns net of transaction costs becomes negligible.

Optimisation vs. Execution Wealth Level. A central contribution of this paper is to separate two conceptually distinct roles of transaction costs: (i) the *mechanical* reduction in realised returns from paying costs at execution, and (ii) the *behavioural/regularisation* role of TC in disciplining portfolio rebalancing *ex ante* by shrinking active trades. To isolate channel (ii), I allow the wealth level that scales the TC penalty inside the optimiser (3.2.4) to differ from the wealth level that scales the costs actually deducted from realised returns (3.2.3).

Formally, define an *optimisation wealth* w_{t-}^{opt} and an *execution wealth* w_{t-}^{exec} . The portfolio optimiser (3.2.4) uses w_{t-}^{opt} when solving for the optimal portfolio vector $\pi_{t-}(w^{\text{opt}})$. Thus, w_{t-}^{exec} is the wealth level under which the manager acts. In contrast, realised net returns are computed in (3.2.3) by charging transaction costs computed with w_{t-}^{exec} while holding the portfolio weights fixed at $\pi_{t-}(w^{\text{opt}})$. Hence, w_{t-}^{exec} is the wealth level under which the manager pays the transaction costs.

This decomposition yields two empirically distinct designs:

1. **Matched design:** $w^{\text{opt}} = w^{\text{exec}}$. Varying the transaction costs also changes the optimal portfolio allocation.
2. **VoT design:** Fix w^{opt} at a high level ("Act as Large") so that the the portfolio allocation $\pi_{t-}(w^{\text{opt}})$ is held constant, and then vary only w^{exec} ("Pay as") when computing net returns.

3.2.5 Numerical Solution

To solve the portfolio choice problem (3.2.4) numerically, I consider the following penalty method

$$\max_{\mathbf{z}_{t-} \in \mathbb{R}^{N_t}} \left\{ \mathbb{E}[\mathbf{r}_t | \mathcal{F}_{t-1}]^\top \boldsymbol{\pi}_{t-} - w_{t-}^{\text{opt}} (\boldsymbol{\pi}_{t-} - \mathbf{G}_{t-1} \boldsymbol{\pi}_{t-1})^\top \mathbb{E}[\boldsymbol{\Lambda}_{t-} | \mathcal{F}_{t-1}] (\boldsymbol{\pi}_{t-} - \mathbf{G}_{t-1} \boldsymbol{\pi}_{t-1}) - \text{ReLU} \left(\boldsymbol{\pi}_{t-}^\top \mathbb{E}[\boldsymbol{\Sigma}_t | \mathcal{F}_{t-1}] \boldsymbol{\pi}_{t-} - \sigma_t^B \right) - \mathbf{1}^\top \text{ReLU}(\boldsymbol{\pi}_{t-} - \boldsymbol{\pi}_{\max}) \right\}, \quad (3.2.8)$$

with $\boldsymbol{\pi}_{t-} = \text{softmax}(\mathbf{z}_{t-})$. I use gradient ascent which I found to be robust and fast¹³. Note that the penalty method nests a mean-variance framework by simply setting $\sigma_t^B := 0$ and the concentration limit to 100%.

The numerical solution is non-trivial as it is sensitive to the initialisation of the portfolio weights and the floating point accuracy. For stocks that remain in the S&P 500, I set the initial guess of the optimal portfolio as the drifted portfolio weight, i.e. a zero transaction cost initialisation. However, when a stock is newly included in the S&P 500 and thus the investable universe, the

¹³I based this on comparisons in the unrestricted case with a BFGS optimiser with very tight convergence criteria and found virtually no difference in the resulting portfolios.

drifted portfolio weight is zero. If this initial portfolio weight is chosen to be zero¹⁴, this induces the vanishing gradient problem so that the derivative of the objective function with respect to this stock is (essentially) zero¹⁵. In this case, the optimiser will never "discover" these stocks and thus never buy them¹⁶.

Moreover, 64-bit floating point accuracy is highly advised. In 32-bit (single) precision, the machine epsilon is approximately 10^{-7} for values around one. In this case, the calculation of the transaction costs can accumulate rounding errors that are larger than the gradients of the return predictions. Furthermore, because the values of Kyle's Lambda are extremely small (ranging from 10^{-9} to 10^{-13}), the "micro-adjustments" required for optimal trading are often lost in the noise of 32-bit calculations.

Consequently, I use Pytorch's 64-bit floating point accuracy and initialise the portfolio weights of newcomer stocks to 0.1%, which is roughly the lowest weight of a stock in the S&P 500. With this initialisation, algorithms e.g. bought TSLA up to highly non-negligible amounts (roughly 5% at the maximum).

3.3 Forecasts

In this Section I explain how I estimate $\mathbb{E}[\mathbf{r}_t|\mathcal{F}_{t-1}]$, $\mathbb{E}[\boldsymbol{\Sigma}_t|\mathcal{F}_{t-1}]$ and $\mathbb{E}[\boldsymbol{\Lambda}_t|\mathcal{F}_{t-1}]$ in order to compute the objective (3.2.8).

3.3.1 Returns

Forecasting with Characteristics. A central debate in asset pricing concerns *Characteristics versus Covariances*. In the canonical risk-based view of Fama & French (1993), financial markets are rational and efficient. Consequently, any systematic return differential across assets must reflect compensation for differential exposure to risk. For instance, firms with high book-to-market (b/m) ratios are argued to share a common distress risk, which induces co-movement in returns and gives rise to a priced risk factor. In this framework, expected returns vary across assets only through their factor loadings: an asset earns a premium if and only if it exhibits greater exposure to the underlying risk factor. Formally, factor loadings vary across assets as functions of their observable characteristics $\mathbf{z}_{i,t}$, while factor prices of risk vary over time with the cross-sectional distribution of these characteristics. Expected returns therefore take the form

$$\mathbb{E}[r_{i,t+1}|\mathcal{F}_t] = \boldsymbol{\beta}_{i,t}^\top(\mathbf{z}_{i,t})\boldsymbol{\lambda}_t(\mathbf{z}_t), \quad (3.3.1)$$

where both the factor loadings $\boldsymbol{\beta}_{i,t}$ and the prices of risk $\boldsymbol{\lambda}_t$ must be estimated.

An alternative perspective, advanced by Daniel & Titman (1997), attributes return premia directly to characteristics rather than to covariances. In this view, investors may systematically misprice

¹⁴In practice, this must be set to e.g. 10^{-16} as else-wise $\mathbf{z}$ is not defined.

¹⁵This is because the derivative of the softmax function with respect to z_n is $\pi_n(1 - \pi_n)$.

¹⁶I have verified this explicitly. For example, when TSLA enters the S&P 500 in late 2020, the optimiser will never buy TSLA if its weight is initialised to zero.

firms with certain observable attributes. For example, firms that appear distressed (high b/m) may be disliked or neglected, leading to underpricing and subsequently higher expected returns. Crucially, such a firm may earn a premium even if it does not covary with a corresponding factor, that is, even if its factor loading is zero. From this perspective, constructing factors based on return co-movement may obscure the true source of predictability: what matters is whether a stock *has* a given characteristic, not how it covaries with a factor portfolio. Therefore, the characteristics-based specification is given by

$$\mathbb{E}[r_{i,t+1}|\mathcal{F}_t] = f(\mathbf{z}_{i,t}). \quad (3.3.2)$$

It is particularly convenient for return prediction because it obviates the need to construct factor portfolios or to estimate factor realisations and prices of risk. The function $f(\cdot)$ is left unrestricted and can be approximated using any machine learning (ML) method. This approach is standard in the literature when it comes to predicting returns with machine learning algorithms.

Synthesis. A synthesis of the characteristics and covariance views is proposed by [Kelly et al. \(2019\)](#) through Instrumented Principal Component Analysis (IPCA). Unlike standard PCA, which is applied to a static covariance matrix, or Fama–French-style portfolio sorts, which impose fixed factor loadings, IPCA allows factor loadings to vary over time as functions of observable asset characteristics. This framework permits characteristics to matter for expected returns, while restricting them to operate only through their role in shaping exposures to systematic risk factors. Empirically, [Kelly et al. \(2019\)](#) show that characteristics predict returns not because they represent mispricing or anomalies, but because they serve as powerful proxies for time-varying covariances with latent risk factors. In this sense, characteristics are covariances.

In a nutshell, IPCA models the excess return of asset i at time $t + 1$ by

$$r_{i,t+1} = \alpha_{i,t} + \boldsymbol{\beta}_{i,t}^\top \mathbf{f}_{t+1} + \epsilon_{i,t+1},$$

where $\mathbf{f}_{t+1}$ denotes a vector of K latent common factors. Both the conditional factor loadings and the conditional intercepts are specified as linear functions of observable characteristics:

$$\boldsymbol{\beta}_{i,t}^\top = \mathbf{z}_{i,t}^\top \boldsymbol{\Gamma}_\beta \quad \text{and} \quad \alpha_{i,t} = \mathbf{z}_{i,t}^\top \boldsymbol{\Gamma}_\alpha.$$

The matrices $\boldsymbol{\Gamma}_\beta$ and $\boldsymbol{\Gamma}_\alpha$ map characteristics into factor exposures and conditional alphas, respectively. Under this specification, characteristics act as *instruments* for unobservable and time-varying betas. Further details are provided in [Appendix C.4](#).

Return Predictors

To approximate the function $f(\cdot)$ in (3.3.2), I consider the following models:

- Gradient-boosted regression trees (XGBoost, XGB). See [Chen & Guestrin \(2016\)](#) and [Appendix C.3](#).

- Transformer neural networks (TF). See Vaswani et al. (2017) and Appendix C.2.
- Instrumented Principal Component Analysis (IPCA). See Kelly et al. (2019) and Appendix C.4.
- Random Fourier Feature (RFF) Regression. See Rahimi & Recht (2007), Jensen et al. (2022) or Kelly et al. (2023).

I estimate multiple ML models to assess robustness. When only a single model is used, it is difficult to distinguish whether the results reflect a genuine and exploitable pattern in the data or an idiosyncratic artifact of the chosen algorithm. In this sense, the choice of model itself can be viewed as a hyperparameter. These models are selected based on their prominence in the literature and practice. Boosted trees have demonstrated strong performance in Kaggle competitions. Moreover, they have existed since the early 2000s, ensuring the absence of a look-ahead bias. Transformers are included due to their central role in Large Language Models. IPCA is a state-of-the-art endogenous factor pricing model with strong empirical performance in asset pricing applications. Finally, RFF regression is included to facilitate comparability with Jensen et al. (2022).

Hence, for each transaction cost regime (3.2.7), I compute four different portfolios where each portfolio uses exclusively one of the above ML predictions.

On top of that, notice that the returns are predicted completely independently of the portfolio choice problem, unlike in Jensen et al. (2022). In their application, where return prediction and portfolio selection is entangled and the goal is to maximise utility, the ML predictor's feature importance show a clear change when the return prediction is forced to be aware of also having to minimise transaction costs (compare their Figure 5). In contrast, in my application none of this applies and the ML return predictors exclusively focus on the features and relationships that yield the best fit.

Hyperparameter Tuning

Each model entails a distinct set of hyperparameters. I briefly describe the tuning procedures below.

XGB. The most extensive tuning is conducted for XGB, primarily due to its relatively low computational cost. Hyperparameters are selected via Bayesian optimisation (Optuna) to minimise validation mean squared error within a rolling-window framework. The search space includes: learning rate $[0.001, 0.05]$ (log-uniform), maximum depth $\{2, 3, 4\}$, minimum child weight $\{1, \dots, 5\}$, subsample ratio $[0.4, 0.8]$, column subsample ratio $[0.4, 0.8]$, and ℓ_2 regularisation penalty $[10^{-8}, 1]$. Because improvements in validation MSE are small, I restrict the search to 15 trials. All continuous features are cross-sectionally rank-standardised.

Transformer. Transformer hyperparameters are selected via early stopping, effectively implementing a one-dimensional search over the training horizon. The maximum number of epochs is set to 100 with a patience of 20, resulting in 20–60 epochs in practice. The batch size is fixed

at 16 cross-sections, the learning rate at 10^{-3} , and dropout at 10%. The embedding dimension is 64, with four attention heads, a feed-forward dimension of 256, and two transformer blocks. To ensure comparability, refitting uses the same initial weight realisation as in tuning. The context window length is fixed at 500. All continuous inputs are cross-sectionally rank-standardised. To enhance comparability across tuning and refit steps, refitting is performed from the same stored initial weight realisation used during tuning. All of these are additional hyperparameters that I didn't tune. I also pick a fix context window size of 500. From the results it will be clear that the Transformer would have benefitted from much stricter regularisation which I could have implemented with the hyperparameters. Therefore, I make no general statements about the suitability of Transformers for stock price prediction. Lastly, all continuous input features are cross-sectionally rank-standardised.

IPCA. IPCA requires specifying the number of factors. I do not tune this parameter and instead fix it at five, following [Kelly et al. \(2019\)](#). Continuous features are standardised to cross-sectional Z-scores to improve numerical conditioning of the moment matrices.

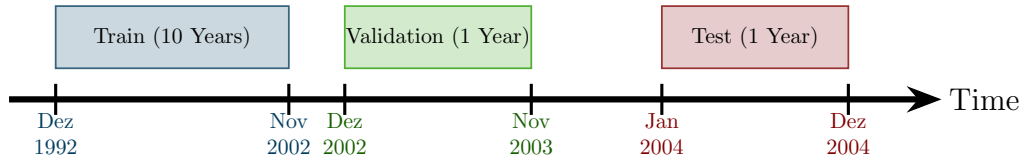
RFF. For RFF regression, I exhaustively search over combinations of the number of features $p \in \{2^6, \dots, 2^{10}\}$, kernel bandwidth $\sigma \in \{0.5, 1, 10, 50\}$, and ridge penalty $\lambda \in \{10^{-3}, \dots, 10^7\}$, selecting the configuration that maximises validation R^2 . Random weights are drawn once and held fixed across all estimation windows. RFFs are constructed only for continuous variables and standardised cross-sectionally. Industry dummies are included without transformation. Since the target variable is the excess market return (with mean approximately zero), no intercept is required.¹⁷ Lastly, all continuous inputs are Z-scores for the same reasons as in IPCA.

Estimation Universe. Each model is trained both on the full CRSP universe and, separately, on active and newly¹⁸ entering S&P 500 constituents. IPCA-based portfolios perform better when trained on CRSP, whereas the other predictors perform better when trained on the S&P 500. This likely reflects the fact that IPCA hyperparameter is taken from [Kelly et al. \(2019\)](#) who calibrated it on CRSP data, and that the larger universe provides additional regularisation for a model with limited built-in shrinkage.

Tuning Schedule. The first out-of-sample forecast is generated for January 2004. Hyperparameters are re-tuned annually, always in January. For 2004, tuning uses data from December 1992 through November 2003, with the final 12 months serving as the validation period. After selecting hyperparameters, the model is re-estimated on the full training sample (11 years) and used to generate forecasts. This procedure is repeated monthly in a rolling-window fashion, so that each return forecast is based on the previous 11 years. Figure 3.1 summarises the timeline.

¹⁷Since Ridge regularisation is used, this regularisation must not operate on the constant, which fixes the level of the target variable. However, since that level is zero, this problem is circumvented.

¹⁸Newly S&P 500 members are known roughly at the middle of the previous month which is sufficient to rule out a look-ahead bias since the return forecasts for end of month t are made at some point between the last trading day and last day of month $t - 1$.

Figure 3.1: Rolling Window

Regularisation of Return Predictors

The predictive power in terms of the R^2 is below 1% and is similar to that found in other studies (e.g. Gu et al. (2020)). However, to better understand the optimal portfolio resulting from each of the four return predictors, it is much more helpful to look at the scale (variance) of the return predictions. Across all cross-sections, the Transformer consistently exhibits, by far, the largest variance in predicted returns. This outcome reflects the Transformer's substantial functional flexibility and its ability to capture complex, non-linear dependencies, which jointly result in highly dispersed cross-sectional forecasts. In general the scale of XGB is the lowest, followed by RFF, IPCA and then the Transformer. The scale of IPCA is substantially higher than that of XGB and RFF, but substantially lower than that of the Transformer. XGB and RFF have generally small scales, but more often than not the scale of XGB is lower than that of RFF by a meaningful margin. In some cases, for XGB this can be extreme and it only predicts four unique values which are tightly clustered together¹⁹. This is the first component in the double regularisation channel, whereas the transaction costs, when building the portfolio, are the second. For this reason, I also say that, e.g., the Transformer is the least regularised predictor.

In summary, the Transformer will trade aggressively, IPCA will trade moderately and both XGB and RFF will trade doveish, whereas XGB is even more cautious about trading than RFF.

3.3.2 Covariance Matrix

The covariance matrix of stock returns, Σ_t , is estimated using a Barra-style factor model. I replicate the exact approach as in Jensen et al. (2022). The procedure begins with a dimensionality reduction of the more than 120 observable firm characteristics. Each characteristic is assigned to one of 13 clusters constructed as in Section 2B of Jensen et al. (2023). Specifically, the pairwise distance between characteristics is defined as one minus their correlation coefficient, and hierarchical clustering is applied to group characteristics with similar time-series behaviour. The resulting clusters correspond to economically interpretable themes: Accruals, Debt Issuance, Investment, Leverage, Low Risk, Momentum, Profit Growth, Profitability, Quality, Seasonality, Size, Short-Term Reversal, and Value. In addition to these 13 characteristic-based clusters, the 12 Fama–French industry classification dummies are included as separate risk factors.

For each month t , factor loadings are constructed in the cross-section. First, each characteristic is transformed into a percentile rank within the month- t cross-section. A stock's loading on a

¹⁹This is the case for 2023 and 2024. I explicitly used low regularisation of XGB throughout the entire sample in order to push up the number of unique predictions for 2023 onwards. This however had no discernible effect on the resulting portfolio performance.

given cluster is then defined as the average of its ranked characteristics belonging to that cluster. Each of the 13 resulting cluster loadings is subsequently standardised cross-sectionally to have mean zero and unit variance, yielding Z-scores. This standardisation implies that each cluster factor is long-short and dollar neutral. The cluster loadings are constructed at the end of month t and held fixed throughout all trading days in month $t + 1$ when estimating factor returns²⁰. Let $\mathbf{S}_t$ denote the resulting $N_t \times (13 + 12)$ matrix of factor loadings, including the industry dummies. Using these time- t loadings, daily factor returns are estimated via cross-sectional regressions. For each trading day d in month $t + 1$, daily stock returns satisfy²¹

$$\mathbf{r}_d = \mathbf{S}_t \hat{\mathbf{f}}_d + \hat{\boldsymbol{\varepsilon}}_d \quad \forall t, d \in \text{month}(t + 1).$$

The estimated coefficients $\hat{\mathbf{f}}_d$ represent daily factor returns, while $\hat{\boldsymbol{\varepsilon}}_d$ are the idiosyncratic residuals. These estimates are used to construct the monthly covariance matrix

$$\boldsymbol{\Sigma}_t = \mathbf{S}_t V(\hat{\mathbf{f}}_t) \mathbf{S}_t' + \text{diag}(V(\hat{\boldsymbol{\varepsilon}}_t)),$$

where daily factor returns and residuals are aggregated into monthly covariance matrices using a 10-year (2,520 trading days) exponentially weighted moving average (EWMA). For covariances, a half-life of 378 trading days is used, while variances are estimated with a half-life of 126 trading days. Because the residual covariance matrix is restricted to be diagonal, dimensionality reduction applies exclusively to the systematic component of return covariances.

Factor Covariance Matrix. The factor covariance matrix $V(\hat{\mathbf{f}}_t)$ is constructed in two steps. First, the factor correlation matrix is estimated, as correlations are numerically more stable, less sensitive to noise, and less prone to generating negative eigenvalues²². Second, the correlation matrix is rescaled using exponentially weighted factor variances. This two-step approach is standard in practice because correlations tend to evolve more smoothly than variances, and estimating them separately reduces the risk of obtaining non-positive-definite covariance matrices²³.

Let $\mathbf{F}_t = \{\mathbf{f}_d\}_{d=t-2519}^t$ denote the $2,520 \times (13 + 12)$ matrix of daily factor returns. Exponential weights are defined as

$$w_j = c \cdot 0.5^{j/378}, \quad j = 1, \dots, 2520$$

where c is a normalising constant ensuring that the weights sum to 1. Denote $\mathbf{w} = (w_1, \dots, w_{2520})'$. For each factor i , i.e. for each column in $\mathbf{F}$, the mean is computed as $\mu_i = \mathbf{w}' \mathbf{F}_t^{[:,i]}$ where the superscript denotes the column. The matrix of de-measured factor returns is then denoted by

²⁰Because the characteristics data are observed at the end of month t , only lagged information is available throughout month $t + 1$.

²¹Because all 12 industry dummies are included, no intercept can be estimated due to perfect multicollinearity.

²²Negative eigenvalues would imply that the covariance matrix is not positive definite.

²³Direct estimation of the covariance matrix indeed resulted in negative variances in some cases.

$F_t^c = F_t - \mathbf{1}\mu$. The exponentially weighted factor covariance matrix is then

$$\hat{\Sigma}_{f,t} = \frac{(\sqrt{\mathbf{w}} \odot F_t^c)^\top (\sqrt{\mathbf{w}} \odot F_t^c)}{1 - \mathbf{w}^\top \mathbf{w}},$$

with the denominator being a degree of freedom correction. The factor correlations are given by

$$\hat{\rho}_t^{[i,j]} = \frac{\hat{\Sigma}_{f,t}^{[i,j]}}{\sigma_i \sigma_j}, \quad \sigma_i = \sqrt{\hat{\Sigma}_{f,t}^{[i,i]}}.$$

For numerical stability, diagonal elements of $\hat{\rho}_t$ are set exactly to one.

Factor variances are estimated analogously using EWMA weights with a half-life of 126 trading days, yielding variances $\tilde{\sigma}_{ii,t}^2$. The final monthly factor covariance matrix is

$$V(\hat{\mathbf{f}}_t) = D_t^{1/2} \hat{\rho}_t D_t^{1/2} \cdot 21, \quad D_t = \text{diag}(\dots, \tilde{\sigma}_{ii,t}^2, \dots),$$

where the factor of 21 converts daily estimates to a monthly frequency.

Idiosyncratic Variance. Idiosyncratic variances are estimated from the daily residuals of the cross-sectional regressions. Let $\{\hat{\varepsilon}_{n,d}\}_{d=t-2519}^t$ denote the sequence of residuals for stock n . The idiosyncratic variance is estimated using a recursive exponentially weighted moving average (EWMA) rather than the usual sample-variance formula because idiosyncratic volatility is time-varying and often exhibits volatility clustering, especially at the daily frequency. The standard sample variance treats all observations as equally informative and implicitly assumes constant volatility, which is inconsistent with empirical return dynamics. The EWMA recursion places more weight on recent residuals and allows the estimated variance to adjust smoothly to changes in market conditions. This improves both responsiveness and numerical stability. In contrast, using the usual sample variance over a long window, such as 252 trading days, would react too slowly to recent volatility spikes and too quickly to temporary noise, and may produce unstable or downward-biased estimates for thinly traded stocks.

Thus, consider the smoothing parameter $\lambda = 0.5^{1/126}$ corresponding to a half-life of 126 trading days. For stocks with sufficient data coverage, the EWMA recursion is $v_{n,d} = \lambda v_{n,d-1} + (1 - \lambda) \hat{\varepsilon}_{n,d-1}^2$, where the initial value is computed as the average of the first 63 valid residual observations (approximately three months of daily data). The daily idiosyncratic volatility estimate is $\sigma_{n,d} = \sqrt{v_{n,d}}$. To obtain a monthly idiosyncratic variance, simply scale up $v_{n,t}$ by 21.

3.3.3 Transaction Costs

Frazzini et al. (2018) estimate that the price impact is 0.1% when trading 1% of the daily dollar volume in a stock, denoted by $V_{n,t}$. This implies that $\lambda_{n,t} \cdot 0.01 V_{n,t} = 0.1\%$ so that

$$\lambda_{t,n} = \frac{0.1}{V_{n,t}}.$$

To forecast $\lambda_{n,t}$, [Frazzini et al. \(2018\)](#) replace $V_{n,t}$ by the average daily dollar volume over the last recent 6 months which is known at the beginning of the trading day.

3.4 Data

Theoretically, expanding the set of predictors (signals) $z_{i,t}$ should monotonically increase the information available to the model. However, financial markets present a unique challenge characterised by an exceptionally low signal-to-noise ratio which is related to the efficient market hypothesis ([Fama 1970](#)). This issue is enhanced in my application since I use monthly return data so that the sample size is limited. Therefore, using too many predictors creates a risk where noise dominates the learning process, masking the subtle relationships that drive returns. Without a rigorous pre-selection of features that possess genuine predictive power, machine learning models are prone to 'learning' noise rather than fundamental market dynamics so that not even a borderline meaningful extraction of information is possible. Therefore, limiting the number of signals is a prerequisite for robust out-of-sample performance. Consequently, the efficacy of ML in finance depends not just on the algorithm's sophistication, but critically on the economic validity and predictive content of the input signals.

Factor Zoo. In light of the "factor zoo" ([Cochrane 2011](#)), there is a vast array of hundreds of predictors to use. See e.g. [Chen & Zimmermann \(2022\)](#) for an overview. In the past decade, there has been an ongoing debate whether or not most of these predictors are a result of data dredging.

For example, by using standard methodology in the literature to identify factors, such as in-sample linear regression, [Novy-Marx \(2014\)](#) finds that variables with no obvious linkage to economics such as the position of the planets are statistically significant predictors of asset prices. For instance, he finds that high levels of solar activity predict the performance of strategies based on price and earnings momentum. In doing so, he emphasises that many factors in the "factor zoo" may be spurious. In other words, when searching through sufficiently many variables, inevitably a statistical *in-sample* significant correlation will be found due to chance, giving rise to many false positives.

[Harvey et al. \(2016\)](#) address the "factor zoo" by introducing a new multiple testing framework to re-evaluate the cross-section of expected returns. They argue that the traditional t-statistic hurdle of 2.0 is no longer appropriate due to extensive data mining and the sheer volume of factors tried, many of which remain unpublished. By applying multiple testing adjustments (such as Bonferroni, Holm, and BHY), they demonstrate that a new factor today should clear a much higher hurdle, typically a t-statistic greater than 3.0, to be considered significant. Their analysis leads to the sobering conclusion that a large proportion of claimed research findings in financial economics are likely false discoveries, with over half of the 296 published "significant" factors failing to meet their recommended thresholds. Hence, they "argue that most claimed research findings in financial economics are likely false".

[Linnainmaa & Roberts \(2018\)](#) provide extensive out-of-sample evidence suggesting that most

accounting-based anomalies are likely artifacts of data-snooping. By gathering comprehensive accounting data extending back to the start of the twentieth century (1918–1970) and merging it with modern databases, the authors examined 36 anomalies across three distinct eras: pre-sample (1926–1969), in-sample (the original discovery period), and post-sample (2005–2016). Their findings reveal a stark contrast between in-sample and out-of-sample performance. When moving either backward or forward in time from the original sample, average returns and Sharpe ratios for most anomalies decrease by 50% to 70%. While every studied anomaly was significant in its original in-sample period, 28 out of 36 earned statistically insignificant average returns during the pre-sample period. The authors show that anomalies are often present only during the precise window of their original discovery; even small perturbations to the in-sample start date can reduce average returns and alphas by 50% to 75%. Combined with McLean & Pontiff (2016), who find that most “anomalies” suffer from post-publication decay, this shows that in-sample testing can be highly sensitive to the underlying sample the hypotheses are tested on.

Hou et al. (2020) conduct a large-scale replication of 452 anomaly variables. They argue that many original findings were likely artifacts of overweighting microcaps²⁴, which represent 60.7% of the total number of stocks but only 3.2% of aggregate market capitalisation. By mitigating the influence of these microcaps using NYSE breakpoints and value-weighted returns, the authors find that 65% of the anomalies fail to clear the standard single test hurdle of the absolute t-statistic being greater than 1.96. When applying the more stringent multiple testing hurdle of 2.78 at the 5% significance level, the failure rate rises to 82%. The trading frictions literature is identified as the “biggest casualty” of this replication, with 96% of its anomalies failing to hold up. Furthermore, even for the anomalies that are successfully replicated, their economic magnitudes are significantly smaller than originally reported, leading the authors to conclude that capital markets are more efficient than previously recognised.

Predictor Set. Despite this information, recent research has also found condensed sets of factors with reasonable predictability. Kozak et al. (2020) shift the focus from individual factor selection to the joint explanatory power of the “factor zoo”. They argue that while many individual anomalies may be the result of data mining, they collectively contain information that characteristics-sparse models—such as the Fama-French three-, five-, or six-factor models—fail to capture.

For my application, the most important contribution is from Jensen et al. (2023). They challenge the previously laid out “replication crisis” narrative by developing a Bayesian model of factor replication that leads to substantially different conclusions. They argue that the pessimistic results in the literature, particularly those of Hou et al. (2020), stem from a reliance on frequentist multiple testing (MT) adjustments that are “unnecessarily crude” and fail to account for the economic structure of factors. While frequentist adjustments like the Benjamini-Yekutieli (BY) method widen confidence intervals to reduce false discoveries, they do so at a severe cost to statistical power, often missing true positives. The authors’ critique of Hou et al. (2020) centers

²⁴See e.g. also Fama & French (2008).

on several methodological choices. First, they argue that testing raw returns rather than risk-adjusted returns (alphas) is theoretically inconsistent with identifying “anomalies”. For example, the low-beta anomaly predicts a positive alpha but a raw return near zero; thus, the failure of [Hou et al. \(2020\)](#) to replicate it actually supports the underlying betting-against-beta theory. Second, [Jensen et al. \(2023\)](#) find that the [Hou et al. \(2020\)](#) replication rate is artificially lowered by including factors that were never found to be significant in their original studies. By using a longer sample, robust factor construction (e.g., terciles instead of deciles), and testing CAPM alphas, [Jensen et al. \(2023\)](#) increase the baseline replication rate from the 35% reported by [Hou et al. \(2020\)](#) to 82.4%.

In more detail, [Jensen et al. \(2023\)](#) propose a hierarchical Bayesian framework that leverages the joint behaviour of factors. Rather than treating hundreds of factors as independent, they acknowledge that factors are often versions of a few underlying economic ideas. They algorithmically cluster factors into 13 distinct themes (such as Value, Momentum, and Quality) that exhibit high within-theme correlation and low across-theme correlation. Their Bayesian model allows them to “borrow estimation strength” across related factors: if a cluster of factors performs well globally, the model’s conviction in the alpha of an individual factor within that theme is strengthened. The authors demonstrate the economic significance of this approach by showing that an investor would have profited from factors deemed significant by their Bayesian method but discarded by frequentist MT hurdles. These “marginal” factors produce an annualised information ratio of 0.93 in the U.S. and 1.10 globally. Ultimately, they conclude that the “factor zoo” is not a sign of data mining, but a natural result of decentralised researchers incrementally improving our collective understanding of a complex risk-return trade-off. Nonetheless, [Jensen et al. \(2023\)](#) do confirm the post-publication decay.

[Jensen et al. \(2023\)](#) have compiled a dataset of the factors with strong predictability which is the dataset I use. See their Section 2 for extensive details. To avoid any confusion, the dataset contains lagged predictors so that the information for any date t was actually publically known (this is also done in [Chen & Zimmermann \(2022\)](#)).

Final Dataset. In total, I use 128 factors from the dataset compiled by [Jensen et al. \(2023\)](#). These factors have good coverage and are the same as in [Jensen et al. \(2022\)](#). On top of that, I build industry dummy variables based on the Fama-French 12 industry classification, which is also done in [Jensen et al. \(2022\)](#). Monthly U.S. risk-free rates, used to compute Sharpe Ratios, are obtained from the Kenneth French Data Library. The adjusted closing price²⁵ for the SPDR S&P 500 ETF Trust was obtained from Yahoo Finance.

Following standard practice, a sequence of economically motivated filters is applied to remove stocks with insufficient or unreliable information:

1. Market equity (size): Observations with missing market equity are removed. Moreover, following [Hou et al. \(2020\)](#) and [Fama & French \(2008\)](#), microcaps are excluded.

²⁵This adjusts the price by accounting for stock splits and dividends to reflect the adequate economic value.

2. Dollar trading volume: Observations with missing or zero 6-month average dollar volume are removed to ensure meaningful liquidity signals and valid computation of Kyle's λ .
3. SIC industry codes: Observations without valid SIC codes are excluded.
4. Feature completeness: The JKP characteristics coverage varies across firms. To ensure consistent predictor quality, each stock at every date must have at least 40% of the characteristics available (i.e., non-missing). Stocks with insufficient informational coverage are dropped.

After screening, the remaining panel retains the vast majority of aggregate market capitalisation, ensuring that exclusions do not distort the economic composition of the investable universe. Most importantly, for the stocks in the S&P 500, i.e. the investable universe for the manager, almost not a single observation is dropped.

Feature Engineering. Here I detail how I pre-process the features used for the machine learning algorithms. In short, the preprocessing of *both* the feature and target data can have significant impacts on the models' predictive performance.

A central requirement for predictive modelling with financial data is the use of cross-sectional transformations of the input features X . Most commonly, these are Z-scores or rank scores. For example, consider market equity. Due to inflation and economic growth, the nominal market capitalisation of firms tends to drift upwards over time. If a model is trained on a rolling window of 10 years and if the non-standardised nominal market equity is used, then the model will perceive all stocks in the early years of the rolling window as "small" (low market equity) and all observations in the final years as "large" (high market equity). Consequently, the feature loses its cross-sectional discriminatory power (identifying large and small firms) and instead becomes a proxy for the time trend. By standardising market equity in each cross-section, this trend is removed, thereby ensuring that throughout the entire rolling window high values of market equity correspond to large firms.

More broadly, financial markets exhibit multiple forms of non-stationarity beyond simple trends (Cont 2001). One of these is volatility clustering, which is still highly relevant today (Ratliff-Crain et al. 2025). During periods of market stress, the cross-sectional dispersion of features such as volatility, momentum, or liquidity can increase sharply. A value that is extreme in a calm regime may be entirely typical in a turbulent one. Without cross-sectional standardisation, feature magnitudes become incomparable across time, preventing the model from learning a stable relationship between features and targets. In effect, the model cannot reliably identify what constitutes a "high" or "low" realisation of a feature across changing market regimes.

Crucially, standardisation must be performed cross-sectionally rather than pooling the data across a rolling window. Pooled means and standard deviations, μ_{pool} and σ_{pool} , assume stationarity across the entire rolling window which may not hold. One exemplary problem are caused by periods of market stress (in the rolling window) which can inflate the scale of both features and targets so that σ_{pool} is dominated by high-volatility periods. Thereby, pooled standardisation

forces observations in calmer periods onto an artificially compressed scale which undermines comparability across time periods.

The above equally pertains to the target variable (in my case, next period's stock returns). In each cross-section, both the level and the variance of returns reflect the prevailing market environment. During boom periods, returns tend to be high; during crises, they are systematically shifted downward. Without enforcing cross-sectional uniformity of means, returns retain exposure to the market factor, encouraging the model to "predict the macroeconomy" rather than to rank stocks by their relative performance. For instance, if the market experiences a 20% drop in a crisis, all stock returns will be shifted to approximately -20% . To minimise the MSE, a model may then attempt to predict this broad market downturn by searching for features that correlate with the macroeconomic state. Since the data from [Jensen et al. \(2023\)](#) is not suitable for predicting the market level as it doesn't contain macro factors²⁶ that affect all stocks equally, the model may realise that the best way to reduce the huge MSE for that month is to predict a large negative return for every stock. As a result, "market noise" can dominate the loss function, leading the model to fit market-wide movements rather than to learn relative performance (alpha). Similarly, during high-volatility periods, the notion of what constitutes a "large" or "small" return changes. This is important because e.g. the MSE is sensitive to outliers. Enforcing unit cross-sectional variance removes this instability by equalising the scale of returns across time. This also highlights the importance of including a constant term in linear models, as the constant absorbs the market level and stabilises learning.²⁷

Technically, cross-sectional standardisation of the target variable is ideal. Indeed, when predicting the Z-score of returns, the performance of rank-based long-short portfolios is best for the predictors. However, this is not the case with the portfolios solving the portfolio choice problem (3.2.8). The reason is that it is sensitive to the scale of returns, unlike rank-based portfolio formations, and predicting Z-scores removes all information on the scale. I had tried to re-scale the Z-scores by OLS, but the out-of-sample predictions of these re-scaling efforts were bad. Therefore, I estimate the excess market return (value-weighted). In each period it roughly has a level of zero. While its variance is not identical over cross-sections (which hurts predictability), the information contained in the scale dramatically improves the performance of the portfolios. To elaborate on why predicting Z-scores yields bad performance, in high-volatility months return dispersion is greater and therefore the potential gains from active trading are larger. Conversely, in quiet months dispersion shrinks and so do gains from active trading. This information is lost when predicting Z-scores.

As a side note, if I had included the risk-free asset, then the level of returns would be important. For example, $(r_1 = -1\%, r_2 = +1\%)$ and $(r_1 = -99\%, r_2 = 101\%)$ yield the same portfolio without the risk-free asset, but with the risk-free asset $(r_1 = -99\%, r_2 = 101\%)$ would induce heavy selling of the first and heavy buying of the second asset as they are now in relation to the

²⁶See e.g. [Chen et al. \(2024\)](#) for how to process macro data which differs from the cross-sectional approach described in this Section.

²⁷If all features and the target are Z-scored cross-sectionally, the constant is necessarily zero and may be omitted.

risk-free rate. With the risk-free asset, it is probably advantageous to estimate the excess return over the risk-free rate, although this has the issue that this risk-premium is further from zero than the excess market return.

Missing Values. Missing values are imputed with 0. Thus, when using cross-sectional Z-scores, the cross-sectional mean is used and for ranks the cross-sectional median is used for imputation. Note that XGBoost has a built-in routine which does not necessitate the imputation of missing values. This is explained in Appendix [C.3](#).

3.5 Results

Table 3.2 sets a demanding baseline: within a long-only universe of active S&P 500 constituents, simple and widely used benchmark constructions (rank-based decile portfolios) do not outperform the S&P 500 net of even tiny transaction costs. This motivates the paper’s central premise that return prediction and portfolio construction are distinct problems, and that implementable alpha requires an explicit optimisation step that disciplines turnover and risk.

Tables 3.3 and 3.4 then evaluate how transaction costs enter this optimisation. Table 3.3 varies transaction-cost regimes under the natural benchmark case $w^{\text{opt}} = w^{\text{exec}}$, so that the manager both *acts under* and *pays* the same level of transaction costs. While informative about overall implementable performance, this design necessarily conflates two channels: (i) the mechanical drag from transaction costs paid in execution and (ii) the regularisation effect of transaction costs on portfolio choice through shrinkage of active trades. By contrast, Table 3.4 isolates the second channel. It fixes portfolio formation to the same high-discipline rule (optimised under large costs) and varies only the execution-cost scaling when computing net returns. In doing so, it cleanly separates trade-quality improvements induced by transaction-cost regularisation from the purely mechanical effect of paying costs.

The main empirical result emerges from this decomposition: portfolios that are constructed as *if* transaction costs were large, thereby strongly regularising active trades, generate substantial and statistically meaningful benchmark-relative performance, and they do so with low and stable turnover and benchmark-like risk, i.e., under a high standard of investability. Figure 3.2 complements the full-sample averages by showing that, for key predictors, the resulting outperformance is not driven by isolated episodes but accumulates persistently and consistently over the backtest period.

3.5.1 Benchmark Portfolios

Table 3.2 reports stylised long-only benchmark portfolios. These portfolios solve (3.2.4) without transaction costs since in this instance it is optimal to load the portfolio onto a number of the stocks with the highest predicted returns since this controls the portfolio’s variance. The results establish that beating the S&P 500 with stocks from the S&P 500 is non-trivial as none of these benchmark portfolios have an edge over the index. Rank-based long-only portfolios formed from the same ML forecasts that will later be used in the optimiser perform poorly: the top-decile portfolios based on XGB, Transformer, and RFF forecasts all deliver *negative* annualised mean returns (e.g., XGB: -5.6% , Transformer: -14.8% , RFF: -9.7%) and strongly negative information ratios. Even IPCA, while positive in mean return (6.2%), exhibits an information ratio of -0.518 , i.e., it underperforms the benchmark after accounting for benchmark-relative risk.

This poor performance is the ground for the “predict, then optimise” separation: forecast quality and portfolio construction are distinct problems, and the latter must explicitly control for turnover and risk constraints. Hence, Table 3.2 provides a clean baseline: any subsequent out-

performance must come from the optimiser translating weak and noisy forecasts into disciplined trades rather than from simple ranking.

Table 3.2: Net Returns of long-only Portfolios.

Portfolio	μ	σ	SR	IR
1/N	0.118	0.167	0.611	0.126
Momentum 1-0	-0.160	0.175	-1.001	-2.853
Momentum 3-1	-0.068	0.177	-0.470	-1.761
Momentum 6-1	0.019	0.181	0.019	-0.883
Momentum 9-1	0.046	0.183	0.169	-0.579
Momentum 12-1	0.052	0.182	0.204	-0.537
Momentum 12-7	0.007	0.193	-0.043	-1.007
Momentum 60-12	0.129	0.196	0.581	0.182
XGB	-0.056	0.185	-0.385	-1.950
Transformer	-0.148	0.192	-0.846	-2.379
IPCA	0.062	0.187	0.250	-0.518
RFF	-0.097	0.174	-0.644	-2.386
S&P 500	0.110	0.147	0.645	0.000

One over N (1/N) simply is an equal weighted portfolio of all stocks in the S&P 500 rebalanced monthly. All other portfolios are value-weighted and decile portfolios which are rebalanced monthly. The performance of the S&P 500 is given for reference. All quantities are annualised.

3.5.2 Virtue of Transaction Costs

I now turn to the optimal portfolios that solve (3.2.4).

Baseline (Act as = Pay as)

Table 3.3 reports the performance of portfolios that solve (3.2.4) under matched optimisation and execution wealth, i.e., $w^{\text{opt}} = w^{\text{exec}}$ ("Act as" equals "Pay as"). Two robust patterns emerge. The table covers the full sample period from 2004–2024 and provides a detailed breakdown of risk, performance, trading intensity, and downside behaviour. Each block of rows corresponds to a return predictor (XGB, Transformer, IPCA, and RFF). For comparison, the first block reports the same (applicable) statistics for the S&P 500 benchmark.

Table 3.3: Portfolio Net Returns. *Act as* equal to *Pay as*.

Act as	Pay as	μ	σ	Sharpe	TO	IR	MaxD	DCap	α
S&P 500									
•	•	0.110	0.147	0.645	•	0.000	-0.509	1.000	0.000
XGB									
Large	Large	0.123	0.146	0.738	0.049	0.408	-0.492	0.959	0.014**
Med	Med	0.121	0.143	0.738	0.052	0.313	-0.490	0.941	0.014**
Small	Small	0.104	0.142	0.623	0.071	-0.138	-0.500	0.935	0.002
Tiny	Tiny	0.117	0.161	0.628	0.120	0.052	-0.452	0.892	0.015
Transformer									
Large	Large	0.110	0.158	0.599	0.073	-0.030	-0.518	1.071	-0.003
Med	Med	0.104	0.164	0.539	0.089	-0.113	-0.558	1.106	-0.010
Small	Small	0.082	0.174	0.385	0.148	-0.323	-0.631	1.180	-0.030
Tiny	Tiny	0.064	0.198	0.247	0.329	-0.388	-0.650	1.231	-0.053*
IPCA									
Large	Large	0.125	0.158	0.694	0.054	0.227	-0.481	1.024	0.014
Med	Med	0.113	0.161	0.607	0.059	0.020	-0.487	1.025	0.005
Small	Small	0.134	0.177	0.667	0.078	0.197	-0.471	0.940	0.028
Tiny	Tiny	0.132	0.235	0.497	0.136	0.118	-0.440	1.085	0.013
RFF									
Large	Large	0.125	0.151	0.730	0.050	0.377	-0.488	0.983	0.014*
Med	Med	0.131	0.156	0.738	0.056	0.394	-0.489	0.996	0.018*
Small	Small	0.117	0.157	0.644	0.076	0.082	-0.511	1.002	0.006
Tiny	Tiny	0.061	0.174	0.261	0.136	-0.472	-0.579	1.037	-0.045*

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. *Act as* denotes w^{opt} , *Pay as* denotes w^{exec} . The portfolio in (3.2.4) is optimised under w^{opt} , but transaction costs are charged at w^{exec} in (3.2.3).

Let r_t^π denote the strategy's realised *net* and r_t^B denote the S&P 500 monthly return. r_t^f denotes the monthly risk-free rate. The following explains the computation of the columns.

μ . Computed as $\mu = 12 \cdot \bar{r}^\pi$. σ . Computed as $\sigma = \sqrt{12} \cdot \text{sd}(r^\pi)$.

Sharpe ratio (SR). Computes as $\sqrt{12} \cdot (r_t^\pi - r_t^f) / \text{sd}(r_t^\pi - r_t^f)$.

Turnover (TO). Monthly gross turnover is computed as $\text{TO}_t = \sum_{i=1}^{N_t} |\pi_{i,t} - \pi_{i,t-1}^g|$, where $\pi_{i,t-1}^g$ denotes the drifted portfolio weights. The table reports the time-series mean $\overline{\text{TO}}$.

Information ratio (IR). Computed as $\sqrt{12} \cdot (r_t^\pi - r_t^B) / \text{sd}(r_t^\pi - r_t^B)$.

Maximum drawdown (MaxD). Computed from the cumulative wealth index $W_t = \prod_{\tau \leq t} (1 + r_\tau^\pi)$ as $\text{MaxD} = \min_t (\frac{W_t}{\max_{s \leq t} W_s} - 1)$, i.e., the most negative peak-to-trough decline in cumulative net returns.

Downside capture (DCap). *Geometric* downside capture ratio. Let $\mathcal{D} = \{t : r_t^B < 0\}$ be the set of benchmark down months and define geometric mean returns over $\mathcal{D}$ as $\bar{r}_{\text{geo}, \mathcal{D}}^\pi = (\prod_{t \in \mathcal{D}} (1 + r_t^\pi))^{1/|\mathcal{D}|} - 1$. Then $\text{DCap} = \bar{r}_{\text{geo}, \mathcal{D}}^\pi / \bar{r}_{\text{geo}, \mathcal{D}}^B$. Values below one indicate the strategy loses less than the benchmark in down markets.

α . Annualised intercept from the time-series regression of strategy excess returns on benchmark excess returns. Significance stars follow conventional 1%/5%/10% thresholds.

Two robust patterns emerge. *First, lower transaction cost regimes mechanically increase turnover.* This is a logical consequence of the resulting weaker shrinkage in (3.2.6): as costs decline, the penalty on active trades falls and the optimiser responds more aggressively to cross-sectional forecast dispersion. The effect is largest for the Transformer, consistent with its large dispersion in predicted returns: turnover rises from 0.073 (Large) to 0.329 (Tiny), and realised risk increases as well (volatility from 0.158 to 0.198).

Second, lower transaction costs degrade performance. If transaction costs were purely a dead-weight loss, reducing them should weakly improve net outcomes. Table 3.3 rejects this implication strongly.

For the Transformer, performance deteriorates monotonically as costs fall: mean return declines from 11.0% (Large) to 6.4% (Tiny), the Sharpe ratio falls from 0.599 to 0.247, and the information ratio drops from -0.030 to -0.388 . Maximum drawdown worsens from -0.518 to -0.650 . This is the signature of *overtrading*: when the trading penalty is weak, residual forecast noise is converted into costly, volatile turnover and inferior benchmark-relative outcomes. Therefore, even though large transaction costs must be paid, the benefit from reducing overtrading is big enough to yield higher net returns compared to having to pay tiny transaction costs.

For RFF, low transaction costs are particularly destructive: despite negligible realised costs in the tiny transaction cost regime, performance collapses (Sharpe ratio drops from 0.730 to 0.261, information ratio drops from 0.377 to -0.472), while turnover almost triples relative to the large transaction cost regime (from 0.050 to 0.136).

XGB outperforms the benchmark in the large and medium transaction cost regimes, with low turnover (around 5% per month), benchmark-like volatility, and favourable downside capture. IPCA is the only predictor that doesn't feature huge performance losses when moving from large to tiny transaction costs. In fact, under smaller transaction costs IPCA even manages to obtain substantially higher mean return than the S&P 500, but its volatility increases sharply so that its risk-adjusted performance remains roughly constant throughout transaction cost regimes. This is empirical evidence that IPCA is the algorithm that finds the strongest signal in the data since it still manages to convert its return forecasts to meaningful portfolio net returns even when its active trades are only slightly regularised.

The patterns observed in Table 3.3 are difficult to reconcile with a "transaction costs destroy ML alpha" narrative. Instead, they support a different interpretation: when the penalty on active trades is small, the portfolio optimiser converts noisy machine learning return forecast into excessive trades, raising variance, worsening drawdowns, and eroding benchmark-relative performance. In contrast, when the trading penalty is large, the strategies not only exhibit very low turnover, but also better performance despite having to pay much higher transaction costs.

Virtue of Transaction Costs. Table 3.3 bundles two channels when moving across transaction costs regimes:

1. Higher *paid* transaction costs mechanically lower net returns, and

2. Solving the portfolio choice problem (3.2.4) under higher transaction costs (Act as) strengthens the regularisation on active trades and alters the portfolio composition.

The VoT requires to isolate the second component. This is straightforward. First, solve the portfolio choice problem under the large transaction cost regime (strong shrinkage of active trades). Second, compute the net returns ex-post by simply using a different execution cost level (w^{exec}) in (3.2.3). This design answers the key question: *Does acting as if transaction costs are large improve the quality of trades, even when the investor ultimately pays small costs?* The results when isolating the second channel are depicted in Table 3.4.

Table 3.4: Portfolio Net Returns. Always *act as* transaction costs are large.

Act as	Pay as	μ	σ	Sharpe	TO	IR	MaxD	DCap	α
XGB									
Large	Large	0.123	0.146	0.738	0.049	0.408	-0.492	0.959	0.014**
Large	Med	0.127	0.146	0.765	0.049	0.553	-0.491	0.948	0.018***
Large	Small	0.130	0.146	0.787	0.049	0.670	-0.490	0.940	0.021***
Large	Tiny	0.131	0.146	0.792	0.049	0.696	-0.489	0.938	0.022***
Transformer									
Large	Large	0.110	0.158	0.599	0.073	-0.030	-0.518	1.071	-0.003
Large	Med	0.121	0.157	0.672	0.073	0.182	-0.511	1.038	0.008
Large	Small	0.130	0.157	0.730	0.073	0.357	-0.505	1.011	0.018
Large	Tiny	0.132	0.157	0.743	0.073	0.396	-0.504	1.005	0.020*
IPCA									
Large	Large	0.125	0.158	0.694	0.054	0.227	-0.481	1.024	0.014
Large	Med	0.133	0.158	0.749	0.054	0.372	-0.477	0.999	0.022*
Large	Small	0.140	0.158	0.792	0.054	0.487	-0.474	0.978	0.029**
Large	Tiny	0.142	0.158	0.802	0.054	0.512	-0.473	0.973	0.031**
RFF									
Large	Large	0.125	0.151	0.730	0.050	0.377	-0.488	0.983	0.014*
Large	Med	0.130	0.151	0.761	0.050	0.504	-0.486	0.969	0.019**
Large	Small	0.134	0.151	0.786	0.050	0.605	-0.485	0.958	0.023***
Large	Tiny	0.135	0.151	0.792	0.050	0.627	-0.485	0.955	0.024***

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector in (3.2.4). *Pay as* denotes w^{exec} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

Together, Table 3.4 and Table 3.3 paint a clear picture. Relative to the corresponding "Act as equal

to *Pay as*²⁸, acting under large transaction cost yields significant improvements. Sharpe and information ratios increase markedly as execution costs fall while the portfolio allocation is held fix under the large transaction cost regime. For example, relative to their Tiny/Tiny counterparts in Table 3.3, the Large/Tiny VoT portfolios achieve higher Sharpe ratios and turn negative information ratios into strongly positive ones (e.g. Transformer: from -0.388 to 0.396; RFF: from -0.472 to 0.627). These gains persist while maintaining benchmark-like volatility and very low turnovers. Additionally, in the Large/Tiny regime the alphas over the S&P 500 become larger and more statistically reliable as execution costs fall while trade discipline is held fixed (e.g., XGB alpha rises to 0.022***; IPCA to 0.031**; RFF to 0.024***; Transformer becomes marginally positive at 0.020*). This pattern is exactly what one would expect if the large-cost optimiser selects higher-quality trades (less noise), and lower paid costs then allow those trades' gross edge to appear in net performance.

Table 3.5 quantifies the incremental value of VoT discipline by directly comparing Sharpe and information ratios between VoT portfolios (*Act as Large*; *Pay as* = Med/Small/Tiny) and their matched baseline counterparts (*Act as* = *Pay as*). The improvements are economically large and statistically strong.

Table 3.5: Comparison: Sharpe and Information Ratios

Pay as	SR (VoT)	SR	Δ SR	IR (VoT)	IR	Δ IR
XGB						
Med	0.765	0.738	0.028	0.553	0.313	0.240**
Small	0.787	0.623	0.164**	0.670	-0.138	0.808***
Tiny	0.792	0.628	0.164	0.696	0.052	0.644**
Transformer						
Med	0.672	0.539	0.132***	0.182	-0.113	0.296***
Small	0.730	0.385	0.345***	0.357	-0.323	0.679***
Tiny	0.743	0.247	0.496***	0.396	-0.388	0.784***
IPCA						
Med	0.749	0.607	0.142***	0.372	0.020	0.351***
Small	0.792	0.667	0.125**	0.487	0.197	0.290***
Tiny	0.802	0.497	0.304***	0.512	0.118	0.394***
RFF						
Med	0.761	0.738	0.023	0.504	0.394	0.110**
Small	0.786	0.644	0.142**	0.605	0.082	0.523***
Tiny	0.792	0.261	0.531***	0.627	-0.472	1.099***

Notes: This table compares Sharpe and information ratios from Table 3.4 (VoT) and Table 3.3. ***, **, and * denote statistical significance at the 1%, 5%, and 10% levels, respectively.

²⁸Note that the *Act as large* and *Pay as large* case is identical in both Tables.

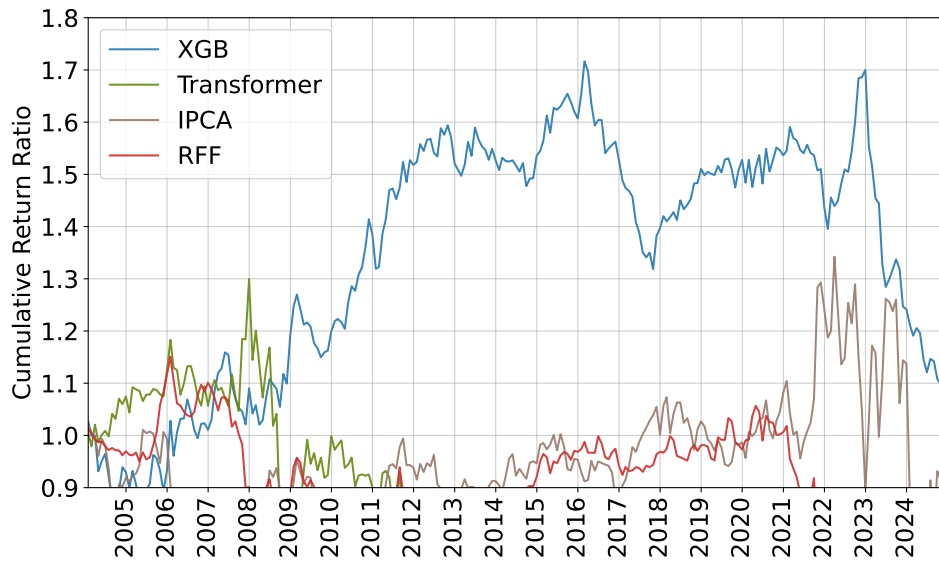
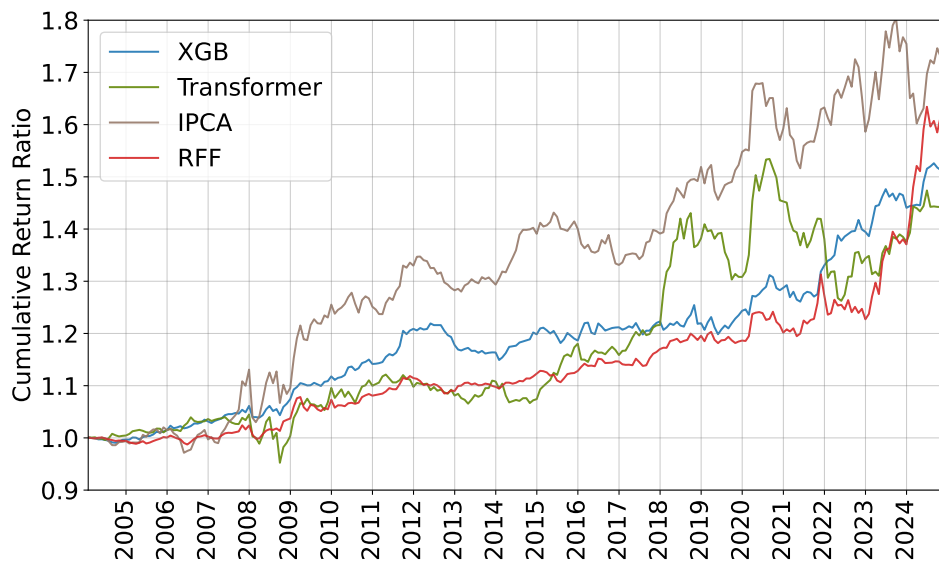
Moreover, two patterns are noteworthy. First, the gains are largest exactly where the baseline suffers from weak discipline: for the Transformer and RFF in the Small and Tiny regimes, ΔSR ranges from 0.345 to 0.531 and ΔIR ranges from 0.679 to 1.099. Second, the information ratio improvements are typically more statistically decisive than Sharpe ratio improvements, consistent with the strategies being benchmark-relative by construction: in a long-only S&P 500 universe, incremental performance is naturally measured as excess return over the benchmark rather than over the risk-free rate.

These results substantially strengthen the case that the VoT is not a fragile artifact of a single predictor. The same regularisation principle improves outcomes across four forecasting architectures, including ones with radically different inductive biases (trees, transformers, latent factor models, and random fourier regression).

A subtle but important nuance concerns the comparison of information ratios across predictors, particularly XGB versus IPCA. Although XGB displays a higher IR than IPCA in several VoT specifications, this does not imply that XGB delivers superior or more reliable benchmark-relative performance. This is also clear when jointly considering the Sharpe and information ratio. In this case, the difference between the Sharpe ratio in the Large/Tiny and Tiny/Tiny regime for XGB is not statistically different from zero, whereas for IPCA this difference is highly significant.

To see why the difference of the information ratio, but not of the Sharpe ratio, is statistically significant, consider Figure 3.2. The Figure depicts the ratio of cumulative strategy returns to cumulative S&P 500 returns. An upward slope indicates sustained outperformance, no slope shows that the strategy performance is the same as that of the S&P 500 and a downward slope shows that the strategy underperforms the S&P 500. In Panel (b), at the end of the sample, IPCA had made 1.7 times the return of the S&P 500. The time series ratio reveals whether apparent outperformance reflects persistent value creation or a front-loaded episode followed by benchmark-like co-movement. Figure 3.2 shows that the reason XGB has a higher IR is mechanical: XGB earns a disproportionate share of its relative gains early in the sample and then largely co-moves with the S&P 500 for long stretches thereafter. As a result, XGB's realised tracking error over the full sample becomes extremely small, so the denominator of the information ratio is close to zero. This inflates the IR, effectively "blowing up" a benchmark-relative statistic that is highly sensitive when active risk is vanishing. By contrast, IPCA's benchmark-relative outperformance is more steadily accumulated through time, producing a more transparent and economically meaningful profile even if its full-sample IR is lower than that of XGB's.

Figure 3.2 also shows that XGB had high predictive power until 2012 since in the Tiny/Tiny regime it displays outstanding performance (1.6 times the cumulative return of the S&P 500 after only 8 years of monthly trading). However, over the entire sample in the Tiny/Tiny regime XGB performs only marginally better than the S&P 500 (1.1 times the cumulative return). This highlights that for short periods an algorithm may work well, but this does not guarantee any form of reliability over longer periods. This further strengthens the case for filtering out the noise in the predicted returns using the VoT.

Figure 3.2: Ratio of Cumulative Strategy Returns relative to the S&P 500**(a)** Act as Tiny, Pay as Tiny (Table 3.3)**(b)** Act as Large, Pay as Tiny (Table 3.4)

A practical implementation critique of ML strategies is that outperformance often comes with extreme turnover, fragility, and poor behaviour in drawdowns. The VoT directly addresses this. Under the VoT, turnover is low and stable across execution regimes. Moreover, the levels of volatility are close to that of the S&P 500. This indicates that cost-based regularisation does not simply “slow down” trading; it selectively removes the trades with the weakest net information content—those that would otherwise amplify noise and contribute to poor tail outcomes.

The economic interpretation is the core of the VoT: transaction costs embedded in (3.2.4) act as a

stock-specific and period-specific ridge penalty on active trades. This penalty suppresses precisely those marginal trades whose expected gains are most likely to be forecast noise, thereby improving the signal-to-noise ratio of *executed* active risk. In this sense, acting as if costs are large is not a pessimistic implementation assumption; it is a disciplined shrinkage device that yields better portfolios.

A false interpretation is that Table 3.4 merely shows that paying fewer costs improves net returns. The relevant counterfactual is not "Large/Tiny versus Large/Large"; it is "Large/Tiny versus Tiny/Tiny." The latter comparison holds execution costs fixed at Tiny while varying only the degree of regularisation on the active trades in (3.2.4). Across predictors, the disciplined Large-rule dominates the undisciplined Tiny-rule despite identical (small) paid costs. The VoT therefore reflects improved *trade selection* and not mechanical cost savings.

Stability over Time. The above Tables report averages across the entire sample (2004–2024). Average performance can be misleading since it may be driven by a short sub-period. Figure 3.2 also examines whether the average gains in Tables 3.3 and 3.4 reflect persistent out-of-sample value creation or are driven by isolated episodes.

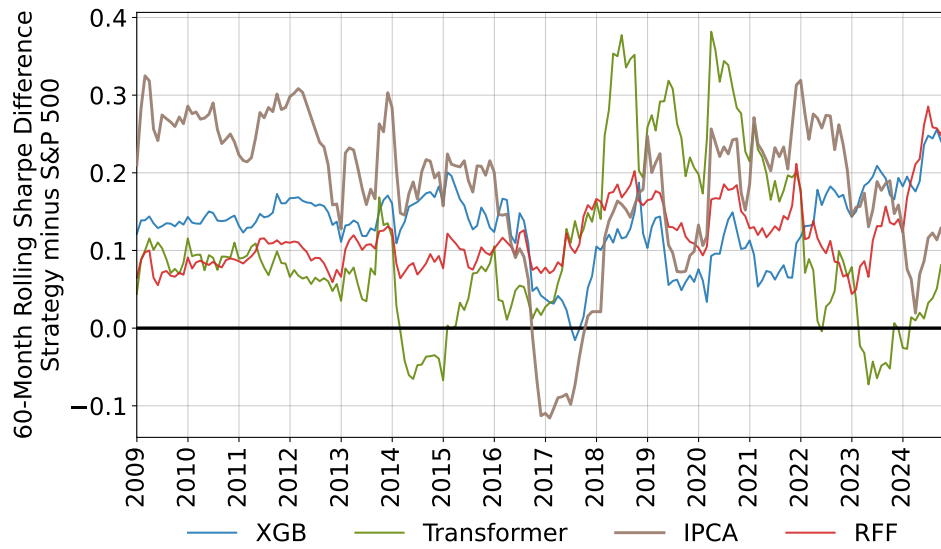
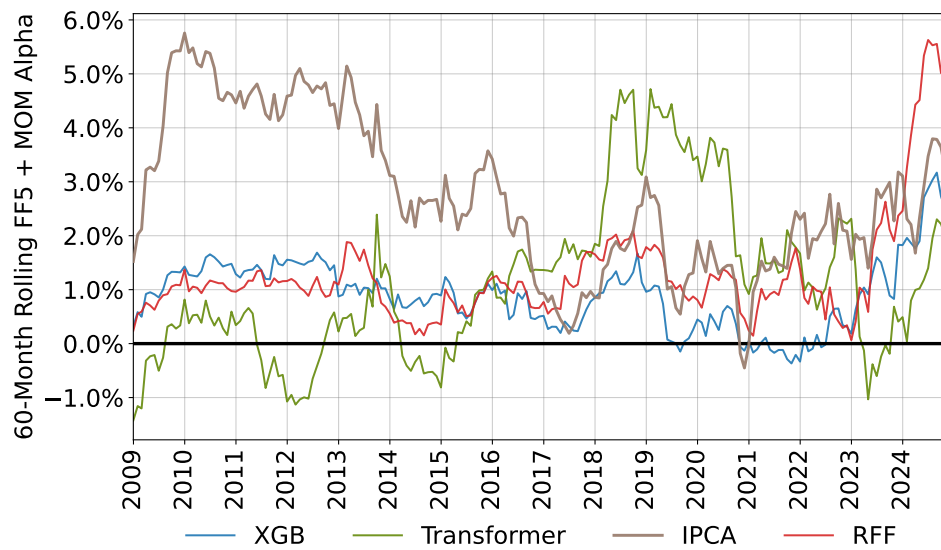
Figure 3.2 Panel (a) (Act as Tiny, Pay as Tiny) illustrates the empirical failure of weak trade regularisation: performance is unstable and frequently mean-reverting. XGB exhibits meaningful early gains, then performs as well as the S&P 500 from 2013 to 2023 and then substantially loses over the S&P 500, thereby basically wiping out all early gains. IPCA and RFF fail to produce a consistently upward trajectory under the low-cost/high-turnover rule, consistent with Table 3.3.

Figure 3.2 Panel (b) (Act as Large, Pay as Tiny) shows a qualitatively different pattern. Under VoT discipline, the cumulative return ratios for IPCA and RFF exhibit a persistent upward drift. Thus, the figure indicates that VoT outperformance is not concentrated, it accrues through multiple market regimes, which is precisely what one would expect if transaction-cost shrinkage is filtering unstable marginal trades rather than exploiting episodic frictions. This time-series evidence therefore underpins the genuine alpha: the VoT portfolios do not merely spike; they compound.

On top of that, Figure 3.3 reports the results from a rolling-window analysis. Panel (a) plots the difference between each strategy's rolling *annualised* Sharpe ratio and that of the S&P 500. IPCA, which is the best-performing strategy overall, exhibits a Sharpe ratio that is predominantly above that of the S&P 500 throughout the sample period, with the main exception occurring during 2017–2018. The magnitude of the Sharpe ratio differential is economically meaningful: for much of the sample, IPCA's rolling Sharpe ratio exceeds that of the S&P 500 by approximately 0.2.

Panel (b) reports rolling *annualised* alphas from a Fama–French five-factor model augmented with the momentum factor²⁹. IPCA's α remains positive throughout most of the sample. Although the estimated alpha declines during 2013–2014 and again during 2017–2018, it subsequently recovers, suggesting that the strategy does not exhibit a persistent deterioration in risk-adjusted performance.

²⁹The full results over the entire sample are captured in Table 3.6.

Figure 3.3: 60-month rolling window results. Act as Large, Pay as Tiny.**(a)** Rolling *annualised* Sharpe Ratio Difference: Strategy minus S&P 500**(b)** Rolling *annualised* α over Fama-French 5 Factor + Momentum.

3.5.3 Factor Regression

Every expected return can be written as a factor model, i.e. $\mathbb{E}[R_i] = \alpha + \beta_{i,m} \cdot \lambda_m$. Further, each factor λ_m can be interpreted as the expected return of a tradable portfolio that isolates a specific source of systematic risk. (see, e.g. [Cochrane \(2001\)](#), Chapter 6).

If the portfolio is constructed such that exposures cancel for all factors except one, say factor k , i.e. $(\beta_{i,m} - \beta_{j,m}) = 0$ for all $m \neq k$, then $\mathbb{E}[R_i - R_j] = (\alpha_i - \alpha_j) + (\beta_{i,k} - \beta_{j,k})\lambda_k$. Under the standard no-arbitrage assumption that alphas are zero for well-diversified portfolios, this implies that the return of such a long-short portfolio isolates the factor premium λ_k . This logic directly motivates the construction of empirical factors as in [Fama & French \(1993\)](#).³⁰

Table 3.6 shows the results when regressing the net return in excess of the risk-free rate of the Large/Tiny VoT portfolios on the Fama–French five factors ([Fama & French 2015](#)), momentum factors ([Carhart 1997](#)), and Pastor–Stambaugh liquidity ([Pastor & Stambaugh 2003](#)). All strategies have market betas close to one, ranging from 0.996 to 1.044, as expected given the long-only S&P 500 constraint. This confirms that outperformance is not achieved by leveraging or materially de-risking the market exposure. Second, SMB loadings are negative for all strategies and statistically significant for XGB, TF, and RFF, consistent with the large-cap investable universe. Third, the remaining factor exposures vary by return predictor. For example, XGB loads positively and significantly on RMW and CMA, while IPCA loads negatively and significantly on HML. This indicates that the ML forecasts translate into economically interpretable but distinct tilts even under the same portfolio objective.

Most importantly, the intercept shows that factor exposures do not fully subsume performance for the strongest strategy. IPCA retains a positive annualised alpha of 2.7%, which is statistically significant at the 5% level. The other strategies have positive but statistically insignificant alphas: 1.1% for XGB, 1.1% for the Transformer, and 1.4% for RFF. Thus, while the alphas remain economically positive, only IPCA delivers residual performance that survives the full factor adjustment in this specification.

Table 3.6 supports the following interpretation: VoT outperformance is not a mechanical proxy for a single anomaly factor, since momentum and liquidity exposures are limited. Where residual alpha remains statistically significant, most notably for IPCA, it survives a demanding factor adjustment.

³⁰The market factor is an exception because it represents the aggregate risky portfolio itself rather than a spread; hence it is implemented as a long-only excess return over the risk-free rate.

Table 3.6: Regression: Net Portfolio Returns in excess of risk-free rate on Factors.

	XGB	TF	IPCA	RFF
Mkt	1.000***	1.044***	0.996***	1.027***
SMB	−0.079***	−0.147***	−0.033	−0.133***
HML	−0.079***	−0.093	−0.220***	−0.113***
RMW	0.190***	−0.093**	−0.022	0.089**
CMA	0.144***	−0.022	0.100	−0.002
UMD	−0.014	0.018	−0.007	0.012
PS Liq	−0.014	0.014	0.015	−0.034
Mom 1-0	0.021*	0.029	0.041	0.006
Mom 3-1	0.016	0.022	−0.002	0.034**
Mom 60-12	0.025	0.073**	0.035	0.008
α_{ann}	0.011	0.011	0.027**	0.014
R^2	0.970	0.929	0.889	0.948

Notes: Reported coefficients are estimated fully in-sample using the monthly data from 2004–2024. ***, **, and * denote statistical significance at the 1%, 5%, and 10% levels, respectively. Standard errors are computed with Newey–West and six lags. The intercept α is annualised.

As an alternative, Table 3.7 reports the same regression coefficients, but with the strategies' net return in excess of the S&P 500 as the dependent variable. The message is analogous. Naturally, the R^2 is much lower, ranging from 0.252 to 0.367, since the market component, which explains most of the portfolio returns in Table 3.6, has been removed. Moreover, the market factor is economically small in this specification, although the Transformer still exhibits a statistically significant market loading of 0.054. The annualised alpha remains positive for all strategies and is statistically significant for XGB, IPCA, and RFF. In particular, IPCA has the largest alpha at 3.1%, significant at the 5% level, while XGB and RFF have significant alphas of 1.5% and 1.8%, respectively. The Transformer's alpha is also positive at 1.5%, but not statistically significant.

Table 3.7: Regression: Net Portfolio Returns in excess of S&P 500 Returns on Factors.

	XGB	TF	IPCA	RFF
Mkt	0.010	0.054**	0.005	0.037
SMB	0.041*	-0.027	0.087	-0.013
HML	-0.086***	-0.100	-0.227***	-0.121***
RMW	0.135***	-0.148***	-0.077	0.034
CMA	0.106**	-0.059	0.062	-0.040
UMD	-0.010	0.022	-0.003	0.016
PS Liq	-0.011	0.016	0.017	-0.031
Mom 1-0	0.026**	0.034	0.046	0.011
Mom 3-1	0.016	0.022	-0.002	0.034**
Mom 60-12	0.026*	0.074**	0.036	0.009
α	0.015**	0.015	0.031**	0.018**
R^2	0.292	0.367	0.252	0.260

Notes: Reported coefficients are estimated fully in-sample using the monthly data from 2004–2024. ***, **, and * denote statistical significance at the 1%, 5%, and 10% levels, respectively. Standard errors are computed with Newey–West and six lags. The intercept α is annualised.

3.5.4 Robustness: VoT not driven by the AUM Calibration

A natural concern is that the Virtue of Transaction Costs (VoT) could be an artifact of how the large transaction cost regime is calibrated. In particular, one might worry that the superior performance of the VoT portfolios in Table 3.4 reflects an extreme choice of the optimisation wealth level w^{opt} (and thus an unusually strong trading penalty), rather than a robust regularisation mechanism. If the VoT is a genuine implementation effect, then similar trade-discipline and performance improvements should obtain under a *moderate* level of ex-ante transaction-cost regularisation.

Table 3.8 addresses this concern by repeating the VoT design and acting under transaction costs being medium high. That is, within each predictor block, the optimiser solves the portfolio choice problem under the medium transaction cost regime and the resulting portfolio rule is held fixed, while execution costs (*Pay as*) vary across Large/Medium/Small/Tiny. This design isolates the same channel as before: it keeps the mapping from forecasts to portfolio weights constant and changes only the costs deducted from realised returns. The key question is whether a *non-extreme* cost discipline continues to deliver the core VoT prediction: improved benchmark-relative performance at low and stable turnover.

Table 3.8: Portfolio Net Returns. Always *act as* transaction costs are medium.

Act as	Pay as	μ	σ	SR	TO	IR	MaxD	DCap	α
XGB									
Med	Large	0.112	0.143	0.677	0.052	0.006	-0.492	0.960	0.005
Med	Med	0.118	0.142	0.721	0.052	0.226	-0.490	0.945	0.011*
Med	Small	0.123	0.142	0.756	0.052	0.405	-0.489	0.933	0.016***
Med	Tiny	0.124	0.142	0.764	0.052	0.446	-0.489	0.930	0.017***
Transformer									
Med	Large	0.075	0.156	0.380	0.091	-0.639	-0.570	1.137	-0.036***
Med	Med	0.099	0.153	0.543	0.091	-0.248	-0.555	1.066	-0.011
Med	Small	0.118	0.151	0.676	0.091	0.124	-0.543	1.011	0.009
Med	Tiny	0.122	0.151	0.706	0.091	0.212	-0.540	0.998	0.013
IPCA									
Med	Large	0.099	0.153	0.545	0.059	-0.225	-0.486	1.074	-0.010
Med	Med	0.116	0.152	0.664	0.059	0.079	-0.479	1.018	0.008
Med	Small	0.130	0.151	0.760	0.059	0.329	-0.474	0.973	0.023*
Med	Tiny	0.133	0.150	0.782	0.059	0.385	-0.473	0.963	0.026**
RFF									
Med	Large	0.114	0.149	0.658	0.055	0.056	-0.491	1.013	0.003
Med	Med	0.121	0.149	0.708	0.055	0.268	-0.489	0.991	0.011
Med	Small	0.127	0.149	0.747	0.055	0.437	-0.487	0.973	0.016**
Med	Tiny	0.128	0.149	0.756	0.055	0.475	-0.486	0.969	0.018**

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector in (3.2.4). *Pay as* denotes w^{exec} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

The results show that this is indeed the case. Across predictors, acting under medium costs yields (i) low and stable turnover, (ii) benchmark-like risk, and (iii) monotonic improvements in risk-adjusted and benchmark-relative performance as paid costs decline.

For example, for XGB the information ratio rises from 0.226 (Med/Med) to 0.446 (Med/Tiny) with turnover fixed at 0.052, and annualised alpha increases from 0.011* to 0.017***. Under IPCA, the Sharpe ratio increases from 0.664 (Med/Med) to 0.782 (Med/Tiny), the information ratio rises from 0.079 to 0.385, and alpha reaches 0.026**, again with turnover held constant at 0.059. RFF exhibits the same pattern, with IR increasing from 0.268 to 0.475 and alpha from 0.011 to 0.018** as *Pay as* falls, while *TO* remains 0.055. Even for the Transformer, which is the most prone to forecast-noise-induced overtrading, imposing medium ex-ante discipline is

sufficient to turn benchmark-relative performance positive once execution costs are small (IR 0.212 at Med/Tiny), with turnover fixed at 0.091 and improved downside behaviour relative to weaker-discipline regimes.

Across all four predictors, the Large/Tiny configuration (Table 3.4) uniformly dominates Med/Tiny in terms of mean return, risk-adjusted performance, and benchmark-relative performance, while also exhibiting lower turnover. Since realised costs are identical in both specifications, any performance differences are entirely driven by the ex-ante trading discipline embedded in the optimisation problem.

However, this dominance is very moderate. For XGB, the Sharpe ratio decreases from 0.792 (Large/Tiny) to 0.764 (Med/Tiny), and the information ratio decreases from 0.696 to 0.446. Annualised alpha declines from 0.022*** to 0.017***. Turnover is basically the same. A similar pattern holds for the Transformer. The Sharpe ratio decreases from 0.743 (Large/Tiny) to 0.706 (Med/Tiny) and the information ratio decreases from 0.396 to 0.212. Turnover declines from 9.1% to 7.3%. Given the large dispersion of the Transformer's predicted returns, this is not surprising. For IPCA, the Sharpe ratio declines from 0.802 (Large/Tiny) to 0.782 (Med/Tiny). The information ratio declines from 0.512 to 0.385. The alpha declines from 0.031** to 0.026**. Notably, turnover remains basically the same. Finally, RFF exhibits the same ordering. The Sharpe ratio decreases from 0.627 to 0.475, and alpha from 0.024*** to 0.018** when moving from Med/Tiny to Large/Tiny. Turnover once again remains basically the same.

Two conclusions follow. First, the VoT mechanism is *not* sensitive to the particular AUM level used to define the large transaction cost regime. A medium level of transaction cost discipline produces qualitatively and quantitatively similar improvements in performance when execution costs are lowered while holding the trading rule fixed. Second, the economic interpretation remains unchanged: transaction costs embedded in the portfolio optimiser function as a stock-specific regularisation term on active rebalancing. Moving from large to medium reduces the strength of that regularisation, but does not eliminate it. Consequently, the central insight survives: the value of transaction-cost-aware portfolio choice does not arise because costs are paid, but because cost-based shrinkage improves trade selection by preventing noisy return forecasts from being mapped into excessive rebalancing. The comparison of Tables 3.4 and 3.8 underpins the VoT since acting as if transaction costs are large yields a better performance than acting as if transaction costs are medium. In short, the VoT is a robust feature of disciplined implementation rather than a knife-edge consequence of an extreme calibration choice. Note that there is an upper limit since acting as if transaction costs are infinitely high induces pure passive investing which has the same return as the S&P 500.

3.5.5 Mean-Variance

A natural concern is that the baseline objective (3.2.4) relies on volatility benchmarking rather than the canonical mean-variance framework used in much of the literature. If the VoT mechanism is genuine, it should not hinge on this modelling choice so that transaction costs should continue to act as an economically grounded regularisation on trades even when risk enters

through a quadratic variance penalty rather than a volatility constraint.

Table 3.9 reports the VoT design (*act as large*) under mean–variance preferences.

Table 3.9: Portfolio Net Returns. *Act as large*. Mean-Variance Preferences.

Act as	Pay as	μ	σ	SR	TO	IR	MaxD	DCap	α
XGB									
Large	Large	0.102	0.124	0.700	0.046	-0.202	-0.456	0.808	0.009
Large	Med	0.106	0.124	0.732	0.046	-0.115	-0.454	0.797	0.013
Large	Small	0.110	0.124	0.758	0.046	-0.045	-0.452	0.787	0.016**
Large	Tiny	0.110	0.124	0.764	0.046	-0.030	-0.452	0.785	0.017**
Transformer									
Large	Large	0.100	0.147	0.578	0.071	-0.249	-0.507	1.014	-0.007
Large	Med	0.111	0.146	0.652	0.071	-0.016	-0.501	0.983	0.004
Large	Small	0.119	0.146	0.712	0.071	0.175	-0.495	0.959	0.013
Large	Tiny	0.121	0.146	0.725	0.071	0.218	-0.494	0.953	0.015
IPCA									
Large	Large	0.113	0.150	0.648	0.053	0.021	-0.479	0.980	0.007
Large	Med	0.122	0.150	0.709	0.053	0.165	-0.475	0.952	0.016
Large	Small	0.129	0.150	0.757	0.053	0.281	-0.471	0.930	0.024*
Large	Tiny	0.130	0.150	0.768	0.053	0.307	-0.471	0.925	0.025*
RFF									
Large	Large	0.110	0.129	0.733	0.047	-0.044	-0.453	0.836	0.013*
Large	Med	0.114	0.129	0.765	0.047	0.056	-0.451	0.823	0.017**
Large	Small	0.117	0.129	0.791	0.047	0.136	-0.449	0.813	0.021***
Large	Tiny	0.118	0.129	0.797	0.047	0.154	-0.449	0.811	0.021***

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3). The optimal portfolio is computed under mean–variance preferences for each ML predictor over the period 2004–2024 (see (3.2.8)). The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector. *Pay as* denotes w^{exec} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

Three robust patterns emerge. First, mean–variance delivers a systematically lower-risk profile than volatility benchmarking (Table 3.3). Second, low and stable turnover persists, and net performance improves monotonically as paid costs fall. Third, benchmark-relative performance remains economically meaningful under mean–variance. These three patterns become clear when comparing the Large/Tiny specification under volatility benchmarking (Table 3.4) with the Large/Tiny specification under mean–variance preferences (Table 3.9).

For XGB, the Sharpe ratio declines from 0.792 to 0.764, while the information ratio drops sharply

from 0.696 to -0.030 and α decreases from 0.022^{***} to 0.017^{**} . For the Transformer, the Sharpe ratio falls from 0.743 to 0.725, IR declines from 0.396 to 0.218, and α decreases from 0.020^* to 0.015. For IPCA, the Sharpe ratio decreases from 0.802 to 0.768, IR from 0.512 to 0.307, and α from 0.031^{**} to 0.025^* . For RFF, the Sharpe ratio is essentially unchanged and slightly higher under mean–variance, rising from 0.792 to 0.797, but IR nevertheless falls markedly from 0.627 to 0.154, and α declines modestly from 0.024^{***} to 0.021^{***} . Overall, mean–variance preferences tend to preserve risk-adjusted performance while reducing benchmark-relative performance. This is a logical consequence since the variance penalty will push the portfolio further away from the S&P 500 than the volatility benchmarking. On top of that, μ declines for all predictors under mean–variance: for XGB from 0.131 to 0.110, for the Transformer from 0.132 to 0.121, for IPCA from 0.142 to 0.130, and for RFF from 0.135 to 0.118. At the same time, realised volatility σ is uniformly lower under mean–variance preferences: for XGB it falls from 0.146 to 0.124, for the Transformer from 0.157 to 0.146, for IPCA from 0.158 to 0.150, and for RFF from 0.151 to 0.129. The reduction in volatility is consistent with the quadratic variance penalty pushing portfolios toward a lower-risk region of the feasible set. The decline in μ indicates that this de-risking comes at the cost of somewhat lower average returns.

Overall, acting under mean–variance preferences in the Large/Tiny setting systematically lowers both μ and σ . The Sharpe ratio is therefore largely preserved, but the compression of active risk leads to a pronounced decline in information ratios and moderately smaller alphas. The results show that the VoT is not an artifact of the specific risk constraint and remains robust under mean–variance preferences.

3.5.6 Concentration Limit

A second practical objection to ML-driven portfolio rules is that their apparent outperformance can be driven by extreme concentration in a few names. This concern is particularly relevant in long-only settings without leverage: if the optimiser is allowed to place very large weights on a small number of stocks, performance may reflect episodic single-name bets rather than robust extraction of forecast information, and investability may be overstated. In the baseline implementation, this issue is not merely theoretical: absent explicit constraints, the optimiser can generate weights far above those observed in the S&P 500. For example, the best performing algorithm, IPCA, in the beginning of 2024 has a weight of 24% on TSLA, while its index weight is only 2%.

To address this concern directly, I impose a hard concentration limit $\pi_{\max} = 10\%$, which bounds single-name exposure at a level consistent with institutional risk controls. Imposing the cap therefore provides a demanding test: if the VoT results are driven by concentration, performance should collapse once extreme positions are ruled out.

Table 3.10 reports the VoT design under the concentration limit $\pi_{\max} = 10\%$.

Table 3.10: Portfolio Net Returns. Act as Large. Concentration limit $\pi_{\max} = 10\%$.

Act as	Pay as	μ	σ	SR	TO	IR	MaxD	DCap	α
•	•	0.110	0.147	0.645	•	0.000	-0.509	1.000	0.000
XGB									
Large	Large	0.120	0.144	0.727	0.049	0.330	-0.492	0.958	0.012**
Large	Med	0.124	0.144	0.755	0.049	0.498	-0.491	0.947	0.016***
Large	Small	0.127	0.144	0.778	0.049	0.634	-0.490	0.938	0.019***
Large	Tiny	0.128	0.144	0.783	0.049	0.664	-0.489	0.936	0.020***
Transformer									
Large	Large	0.105	0.151	0.593	0.072	-0.158	-0.515	1.051	-0.005
Large	Med	0.116	0.150	0.670	0.072	0.112	-0.509	1.018	0.006
Large	Small	0.125	0.150	0.732	0.072	0.336	-0.503	0.991	0.015*
Large	Tiny	0.127	0.150	0.746	0.072	0.386	-0.502	0.985	0.017*
IPCA									
Large	Large	0.120	0.153	0.688	0.054	0.181	-0.476	1.027	0.010
Large	Med	0.129	0.152	0.749	0.054	0.373	-0.472	0.999	0.019*
Large	Small	0.136	0.152	0.798	0.054	0.525	-0.469	0.976	0.027**
Large	Tiny	0.138	0.152	0.809	0.054	0.559	-0.468	0.971	0.028***
RFF									
Large	Large	0.118	0.147	0.697	0.050	0.219	-0.488	0.986	0.008
Large	Med	0.123	0.147	0.729	0.050	0.389	-0.486	0.972	0.013**
Large	Small	0.126	0.147	0.755	0.050	0.524	-0.485	0.961	0.017***
Large	Tiny	0.127	0.147	0.761	0.050	0.554	-0.485	0.958	0.017***

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector in (3.2.4). *Pay as* denotes w^{exec} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

When comparing the Large/Tiny regime with that of Table 3.3, the VoT is robust with respect to a concentration limit. For XGB, the Sharpe ratio declines slightly from 0.792 to 0.783, the information ratio decreases from 0.696 to 0.664, and alpha falls from 0.022*** to 0.020***. Hence, performance is modestly attenuated but remains economically strong and statistically significant. For the Transformer, the Sharpe ratio increases marginally from 0.743 to 0.746, while the information ratio declines slightly from 0.396 to 0.386. Alpha decreases from 0.020* to 0.017*. Thus, the cap mildly compresses benchmark-relative performance but leaves overall risk-adjusted performance essentially unchanged. For IPCA, the concentration limit slightly improves both Sharpe

and benchmark-relative performance. The Sharpe ratio increases from 0.802 to 0.809, the information ratio rises from 0.512 to 0.559, and alpha declines only marginally from 0.031** to 0.028***. This suggests that IPCA benefits from the additional regularisation implied by the concentration limit. For RFF, performance weakens more visibly. The Sharpe ratio decreases from 0.792 to 0.761, the information ratio falls from 0.627 to 0.554, and alpha declines from 0.024*** to 0.017***. The cap therefore dampens both absolute and benchmark-relative performance for RFF, although alpha remains economically meaningful and statistically significant.

Table 3.11 depicts the results under the concentration limit when *act as* is equal to *pay as*. The results shows that regularisation improves performance even when it is implemented through a simple concentration constraint, but it also shows that transaction-cost regularisation is the more effective (and more investable) discipline device. Consider first the Tiny/Tiny setting without the concentration limit from Table 3.3. For IPCA, moving from Tiny/Tiny without a concentration limit to Tiny/Tiny with $\pi_{\max} = 10\%$ (Table 3.11) materially improves risk-adjusted performance: the Sharpe ratio rises from 0.497 (Table 3.3) to 0.786 (Table 3.11), while turnover is essentially unchanged (from 0.136 to 0.131). Downside behaviour also improves: downside capture falls from 1.085 to 0.942 (i.e., the portfolio loses less than the benchmark in down markets), and maximum drawdown improves slightly from -0.440 to -0.436 . This confirms the general point that additional regularisation helps by filtering noisy, extreme reallocations that would otherwise be induced by ML forecast noise when the trading penalty is weak.

However, the concentration limit is not a substitute for transaction-cost regularisation. The more stringent comparison is between Tiny/Tiny with $\pi_{\max} = 10\%$ (Table 3.11) and the Large/Tiny VoT portfolio (Table 3.4). Even for IPCA, the best performing return predictor, the VoT portfolio without the concentration limit delivers higher benchmark-relative performance while being dramatically more investable: the Sharpe ratio is higher (VoT: 0.802 vs. capped Tiny/Tiny: 0.786), the information ratio is materially higher (VoT: 0.512 vs. 0.351), and turnover is less than half (VoT: 0.054 vs. 0.131).

The dominance of transaction-cost regularisation is even clearer for predictors that are more prone to forecast-induced overtrading. For the Transformer, Tiny/Tiny with $\pi_{\max} = 10\%$ still performs poorly and is extremely turnover-intensive (SR 0.247, TO 0.391, MaxD -0.631 , DCap 1.265; Table 3.11), whereas the Large/Tiny VoT portfolio without the concentration limit (Table 3.4) achieves strong performance with an order-of-magnitude reduction in turnover (SR 0.743, TO 0.073, MaxD -0.504 , DCap 1.005; Table 3.4). Similarly for RFF, capped Tiny/Tiny delivers SR 0.512 with TO 0.132 and MaxD -0.529 , while the unconstrained VoT Large/Tiny delivers SR 0.792 with TO 0.050 and MaxD -0.485 (and downside capture improves from 1.009 to 0.955).

Table 3.11: Portfolio Net Returns. Act as Pay. Concentration limit $\pi_{\max} = 10\%$.

Act as	Pay as	μ	σ	SR	TO	IR	MaxD	DCap	α
•	•	0.110	0.147	0.645	•	0.000	-0.509	1.000	0.000
XGB									
Large	Large	0.120	0.144	0.727	0.049	0.330	-0.492	0.958	0.012**
Med	Med	0.118	0.142	0.721	0.052	0.226	-0.490	0.945	0.011*
Small	Small	0.106	0.142	0.636	0.072	-0.107	-0.496	0.930	0.004
Tiny	Tiny	0.115	0.157	0.633	0.121	0.037	-0.487	0.921	0.010
Transformer									
Large	Large	0.105	0.151	0.593	0.072	-0.158	-0.515	1.051	-0.005
Med	Med	0.099	0.153	0.543	0.091	-0.248	-0.555	1.066	-0.011
Small	Small	0.085	0.163	0.427	0.156	-0.369	-0.619	1.154	-0.027*
Tiny	Tiny	0.061	0.186	0.247	0.391	-0.537	-0.631	1.265	-0.060***
IPCA									
Large	Large	0.120	0.153	0.688	0.054	0.181	-0.476	1.027	0.010
Med	Med	0.116	0.152	0.664	0.059	0.079	-0.479	1.018	0.008
Small	Small	0.116	0.147	0.682	0.078	0.058	-0.452	0.947	0.015
Tiny	Tiny	0.139	0.157	0.786	0.131	0.351	-0.436	0.942	0.034**
RFF									
Large	Large	0.118	0.147	0.697	0.050	0.219	-0.488	0.986	0.008
Med	Med	0.121	0.149	0.708	0.055	0.268	-0.489	0.991	0.011
Small	Small	0.120	0.152	0.692	0.075	0.187	-0.520	0.992	0.011
Tiny	Tiny	0.095	0.156	0.512	0.132	-0.257	-0.529	1.009	-0.014

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector in (3.2.4). *Pay as* denotes w^{exc} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

3.5.7 Permuted Returns

A core identification concern is that the outperformance of the VoT portfolios could be an artifact of the portfolio-construction machinery itself rather than evidence that ML forecasts contain exploitable information once implemented with disciplined trading. In particular, a sophisticated optimiser combined with a realistic risk model and strong trading frictions might induce systematic tilts (e.g. toward diversified or low-risk holdings) that generate benchmark-relative performance even if the return forecasts are uninformative. A credible test must therefore hold the entire “predict-then-optimise” pipeline fixed and remove only the information content of the

forecasts.

I implement such a placebo via a random permutation test. For each return predictor, I randomly permute the predicted returns time series, destroying any alignment between a stock's forecast and its subsequent realised return. I then re-run the portfolio optimiser under the same VoT discipline (Act as Large) and compute realised net returns under varying execution-cost regimes.

Table 3.12 reports the performance of these placebo portfolios. Two features are reassuring and economically coherent. First, turnover remains low and stable when acting as if transaction costs are large, confirming that the placebo isolates forecast information rather than changing the trading rule. Second, the outcome is exactly what one would expect when the return signal is destroyed: with no information about which stocks should be bought or sold, the optimiser's rebalancing decisions become effectively random. Under long-only full investment and strong transaction-cost regularisation, such random active trades are optimally suppressed, so that the resulting portfolios remain close to the benchmark and, in the limit, behave like a broadly diversified near- $1/N$ allocation³¹ (Table 3.2) rather than generating persistent benchmark-relative alpha. Permuted-forecast portfolios can still exhibit non-zero benchmark-relative performance because the optimiser and constraints induce systematic exposures starting from the value-weighted benchmark.

Taken together, the placebo results provide an internal validity check: once the forecast information is removed, the optimisation machinery does not "manufacture" alpha. Instead, it produces benchmark-like, diversification-driven performance with subdued turnover and volatility, exactly as implied by the transaction-cost regularisation mechanism. Hence, the placebo portfolios do not reproduce the economically large VoT improvements documented earlier when forecasts are not permuted.

To formally assess whether VoT performance exceeds the placebo portfolios (no-signal forecasts), Table 3.13 reports a one-sided test on the return differences. Specifically, Δ is the mean excess return advantage of the VoT portfolio constructed from the true forecasts relative to the analogous portfolio constructed from permuted forecasts, and the p-value is computed under the null that this advantage is non-positive. The results reject the optimiser-artifact explanation for three out of four predictors at conventional levels: XGB ($\Delta = 0.011$, $p = 0.0044$), IPCA ($\Delta = 0.024$, $p = 0.0397$), and RFF ($\Delta = 0.016$, $p = 0.0308$). The Transformer shows a weaker but directionally consistent pattern ($\Delta = 0.014$, $p = 0.0965$), which matches its weaker baseline forecasting discipline due to being the worst performing algorithm.

³¹The match is not exact because the backtest starts from a value-weighted S&P 500 portfolio, so the strategy inherits a baseline value-weight tilt at inception and only gradually drifts toward a more diversified allocation as the optimiser trades.

Table 3.12: Portfolio Net Returns. Act as large. Permuted Predicted Returns.

Act as	Pay as	μ	σ	SR	TO	IR	MaxD	DCap	α
•	•	0.110	0.147	0.645	•	0.000	-0.509	1.000	0.000
XGB									
Large	Large	0.114	0.141	0.696	0.049	0.108	-0.484	0.950	0.007
Large	Med	0.117	0.141	0.717	0.049	0.267	-0.482	0.941	0.010**
Large	Small	0.119	0.141	0.735	0.049	0.393	-0.481	0.934	0.012***
Large	Tiny	0.120	0.141	0.739	0.049	0.421	-0.480	0.932	0.013***
Transformer									
Large	Large	0.095	0.149	0.531	0.097	-0.901	-0.525	1.059	-0.018***
Large	Med	0.106	0.149	0.612	0.097	-0.283	-0.513	1.030	-0.006
Large	Small	0.116	0.149	0.676	0.097	0.235	-0.504	1.007	0.004
Large	Tiny	0.118	0.149	0.690	0.097	0.352	-0.502	1.002	0.006
IPCA									
Large	Large	0.109	0.141	0.666	0.061	-0.130	-0.491	0.952	0.002
Large	Med	0.114	0.141	0.699	0.061	0.115	-0.487	0.939	0.007*
Large	Small	0.117	0.141	0.724	0.061	0.310	-0.485	0.928	0.011***
Large	Tiny	0.118	0.141	0.730	0.061	0.354	-0.484	0.926	0.011***
RFF									
Large	Large	0.113	0.139	0.697	0.049	0.051	-0.478	0.937	0.006*
Large	Med	0.116	0.139	0.719	0.049	0.222	-0.476	0.928	0.009***
Large	Small	0.118	0.139	0.736	0.049	0.358	-0.475	0.921	0.012***
Large	Tiny	0.119	0.139	0.740	0.049	0.388	-0.474	0.920	0.012***

Notes: This table reports summary statistics of *net* portfolio returns (3.2.3) based on the optimal portfolios computed in (3.2.4) for each ML predictor over the period 2004–2024. The column description and the S&P 500 benchmark is given in Table 3.3. *Act as* denotes w^{opt} , i.e. the wealth trajectory used to solve for the optimal portfolio vector in (3.2.4). *Pay as* denotes w^{exec} , i.e. the wealth trajectory used to compute the net return (3.2.3) given an optimal portfolio vector. Therefore, within each predictor block, *TO* is identical across rows because the portfolio allocation remains identical.

This placebo test is particularly stringent for the paper’s mechanism because it conditions on (i) the same investable universe, (ii) the same covariance and transaction-cost forecasts, (iii) the same constraints, and (iv) the same trade-regularisation strength. Any residual performance under permutation therefore reflects only incidental systematic exposures induced by the optimisation problem, not the predictive content of the ML forecasts. The fact that the true-forecast VoT portfolios reliably outperform their permuted counterparts supports the paper’s central claim: *the incremental economic value comes from forecast information that survives disciplined implementation*, not from mechanical properties of the optimiser or from an unconditional “smart

beta” tilt.

Table 3.13: Test on Excess Returns. VoT vs. random permute

Model	Δ	p-value
XGB	0.011	0.0044
Transformer	0.014	0.0965
IPCA	0.024	0.0397
RFF	0.016	0.0308

Note: Δ represents the mean of excess returns. p-values are for a one-sided test ($H_1 : \mu > 0$).

In combination with the VoT decomposition (Tables 3.3–3.4), the permutation evidence tightens the interpretation. Transaction costs are not merely an execution drag; when they act as a stock-specific and period-specific penalty in the portfolio choice problem, they function as an economically identified regularisation device that prevents forecast noise from being translated into excessive, low-quality trading. The gains from acting as if transaction costs are large therefore represent genuine improvements in out-of-sample trade selection rather than a by-product of optimisation under constraints.

3.6 Conclusion

I flip the interpretation of trading frictions: when added on top of standard ML return predictions, transaction costs act as an *economically identified regularisation device* on active trades since transaction costs can be easily calibrated fully out-of-sample without the requirement of any cross-validation routine. I refer to this mechanism as *double regularisation* where the second stage uses shrinkage to extract the little signal present in noisy machine learning return forecasts.

I study the portfolio-construction problem directly in the institutional environment where most capital is deployed: a benchmarked, long-only mandate in the S&P 500 universe with realistic transaction costs, risk controls, and optional concentration limits. Taking ML return forecasts as given, I introduce an implementation filter within a predict-then-optimize framework. The optimiser solves a constrained portfolio choice problem that trades off forecasted returns against a transparent quadratic price-impact cost, under benchmark-like risk. The central insight is that the transaction-cost term functions as a stock- and time-specific ridge penalty on rebalancing. It does not merely reduce net returns mechanically; it changes *which* trades are executed by shrinking those most likely to be driven by forecast noise.

Empirically, this discipline matters. Naive forecast-to-trade rules fail to outperform the benchmark net of even tiny costs in the active S&P 500 universe. In contrast, passing the same forecasts through the cost-aware optimiser delivers economically meaningful outperformance at benchmark-like volatility and low turnover. The key identification is the *Act-as versus Pay-as*

decomposition. By holding the portfolio rule fixed at a high-discipline (high-cost) optimisation regime while varying only the costs paid in realised returns, I separate the regularisation channel from the accounting drag of transaction costs. The resulting evidence supports the paper’s main finding, the *Virtue of Transaction Costs (VoT)*: acting as if costs are large improves out-of-sample trade selection, even when the investor ultimately pays small costs. This pattern is strongest for predictors prone to high forecast dispersion (e.g., Transformers), where weak discipline leads to overtrading, higher realised risk, worse drawdowns, and inferior benchmark-relative performance.

The VoT survives demanding robustness checks. It holds under alternative risk objectives (mean-variance versus volatility benchmarking) and remains economically meaningful under strict single-name concentration limits. Factor regressions show that performance is not mechanically explained by standard exposures; where residual alpha remains (notably for IPCA and, to a lesser extent, RFF), it survives adjustment for common factors including liquidity and momentum. Placebo tests that destroy the information content of the forecasts while leaving the entire optimisation pipeline intact sharply reduce performance toward benchmark-like outcomes, indicating that the optimiser does not “manufacture” alpha. Instead, it selectively extracts and implements the modest predictive content that is present.

These results have two practical implications. First, “ML alpha versus transaction costs” is a misleading framing in low-signal environments. The economically relevant object is the *mapping* from forecasts to trades. Second, transaction costs provide a particularly attractive regularisation device because their strength is economically identified and can be calibrated out-of-sample from observable liquidity and an institution’s AUM. This avoids ad hoc turnover penalties and avoids re-tuning portfolio hyperparameters in a way that risks overfitting. In settings where the forecast signal is weak—as in the liquid large-cap universe—this second-stage discipline is not optional; it is central to implementability.

There are several natural extensions. A dynamic (multi-period) formulation could allow the optimiser to internalise the option value of waiting and the intertemporal structure of trading costs, potentially strengthening the regularisation channel further. Incorporating additional real-world frictions—such as linear commissions, bid-ask spreads, constraints on trading in index inclusions/exclusions, or tracking-error constraints that vary with mandate risk budgets—would refine investability tests. Finally, applying the Act-as/Pay-as identification strategy to other asset classes, alternative benchmarks, and higher-frequency rebalancing would clarify the generality of the VoT and help map when disciplined implementation is most valuable.

Overall, the evidence supports a simple conclusion: in liquid, competitive universes, machine learning forecasts may contain only a small amount of information, but a by embedding transaction costs into the portfolio choice problem and acting as a force of regularisation, they can be a virtue.

Bibliography

- Adam, K. & Nagel, S. (2023), Expectations data in asset pricing, in R. Bachmann, G. Topa & W. van der Klaauw, eds, 'Handbook of Economic Expectations', Academic Press, pp. 477–506.
- Ahn, S., Kaplan, G., Moll, B., Winberry, T. & Wolf, C. (2018), 'When inequality matters for macro and macro matters for inequality', *NBER Macroeconomics Annual* **32**, 1–75.
- Anderson, A. & Brockman, P. (2018), 'An examination of 13F filings', *Journal of Financial Research* **41**(3), 295–324.
- Auclert, A. (2025), 'Hank: A new core of usable macroeconomics', *AEA Papers and Proceedings* **115**, 153–57.
- Auclert, A., Bardóczy, B., Rognlie, M. & Straub, L. (2021), 'Using the Sequence-Space Jacobian to Solve and Estimate Heterogeneous-Agent Models', *Econometrica* **89**(5), 2375–2408.
- Avramov, D., Cheng, S. & Metzker, L. (2023), 'Machine learning vs. economic restrictions: Evidence from stock return predictability', *Management Science* **69**(5), 2587–2619.
- Azinovic, M., Gaegauf, L. & Scheidegger, S. (2022), 'Deep equilibrium nets', *International Economic Review* **63**(4), 1471–1525.
- Azinovic, M. & Žemlička, J. (2025), 'Deep learning in the sequence space'.
URL: <https://arxiv.org/abs/2509.13623>
- Ba, J. L., Kiros, J. R. & Hinton, G. E. (2016), 'Layer normalization'.
URL: <https://arxiv.org/abs/1607.06450>
- Backus, M., Conlon, C. & Sinkinson, M. (2021), 'Common ownership in america: 1980–2017', *American Economic Journal: Microeconomics* **13**(3), 273–308.
- Bahaj, S., Czech, R., Ding, S. & Reis, R. (2023), 'The Market for Inflation Risk', *SSRN Working Paper* 4488881.
- Bahdanau, D., Cho, K. & Bengio, Y. (2016), 'Neural machine translation by jointly learning to align and translate'.
URL: <https://arxiv.org/abs/1409.0473>
- Bank, D., Koenigstein, N. & Giryes, R. (2021), 'Autoencoders'.
URL: <https://arxiv.org/abs/2003.05991>
- Bayer, C. & Luetticke, R. (2020), 'Solving discrete time heterogeneous agent models with aggre-

- gate risk and many idiosyncratic states by perturbation', *Quantitative Economics* **11**(4), 1253–1288.
- Ben-David, I., Li, J., Rossi, A. & Song, Y. (2022), 'What do mutual fund investors really care about?', *The Review of Financial Studies* **35**(4), 1723–1774.
- Bernanke, B., Gertler, M. & Gilchrist, S. (1999), 'The financial accelerator in a quantitative business cycle framework', *Handbook of Macroeconomics* **1**, 1341–1393.
- Bertucci, C., Bertucci, L., Lasry, J.-M. & Lions, P.-L. (2024), 'A mean field game approach to bitcoin mining', *SIAM Journal on Financial Mathematics* **15**(3), 960–987.
- Bilal, A. (2023), 'Solving Heterogeneous Agent Models with the Master Equation', *NBER Working Paper No. 31103*.
- Boppart, T., Krusell, P. & Mitman, K. (2018), 'Exploiting MIT shocks in heterogeneous-agent economies: the impulse response as a numerical derivative', *Journal of Economic Dynamics and Control* **89**, 68–92.
- Branger, N., Konermann, P. & Schlag, C. (2019), 'Optimists and Pessimists in (In)Complete Markets', *Journal of Financial and Quantitative Analysis* **55**(8), 2466–2499.
- Breiman, L. (2001), 'Random forests', *Machine Learning* **45**(1), 5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A. & Stone, C. J. (2017), *Classification And Regression Trees*, Routledge.
- Bretscher, L., Schmid, L., Sen, I. & Sharma, V. (2025), 'Institutional corporate bond pricing', *The Review of Financial Studies* **39**(3), 605–660.
- Brunnermeier, M., Farhi, E., Koijen, R., Krishnamurthy, A., Ludvigson, S., Lustig, H., Nagel, S. & Piazzesi, M. (2021), 'Review article: Perspectives on the future of asset pricing', *The Review of Financial Studies* **34**(4), 2126–2160.
- Brunnermeier, M. & Sannikov, Y. (2014), 'A Macroeconomic Model with a Financial Sector', *American Economic Review* **104**(2), 379–421.
- Buraschi, A. & Jiltsov, A. (2006), 'Model uncertainty and option markets with heterogeneous beliefs', *The Journal of Finance* **61**(6), 2841–2897.
- Cakici, N., Fieberg, C., Metko, D. & Zaremba, A. (2023), 'Machine learning goes global: Cross-sectional return predictability in international stock markets', *Journal of Economic Dynamics and Control* **155**, 104725.
- Caner, M. (2009), 'Lasso-type gmm estimator', *Econometric Theory* **25**(1), 270–290.
- Caner, M. & Zhang, H. (2014), 'Adaptive elastic net for generalized methods of moments', *Journal of Business & Economic Statistics* **32**(1), 30–47.
- Caramp, N. & Silva, D. (2026), 'Monetary policy and wealth effects: The role of risk and heterogeneity', *The Journal of Finance* (preprint).
- Carhart, M. (1997), 'On persistence in mutual fund performance', *The Journal of Finance* **52**(1), 57–82.
- Carmona, R. & Delarue, F. (2013), 'Probabilistic analysis of mean-field games', *SIAM Journal on Control and Optimization* **51**(4), 2705–2734.

- Carvalho, V. M. & Grassi, B. (2019), 'Large Firm Dynamics and the Business Cycle', *American Economic Review* **109**(4), 1375–1425.
- Chen, A. & Velikov, M. (2022), 'Zeroing in on the expected returns of anomalies', *Journal of Financial and Quantitative Analysis* **58**(3), 968–1004.
- Chen, A. Y. & Zimmermann, T. (2022), 'Open source cross-sectional asset pricing', *Critical Finance Review* **11**(2), 207–264.
URL: <http://dx.doi.org/10.1561/104.00000112>
- Chen, L., Pelger, M. & Zhu, J. (2024), 'Deep learning in asset pricing', *Management Science* **70**(2), 714–750.
- Chen, T. & Guestrin, C. (2016), 'XGBoost: A Scalable Tree Boosting System'.
URL: <https://arxiv.org/abs/1603.02754>
- Cochrane, J. (2001), *Asset pricing*, Princeton Univ. Press.
- Cochrane, J. (2011), 'Presidential Address: Discount Rates', *Journal of Finance* **66**(4), 1047–1108.
- Cohen, S., Hebner, J., Jiang, D. & Sirignano, J. (2025), 'Neural actor-critic methods for hamilton-jacobi-bellman pdes: Asymptotic analysis and numerical studies'.
URL: <https://arxiv.org/abs/2507.06428>
- Coimbra, N. & Rey, H. (2023), 'Financial cycles with heterogeneous intermediaries', *Review of Economic Studies* **91**(2), 817–857.
- Constantinides, G. & Ghosh, A. (2017), 'Asset Pricing with Countercyclical Household Consumption Risk', *The Journal of Finance* **72**(1), 415–460.
- Cont, R. (2001), 'Empirical properties of asset returns: stylized facts and statistical issues', *Quantitative Finance* **1**(2), 223–236.
- Daniel, K. & Titman, S. (1997), 'Evidence on the Characteristics of Cross Sectional Variation in Stock Returns', *The Journal of Finance* **52**(1), 1–33.
- Dello Preite, M., Uppal, R., Zaffaroni, P. & Zviadadze, I. (2025), 'Cross-Sectional Asset Pricing with Unsystematic Risk', *SSRN Working Paper 4135146*.
- Detzel, A., Novy-Marx, R. & Velikov, M. (2023), 'Model comparison with transaction costs', *The Journal of Finance* **78**(3), 1743–1775.
- Duarte, V., Duarte, D. & Silva, D. H. (2024), 'Machine learning for continuous-time finance', *The Review of Financial Studies* **37**(11), 3217–3271.
- Ehling, P., Graniero, A. & Heyerdahl-Larsen, C. (2017), 'Asset Prices and Portfolio Choice with Learning from Experience', *The Review of Economic Studies* **85**(3), 1752–1780.
- Elenev, V., Landvoigt, T. & Van Nieuwerburgh, S. (2021), 'A Macroeconomic Model With Financially Constrained Producers and Intermediaries', *Econometrica* **89**(3), 1361–1418.
- Fama, E. (1970), 'Efficient capital markets: A review of theory and empirical work', *The Journal of Finance* **25**(2), 383.
- Fama, E. & French, K. (1993), 'Common risk factors in the returns on stocks and bonds', *Journal of Financial Economics* **33**(1), 3–56.
- Fama, E. & French, K. (2008), 'Dissecting anomalies', *The Journal of Finance* **63**(4), 1653–1678.

- Fama, E. & French, K. (2015), 'A five-factor asset pricing model', *Journal of Financial Economics* **116**(1), 1–22.
- Fernández-Villaverde, J. (2025), Deep learning for solving economic models, Working Paper 34250, National Bureau of Economic Research.
- Fernández-Villaverde, J., Hurtado, S. & Nuño, G. (2023), 'Financial Frictions and the Wealth Distribution', *Econometrica* **91**(3), 869–901.
- Fernández-Villaverde, J. & Levintal, O. (2018), 'Solution methods for models with rare disasters', *Quantitative Economics* **9**(2), 903–944.
- Fernández-Villaverde, J., Nuño, G. & Perla, J. (2024), 'Taming the curse of dimensionality: Quantitative economics with deep learning', *NBER Working Paper no. 33117*.
- Frazzini, A., Israel, R. & Moskowitz, T. (2018), 'Trading costs', *SSRN Electronic Journal*.
- Freyberger, J., Neuhierl, A. & Weber, M. (2020), 'Dissecting characteristics nonparametrically', *The Review of Financial Studies* **33**(5), 2326–2377.
- Friedman, J. H. (2001), 'Greedy function approximation: A gradient boosting machine.', *The Annals of Statistics* **29**(5).
- Fuchs, W. M., Fukuda, S. & Neuhann, D. (2023), 'Demand-system asset pricing: Theoretical foundations', *SSRN Working Paper 4672473*.
- Gabaix, X. (2009), 'Power laws in economics and finance', *Annual Review of Economics* **1**(1), 255–294.
- Gabaix, X. (2011), 'The Granular Origins of Aggregate Fluctuations', *Econometrica* **79**(3), 733–772.
- Galí, J., López-Salido, J. & Vallés, J. (2007), 'Understanding the effects of government spending on consumption', *Journal of the European Economic Association* **5**(1), 227–270.
- Gao, H., Harvey, C., Liu, Y., Tao, X. & Wang, Y. (2026), 'Machine Learning Meets Markowitz', *NBER Working Paper No. 34861*.
- Glorot, X. & Bengio, Y. (2010), Understanding the difficulty of training deep feedforward neural networks, in Y. W. Teh & M. Titterton, eds, 'Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics', Vol. 9 of *Proceedings of Machine Learning Research*, PMLR, pp. 249–256.
- Green, J., Hand, J. & Zhang, X. (2017), 'The Characteristics that Provide Independent Information about Average U.S. Monthly Stock Returns', *The Review of Financial Studies* **30**(12), 4389–4436.
- Greene, W. (2019), *Econometric Analysis, Global Edition*, 8 edn, Pearson.
- Gu, S., Kelly, B. & Xiu, D. (2020), 'Empirical asset pricing via machine learning', *The Review of Financial Studies* **33**(5), 2223–2273.
- Gu, Z., Laurière, M., Merkel, S. & Payne, J. (2024), 'Global solutions to master equations for continuous time heterogeneous agent macroeconomic models'.
URL: <https://arxiv.org/abs/2406.13726>
- Guvenen, F. (2009), 'A Parsimonious Macroeconomic Model for Asset Pricing', *Econometrica* **77**(6), 1711–1750.
- Gârleanu, N. & Panageas, S. (2015), 'Young, old, conservative, and bold: The implications of

- heterogeneity and finite lives for asset pricing', *Journal of Political Economy* **123**(3), 670–685.
- Haddad, V., Huebner, P. & Loualiche, E. (2025), 'How competitive is the stock market? theory, evidence from portfolios, and implications for the rise of passive investing', *American Economic Review* **115**(3), 975–1018.
- Han, X., Roussanov, N. L. & Ruan, H. (2021), 'Mutual fund risk shifting and risk anomalies', *SSRN Working Paper 3931449*.
- Hansen, L. P. (1982), 'Large Sample Properties of Generalized Method of Moments Estimators', *Econometrica* **50**(4), 1029.
- Harvey, C., Liu, Y. & Zhu, H. (2016), '... and the cross-section of expected returns', *Review of Financial Studies* **29**(1), 5–68.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009), *The Elements of Statistical Learning*, Springer New York.
- He, K., Zhang, X., Ren, S. & Sun, J. (2016), Deep residual learning for image recognition, in '2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)', pp. 770–778.
- He, Z., Kelly, B. & Manela, A. (2017), 'Intermediary asset pricing: New evidence from many asset classes', *Journal of Financial Economics* **126**(1), 1–35.
- He, Z. & Krishnamurthy, A. (2013), 'Intermediary Asset Pricing', *American Economic Review* **103**(2), 732–770.
- He, Z. & Krishnamurthy, A. (2018), 'Intermediary Asset Pricing and the Financial Crisis', *Annual Review of Financial Economics* **10**(1), 173–197.
- He, Z. & Krishnamurthy, A. (2019), 'A Macroeconomic Framework for Quantifying Systemic Risk', *American Economic Journal: Macroeconomics* **11**(4), 1–37.
- Hornik, K., Stinchcombe, M. & White, H. (1989), 'Multilayer feedforward networks are universal approximators', *Neural Networks* **2**(5), 359–366.
- Hou, K., Xue, C. & Zhang, L. (2020), 'Replicating anomalies', *The Review of Financial Studies* **33**(5), 2019–2133.
- Hu, Z., Shukla, K., Karniadakis, G. & Kawaguchi, K. (2024), 'Tackling the curse of dimensionality with physics-informed neural networks', *Neural Networks* **176**, 106369.
- Huggett, M. (1993), 'The risk-free rate in heterogeneous-agent incomplete-insurance economies', *Journal of Economic Dynamics and Control* **17**(5-6), 953–969.
- Hyafil, L. & Rivest, R. (1976), 'Constructing optimal binary decision trees is np-complete', *Information Processing Letters* **5**(1), 15–17.
- Ioffe, S. & Szegedy, C. (2015), Batch normalization: Accelerating deep network training by reducing internal covariate shift, in 'Proceedings of the 32nd International Conference on Machine Learning', Vol. 37 of *Proceedings of Machine Learning Research*, PMLR, pp. 448–456.
- Jamilov, R. & Monacelli, T. (2025), 'Bewley banks', *Review of Economic Studies*.
- Jensen, T., Kelly, B., Malamud, S. & Pedersen, L. (2022), 'Machine learning and the implementable efficient frontier', *Swiss Finance Institute Research Paper No. 22-63*.
- Jensen, T., Kelly, B. & Pedersen, L. (2023), 'Is There a Replication Crisis in Finance?', *The Journal*

- of Finance* **78**(5), 2465–2518.
- Jiang, J., Kelly, B. & Xiu, D. (2023), ‘(re-)imag(in)ing price trends’, *The Journal of Finance* **78**(6), 3193–3249.
- Jolliffe, I. (2004), *Principal component analysis*, 2 edn, Springer, New York.
- Kargar, M. (2021), ‘Heterogeneous intermediary asset pricing’, *Journal of Financial Economics* **141**(2), 505–532.
- Kelly, B., Malamud, S. & Zhou, K. (2023), ‘The virtue of complexity in return prediction’, *The Journal of Finance* **79**(1), 459–503.
- Kelly, B., Pruitt, S. & Su, Y. (2019), ‘Characteristics are covariances: A unified model of risk and return’, *Journal of Financial Economics* **134**(3), 501–524.
- Kirkby, R. (2018), ‘Bewley-Hugget-Aiyagari Models: Computation, Simulation, and Uniqueness of General Equilibrium’, *Macroeconomic Dynamics* **23**(06), 2469–2508.
- Koijen, R. & Gabaix, X. (2024), ‘Granular Instrumental Variables’, *Journal of Political Economy* **132**(7), 2274–2303.
- Koijen, R., Richmond, R. & Yogo, M. (2023), ‘Which investors matter for equity valuations and expected returns?’, *Review of Economic Studies* **91**(4), 2387–2424.
- Koijen, R. & Yogo, M. (2019), ‘A demand system approach to asset pricing’, *Journal of Political Economy* **127**(4), 1475–1515.
- Kozak, S., Nagel, S. & Santosh, S. (2020), ‘Shrinking the cross-section’, *Journal of Financial Economics* **135**(2), 271–292.
- Krishnamurthy, A. & Li, W. (2025), ‘Dissecting mechanisms of financial crises: Intermediation and sentiment’, *Journal of Political Economy* **133**(3), 935–985.
- Krusell, P. & Smith, A. (1998), ‘Income and Wealth Heterogeneity in the Macroeconomy’, *Journal of Political Economy* **106**(5), 867–896.
- Kyle, A. S. (1985), ‘Continuous auctions and insider trading’, *Econometrica* **53**(6), 1315.
- Lasry, J.-M. & Lions, P.-L. (2007), ‘Mean field games’, *Japanese Journal of Mathematics* **2**(1), 229–260.
- Leng, C., Lin, Y. & Wahba, G. (2006), ‘A note on the lasso and related procedures in model selection’, *Statistica Sinica* **16**(4), 1273–1284.
- Li, J. (2022), ‘What drives the size and value factors?’, *The Review of Asset Pricing Studies* **12**(4), 845–885.
- Lillicrap, T., Hunt, J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. & Wierstra, D. (2015), ‘Continuous control with deep reinforcement learning’.
URL: <https://arxiv.org/abs/1509.02971>
- Linnainmaa, J. & Roberts, M. (2018), ‘The history of the cross-section of stock returns’, *The Review of Financial Studies* **31**(7), 2606–2649.
- Maliar, L., Maliar, S. & Winant, P. (2021), ‘Deep learning for solving dynamic economic models.’, *Journal of Monetary Economics* **122**, 76–101.
- McLean, R. & Pontiff, J. (2016), ‘Does academic research destroy stock return predictability?’, *The*

- Journal of Finance* **71**(1), 5–32.
- Mehra, R. & Prescott, E. C. (1985), ‘The equity premium: A puzzle’, *Journal of Monetary Economics* **15**(2), 145–161.
- Merton, R. (1969), ‘Lifetime portfolio selection under uncertainty: The continuous-time case’, *The Review of Economics and Statistics* **51**(3), 247.
- Meyer-Gohde, A. (2021), ‘On the accuracy of linear dsge solution methods and the consequences for log-normal asset pricing’, *IMFS Working Paper Series*, No. 154.
- Mikolov, T., Chen, K., Corrado, G. & Dean, J. (2013), ‘Efficient estimation of word representations in vector space’.
- Moll, B. (2025), ‘The trouble with rational expectations in heterogeneous agent models: A challenge for macroeconomics’.
URL: <https://arxiv.org/abs/2508.20571>
- Moll, B. & Ryzhik, L. (2025), ‘Mean field games without rational expectations’.
URL: <https://arxiv.org/abs/2506.11838>
- Muravyev, D., Pearson, N. & Pollet, J. M. (2025), ‘Anomalies and their short-sale costs’, *Journal of Finance* **80**(6), 3639–3694.
- Newey, W. & McFadden, D. (1994), ‘Large sample estimation and hypothesis testing’, *Handbook of Econometrics* **4**, 2111–2245.
- Noel, M., Banerjee, A., Oswal, Y., Amali, G. & Muthiah-Nakarajan, V. (2024), ‘Alternate loss functions for classification and robust regression can improve the accuracy of artificial neural networks’.
URL: <https://arxiv.org/abs/2303.09935>
- Novy-Marx, R. (2014), ‘Predicting anomaly performance with politics, the weather, global warming, sunspots, and the stars’, *Journal of Financial Economics* **112**(2), 137–146.
- Pastor, L. & Stambaugh, R. (2003), ‘Liquidity risk and expected stock returns’, *Journal of Political Economy* **111**(3), 642–685.
- Pohl, W., Schmedders, K. & Wilms, O. (2021), ‘Asset pricing with heterogeneous agents and long-run risk’, *Journal of Financial Economics* **140**(3), 941–964.
- Rahimi, A. & Recht, B. (2007), Random features for large-scale kernel machines, in J. Platt, D. Koller, Y. Singer & S. Roweis, eds, ‘Advances in Neural Information Processing Systems’, Vol. 20, Curran Associates, Inc.
- Raissi, M. (2018), ‘Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations’.
URL: <https://arxiv.org/abs/1804.07010>
- Raissi, M., Perdikaris, P. & Karniadakis, G. (2019), ‘Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations’, *Journal of Computational Physics* **378**, 686–707.
- Ratliff-Crain, E., Van Oort, C., Koehler, M. & Tivnan, B. (2025), ‘Revisiting cont’s stylized facts for modern stock markets’, *Quantitative Finance* **25**(9), 1343–1373.

- Reiter, M. (2009), 'Solving heterogeneous-agent models by projection and perturbation', *Journal of Economic Dynamics and Control* **33**(3), 649–665.
- Sammon, M. & Shim, J. (2024), 'Who Clears the Market When Passive Investors Trade?', *SSRN Working Paper* 4777585 .
- Santurkar, S., Tsipras, D., Ilyas, A. & Madry, A. (2018), 'How does batch normalization help optimization?'.
- Schmitt-Grohé, S. & Uribe, M. (2004), 'Solving dynamic general equilibrium models using a second-order approximation to the policy function', *Journal of Economic Dynamics and Control* **28**(4), 755–775.
- Schneider, A. (2022), 'Risk-sharing and the term structure of interest rates', *The Journal of Finance* **77**(4), 2331–2374.
- Shiller, R. (1981), 'Do stock prices move too much to be justified by subsequent changes in dividends?', *The American Economic Review* **71**(3), 421–436.
- van der Beck, P. (2022a), 'Flow-Driven ESG Returns', *Swiss Finance Institute Research Paper Series*, No. 21-71 .
- van der Beck, P. (2022b), 'On the estimation of demand-based asset pricing models', *Swiss Finance Institute Research Paper* 22-67 **22-67**.
URL: <http://dx.doi.org/10.2139/ssrn.4111329>
- van der Linden, J., Vos, D., de Weerd, M., Verwer, S. & Demirović, E. (2025), 'Optimal or greedy decision trees? revisiting their objectives, tuning, and performance'.
URL: <https://arxiv.org/abs/2409.12788>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. & Polosukhin, I. (2017), 'Attention is all you need'.
URL: <https://arxiv.org/abs/1706.03762>
- Villemot, S. (2011), 'Solving rational expectations models at first order: what Dynare does', *Dynare Working Paper* no. 2 .
- Weinbaum, D. (2009), 'Investor heterogeneity, asset pricing and volatility dynamics', *Journal of Economic Dynamics and Control* **33**(7), 1379–1397.
- Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J. & Sun, L. (2023), Transformers in time series: A survey, in 'Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence', pp. 6778–6786.
- Wharton Research Data Services (2017), 'Research Note Regarding Thomson-Reuters Ownership Data Issues'.
- Winberry, T. (2018), 'A method for solving and estimating heterogeneous agent macro models', *Quantitative Economics* **9**(3), 1123–1151.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L. & Liu, T.-Y. (2020), 'On layer normalization in the transformer architecture'.
URL: <https://arxiv.org/abs/2002.04745>
- Xiong, W. & Yan, H. (2009), 'Heterogeneous expectations and bond markets', *Review of Financial*

Studies **23**(4), 1433–1466.

Zheng, M., Xu, J., Shen, Y., Tian, C., Li, J., Fei, L., Zong, M. & Liu, X. (2022), ‘Attention-based CNNs for Image Classification: A Survey’, *Journal of Physics: Conference Series* **2171**(1), 012068.

Zou, H. (2006), ‘The adaptive lasso and its oracle properties’, *Journal of the American Statistical Association* **101**(476), 1418–1429.

Chapter C

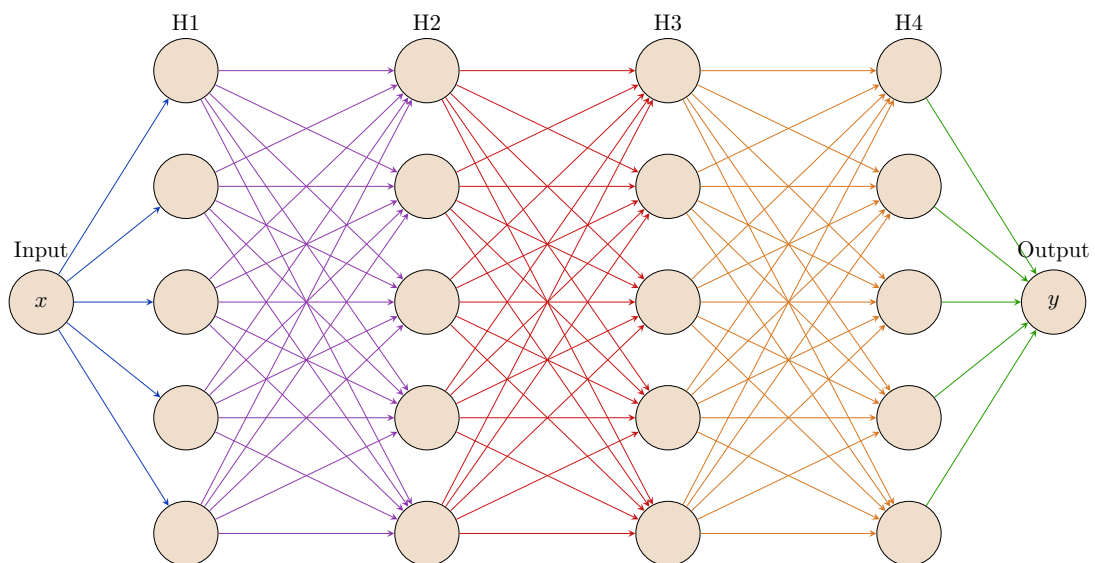
Beating the S&P 500

The Virtue of Transaction Costs

C.1 Neural Networks

This appendix explains the basics of neural networks. For the sake of illustration, a neural network is depicted that takes a one-dimensional input and its output is also one-dimensional. Multivariate extensions immediately follow from the exposition. Moreover, a total of $L = 4$ hidden layers is considered where each layer has 5 neurons each. The neural network is depicted in Figure C.1.

Figure C.1: Neural Network with 4 Hidden Layers with 5 neurons each



Hence, the neural network is a function mapping a real number x (input) to another real number y (output). The edges in Figure C.1 are weights. A weight multiplies the value coming from the previous neuron. The neurons in the four hidden layers H1-H4 are called hidden neurons because they are neither input nor output. Instead, their values are internal to the network and are intermediate quantities used to build the final output y . Each edge in the network connects one neuron in a layer to a neuron in the next layer and carries a real number. In total, the mapping $f : \mathbb{R} \rightarrow \mathbb{R}$ of the neural network works as follows

$$NN(x) = \mathbf{w}^{(5)\top} \cdot \mathbf{W}^{(4)} \cdot \mathbf{W}^{(3)} \cdot \mathbf{W}^{(2)} \cdot \mathbf{w}^{(1)} x_i$$

To make it explicit, the weight vector and matrices are

- $w_{i1}^{(1)}$: Weight from the Input to the i -th Neuron in Hidden Layer 1
- $w_{ij}^{(2)}$: Weight to the i -th Neuron in Hidden Layer 2 from the j -th Neuron in Hidden Layer 1
- $w_{ij}^{(3)}$: Weight to the i -th Neuron in Hidden Layer 3 from the j -th Neuron in Hidden Layer 2
- $w_{ij}^{(4)}$: Weight to the i -th Neuron in Hidden Layer 4 from the j -th Neuron in Hidden Layer 3
- $w_{1j}^{(5)}$: Weight to the 1st Neuron in Output from the j -th Neuron in Hidden Layer 4

Activation Function. Without activation functions, the neural network is simply a linear function. An activation function is applied to each hidden neuron (and potentially also the output neuron). Let this activation function be denoted by $\sigma(\cdot)$. Moreover, let each hidden neuron also feature a bias which is not depicted in Figure C.1. Then, the mapping of the neural network is given by

$$NN(x) = \mathbf{w}^{(5)\top} \cdot \sigma \left(\mathbf{W}^{(4)} \cdot \sigma \left(\mathbf{W}^{(3)} \cdot \sigma \left(\mathbf{W}^{(2)} \cdot \left(\mathbf{w}^{(1)} x_i + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \right) + \mathbf{b}^{(3)} \right) + \mathbf{b}^{(4)} \right) + \mathbf{b}^{(5)}$$

The activation function is applied to each element of the respective vectors. For clarity, the matrices have the following dimension (the dimensionality of the bias vectors is accordingly)

$$\underbrace{\mathbf{w}^{(1)}}_{5 \times 1}, \quad \underbrace{\mathbf{W}^{(2)}}_{5 \times 5}, \quad \underbrace{\mathbf{W}^{(3)}}_{5 \times 5}, \quad \underbrace{\mathbf{W}^{(4)}}_{5 \times 5}, \quad \underbrace{\mathbf{w}^{(5)\top}}_{1 \times 5}.$$

More compactly, the mapping of a neural network is characterised by the following

$$\left(f_{\mathbf{w}^{(L+1)}, \mathbf{b}^{(L+1)}}^{(L+1)} \circ f_{\mathbf{W}^{(L)}, \mathbf{b}^{(L)}}^{(L)} \circ \dots \circ f_{\mathbf{w}^{(1)}, \mathbf{b}^{(1)}}^{(1)} \right) (x)$$

with $f_{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}}^{(l)}(x) = \sigma(\mathbf{W}^{(l)}x + \mathbf{b}^{(l)})$ and $\sigma(\cdot)$ being the activation function.

To highlight the recursive nature, the mapping can be written as follows

$$\begin{aligned}
NN(x_i) &= f_{L+1}(z^{(L+1)}) \\
z^{(L+1)} &= \mathbf{w}^{(L+1).T} \mathbf{a}^{(L)} + b^{(L+1)} \\
\mathbf{a}^{(L)} &= f_L(\mathbf{z}^{(L)}) \\
\mathbf{z}^{(L)} &= \mathbf{W}^{(L)} \mathbf{a}^{(L-1)} + \mathbf{b}^{(L)} \\
\mathbf{a}^{(L-1)} &= f_{L-1}(\mathbf{z}^{(L-1)}) \\
\mathbf{z}^{(L-1)} &= \mathbf{W}^{(L-1)} \mathbf{a}^{(L-2)} + \mathbf{b}^{(L-1)} \\
&\vdots \\
\mathbf{a}^{(l)} &= f_l(\mathbf{z}^{(l)}) \\
\mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\
&\vdots \\
\mathbf{a}^{(2)} &= f_2(\mathbf{z}^{(2)}) \\
\mathbf{z}^{(2)} &= \mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)} \\
\mathbf{a}^{(1)} &= f_1(\mathbf{z}^{(1)}) \\
\mathbf{z}^{(1)} &= \mathbf{w}^{(1)} x_i + \mathbf{b}^{(1)}
\end{aligned}$$

where f_l are the activation functions, $\mathbf{a}^{(l)}$ the hidden neurons and $\mathbf{z}^{(l)}$ the hidden neurons before the activation function is applied.

C.1.1 Training

The goal is to find a mapping, that is weight matrices and bias vectors, such that the inputs (training data) $\mathbf{x} = \{x_i\}_{i=1}^n$ are mapped to a set of outputs (targets) $\mathbf{y} = \{y_i\}_{i=1}^n$. Finding this mapping is called *training*. To find this mapping, initialise the matrices $\{\mathbf{W}_i\}_{i=1}^5$ and bias vectors $\{\mathbf{b}_{i=1}^5\}$ randomly and independently according to any chosen distribution¹.

Moreover, we need an error criterion (loss function) to measure the distance between the mapping of the neural network and the desired error. This can be given by the mean-squared error (MSE). Hence, the distance between the output of the neural networks and the desired output is given by

$$MSE(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n [NN(x_i) - y_i]^2$$

Gradient Descent. Gradient Descent is the general idea of how to find the weight matrices and bias vectors such that $NN(x_i) = y_i \forall i$ (or be as close as possible where the notion of closeness is determined by the error criterion). To explain gradient descent, consider a differentiable loss function $L(\cdot)$. Each element of the weight matrices and bias vectors is a tunable parameter. These parameters are stored in the vector $\boldsymbol{\theta} \in \mathbb{R}^m$ and the neural network is differentiable with respect to $\boldsymbol{\theta}$ (as long as the activation function is differentiable).

With this notation, the aim of the neural network is to solve the following

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^N L(y_i, NN_{\boldsymbol{\theta}}(x_i))$$

for some data $\{x_i, y_i\}_{i=1}^N$. In general, solving this minimisation problem is too computationally intensive so that an iterative approach is chosen²

¹Smart initialisations that facilitate training the neural network exist.

²The iterative approach also has statistical reasons such as reducing overfitting. For example, regularisation such

Let's consider a single generic data point x and y and write $L(\theta)$. Assume we have some initialisation of θ . Using a first order Taylor expansion, for a single parameter θ_j we can write

$$L(\theta_j + \Delta_j) \approx L(\theta_j) + \Delta_j \frac{\partial L}{\partial \theta_j} \quad \Rightarrow \quad L_{new} \approx L_{old} + \Delta_j g_j,$$

where $\Delta_j \in \mathbb{R}$ is a (small) increment of the parameter θ_j and g is the abbreviation for the gradient. Evidently, to reduce the loss, we require $\Delta_j g_j < 0$. A practical way to do this is to pick $\Delta_j = -\eta g_j$ since $-\eta g_j^2$ is evidently always negative for some $\eta > 0$. In this case, η is the learning rate, reflecting that the increment of the parameter θ_j ought to be small so that the Taylor expansion remains valid. Thus, after one gradient descent step, the parameter update is

$$\theta_j \leftarrow \theta_j + \Delta_j \quad \Rightarrow \quad \theta_j \leftarrow \theta_j - \eta g_j.$$

This can be immediately extended to all parameters to that the update step of the parameters given by

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L,$$

where by linearity $\nabla_{\theta} L$ can be evaluated on the empirical loss $1/N \sum_{i=1}^N L(y_i, NN_{\theta}(x_i))$ since the neural network is differentiable with respect to the parameters. Hence, the neural network "learns" (is trained) by adjusting the weights such that the error is minimised.

C.1.2 Backpropagation

Backpropagation is the algorithm used to compute the derivatives of the neural network with respect to its parameters, i.e. $\frac{\partial NN}{\partial \theta}(x)$.

In order to tune the weights θ , we need to know how the error changes when a weight is changed. Thus, we need to know

$$\begin{aligned} \frac{\partial MSE}{\partial w_{ij}^{(l)}} &= \frac{2}{n} \sum_{i=1}^n [NN(x_i) - y_i] \cdot \frac{\partial NN(x_i)}{\partial w_{ij}^{(l)}} \\ \frac{\partial MSE}{\partial b_i^{(l)}} &= \frac{2}{n} \sum_{i=1}^n [NN(x_i) - y_i] \cdot \frac{\partial NN(x_i)}{\partial b_i^{(l)}} \end{aligned}$$

In light of this, it is clear that the challenge is to compute

$$\frac{\partial NN}{\partial w_{ij}^{(l)}} \quad \text{and} \quad \frac{\partial NN(x_i)}{\partial b_i^{(l)}}.$$

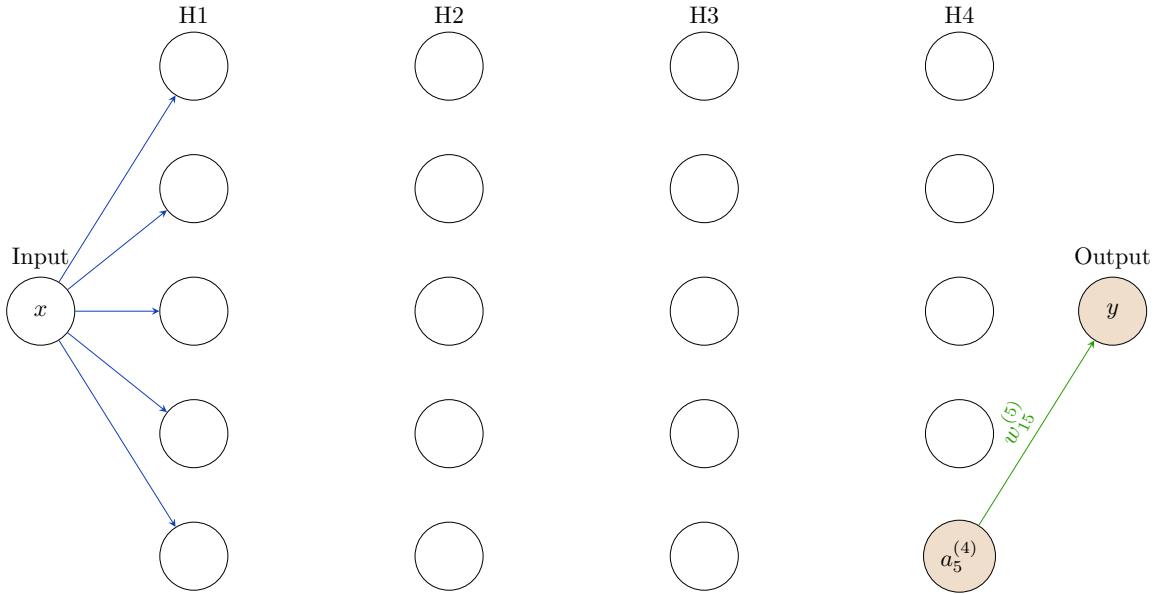
It is easy to compute this derivative when the neurons are passed to the output. To see this, consider Figure C.2 where all neurons that are not relevant for the illustration are muted. Changing $w_{15}^{(L+1)}$ (hidden neuron 5 goes to the first neuron in the next layer) only impacts the output (NN) through the highlighted edge. Thus, it holds that

$$\frac{\partial NN}{\partial w_{15}^{(L+1)}} = \frac{\partial NN}{\partial z_1^{(L+1)}} \frac{\partial z_1^{(L+1)}}{\partial w_{15}^{(5)}} = f'_{L+1}(z_1^{(5)}) a_5^{(L)}$$

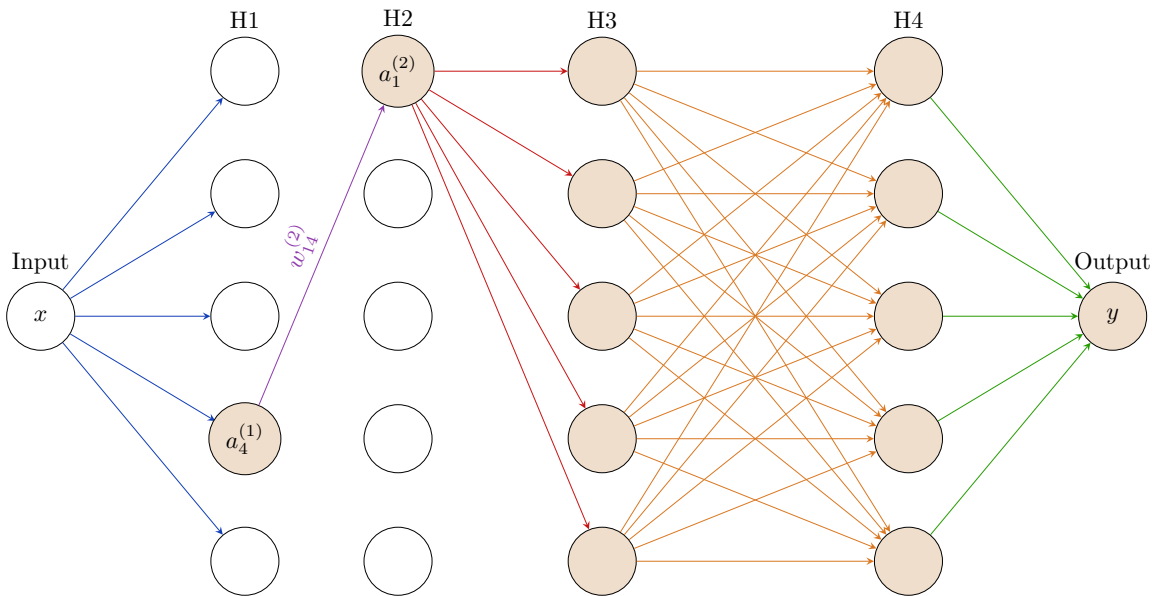
In other words, changing the weights from the final hidden layer to the output neuron linearly scales the output. Hence, we can summarise this by

$$\frac{\partial NN}{\partial \mathbf{w}^{(L+1)}} = \left(\frac{\partial NN}{\partial w_{1i}^{(L+1)}} \quad \cdots \quad \frac{\partial NN}{\partial w_{15}^{(L+1)}} \right)' = \left(a_1^{(L)} \quad \cdots \quad a_5^{(L)} \right)'.$$

as weight decay may not have closed-form minimisers, but they are differentiable. Moreover, the loss landscape can be highly non-convex with respect to θ , so that minimisers such as BFGS may not work well.

Figure C.2: Changing Weights in the Final Layer

Lower Layers. For the final layer computing the contribution of the weights to the error is easy. To compute the contribution of the weights to the error at any layer, consider Figure C.3 where all irrelevant neurons are muted. Clearly, computing $\frac{\partial NN}{\partial w_{14}^{(2)}}$ is not as trivial anymore as changing weights in lower layers affects an increasing number

Figure C.3: Illustration Backpropagation when changing Weight $w_{14}^{(2)}$.

of neurons in higher layers. Although seemingly daunting, there exists a helpful recursive relationship to achieve this task.

Firstly, notice the following points

1. $\frac{\partial NN}{\partial a_1^{(2)}}$ captures the combined contribution of the red, orange and green edges to the error.
2. How a change of $w_{ij}^{(l)}$ impacts $a_i^{(l)}$ is simple.

3. If we know $\frac{\partial NN}{\partial \mathbf{a}^{(l)}}$, we can therefore quickly compute $\frac{\partial NN}{\partial w_{ij}^{(l)}}$.

The *first* is simply the definition of the derivative. If $a_1^{(2)}$ changes, i.e. if the value of the neuron changes, then this change is passed on to all other neurons in the succeeding layers, where the effects become increasingly complicated as the further one proceeds through the layers, the more entangled the neurons become.

The *second* is also easy since $a_i^{(l)} = f_l(z_i^{(l)})$ with $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$. Thus,

$$\frac{\partial a_i^{(l)}}{\partial w_{ij}^{(l)}} = \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} = f'_l(z_i^{(l)}) a_j^{(l-1)}.$$

To see the *third*, we can simply compute

$$\frac{\partial NN}{\partial w_{ij}^{(l)}} = \frac{\partial NN}{\partial a_i^{(l)}} \frac{\partial a_i^{(l)}}{\partial w_{ij}^{(l)}} = \underbrace{\frac{\partial NN}{\partial a_i^{(l)}}}_{\text{Key}} f'_l(z_i^{(l)}) a_j^{(l-1)}$$

Thus, conducting backpropagation can be reduced to computing $\frac{\partial NN}{\partial \mathbf{a}^{(l)}}$. In matrix notation we have

$$\frac{\partial NN}{\partial \mathbf{W}^{(l)}} = \begin{pmatrix} \frac{\partial NN}{\partial w_{11}^{(l)}} & \frac{\partial NN}{\partial w_{12}^{(l)}} & \cdots & \frac{\partial NN}{\partial w_{1N_l}^{(l)}} \\ \frac{\partial NN}{\partial w_{21}^{(l)}} & \frac{\partial NN}{\partial w_{22}^{(l)}} & \cdots & \frac{\partial NN}{\partial w_{2N_l}^{(l)}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial NN}{\partial w_{N_l1}^{(l)}} & \frac{\partial NN}{\partial w_{N_l2}^{(l)}} & \cdots & \frac{\partial NN}{\partial w_{N_lN_l}^{(l)}} \end{pmatrix} = \underbrace{\frac{\partial NN}{\partial \mathbf{a}^{(l)}}}_{i\text{-th Row}} \odot \underbrace{ReLU'(\mathbf{z}^{(l)})}_{j\text{-th Column}} \cdot \mathbf{a}^{(l-1).T}$$

Induction. What remains to be shown is how to constructively compute $\frac{\partial NN}{\partial \mathbf{a}^{(l)}}$. This is provided by the following induction.

1. Compute the base case of the final hidden layer, that is

$$\frac{\partial NN}{\partial \mathbf{a}^{(L)}} = \begin{pmatrix} \frac{\partial NN}{\partial a_1^{(L)}} \\ \vdots \\ \frac{\partial NN}{\partial a_{N_L}^{(L)}} \end{pmatrix} = \mathbf{w}^{(L+1)}$$

which is always easily computed even with an activation function present in the output neuron.

2. Assume $\frac{\partial NN}{\partial \mathbf{a}^{(l)}}$ has been computed. We can then write

$$\frac{\partial NN}{\partial a_i^{(l-1)}} = \sum_{j=1}^{N_l} \frac{\partial NN}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial a_i^{(l-1)}} = \sum_{j=1}^{N_l} \underbrace{\frac{\partial NN}{\partial a_j^{(l)}}}_{\text{Known}} f'_l(z_j^{(l)}) \cdot w_{ji}^{(l)}$$

Within the sum only the chain rule is used.

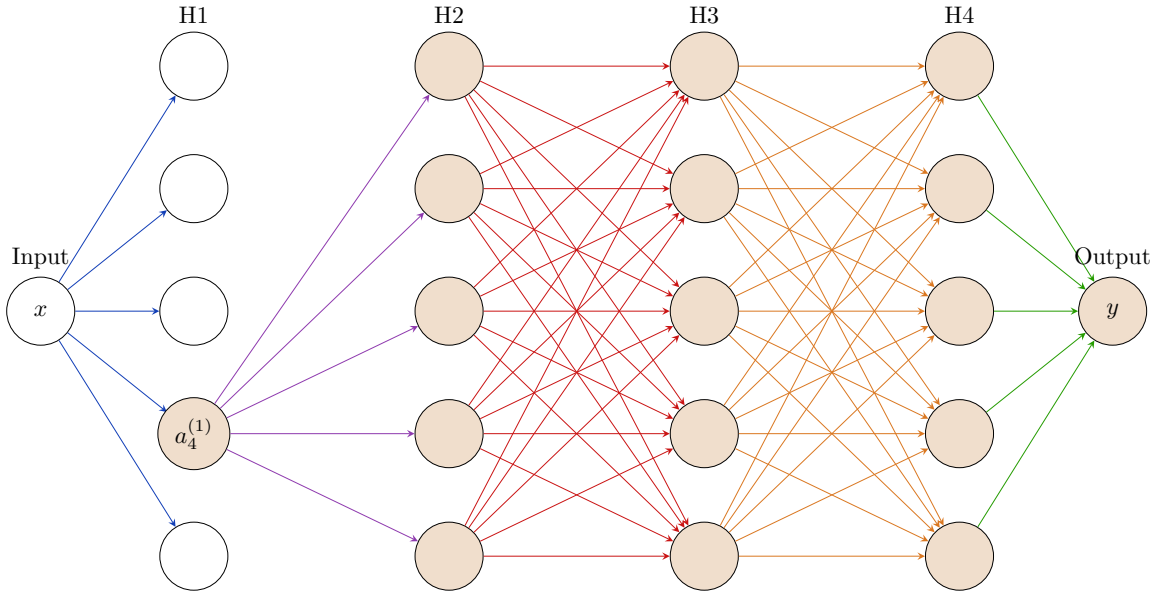
To see where the sum comes from, a graphical argument provides intuition which is given in Figure C.4. Since a neuron in layer $l-1$ is connected to all other neurons in layer l , all these effects must be summed up. Thereby, $\frac{\partial NN}{\partial a_j^{(l)}}$ are all succeeding effects from the neurons in layer l , that is the red, orange and green edges.

The term $\frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial a_i^{(l-1)}}$ captures how changing neuron $a_i^{(l-1)}$ will impact the input of the neurons in the next layer (2nd term) and how this change in input changes the neuron (first term).

Mathematically, we compute

$$\frac{\partial NN}{\partial a_i^{(l-1)}} = \frac{\partial NN}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial a_i^{(l-1)}}$$

Figure C.4: Illustrating the Effect when changing the Neuron $a_4^{(1)}$.



with

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}) \quad \text{and} \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \text{and} \quad \mathbf{a}^{(l-1)} = \begin{pmatrix} a_1^{(l-1)} \\ \vdots \\ a_i^{(l-1)} \\ \vdots \\ a_{N_l}^{(l-1)} \end{pmatrix},$$

so that

$$\frac{\partial \mathbf{z}^{(l)}}{\partial a_i^{(l-1)}} = \begin{pmatrix} \frac{\partial z_1^{(l)}}{\partial a_i^{(l-1)}} \\ \vdots \\ \frac{\partial z_{N_l}^{(l)}}{\partial a_i^{(l-1)}} \end{pmatrix} = \begin{pmatrix} w_{1,i}^{(l)} \\ \vdots \\ w_{N_l,i}^{(l)} \end{pmatrix} \quad \text{and} \quad \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \begin{pmatrix} \frac{\partial a_1^{(l)}}{\partial z_1^{(l)}} & 0 & \dots & 0 \\ 0 & \frac{\partial a_2^{(l)}}{\partial z_2^{(l)}} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \dots & \frac{\partial a_{N_l}^{(l)}}{\partial z_{N_l}^{(l)}} \end{pmatrix}$$

and

$$\frac{\partial NN}{\partial \mathbf{a}^{(l)}} = \left(\frac{\partial NN}{\partial a_1^{(l)}} \quad \dots \quad \frac{\partial NN}{\partial a_{N_l}^{(l)}} \right).$$

Note that $\frac{\partial NN}{\partial \mathbf{a}^{(l)}}$ in this case must be a row vector since $\frac{\partial NN}{\partial a_i^{(l-1)}}$ is a scalar (derivative of scalar with respect to a scalar remains a scalar).

Multiplying all objects together yields the sum.

In matrix notation we have

$$\frac{\partial NN}{\partial \mathbf{a}^{(l-1)}} = \mathbf{W}^{(l)T} \cdot \frac{\partial NN}{\partial \mathbf{a}^{(l)}} \odot f'_l(\mathbf{z}^{(l)})$$

where the transpose of $\mathbf{W}$ changes the running index to the row index in matrix multiplication

Bias. The bias is easier as it is not a matrix and just a scalar or a vector. We can use the same recursive structure, that is

$$\begin{aligned}\frac{\partial NN}{\partial b_i^{(l)}} &= \frac{\partial NN}{\partial a_i^{(l)}} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = \frac{\partial NN}{\partial a_i^{(l)}} f'_l(z_i^{(l)}) \cdot 1 \\ \frac{\partial NN}{\partial \mathbf{b}^{(l)}} &= \frac{\partial NN}{\partial \mathbf{a}^{(l)}} \odot f'_l(\mathbf{z}^{(l)}).\end{aligned}$$

Summary. We firstly note the two base cases

$$\frac{\partial NN}{\partial \mathbf{a}^{(L+1)}} = 1 \quad \text{and} \quad \frac{\partial NN}{\partial \mathbf{a}^{(L)}} = \mathbf{w}^{(L+1)}.$$

We can then iteratively for $l = \{L+1, L, \dots, 2\}$ compute

$$\begin{aligned}\frac{\partial MSE(\mathbf{x})}{\partial \mathbf{W}^{(l)}} &= \frac{2}{n} \sum_{i=1}^n [NN(x_i) - target(x_i)] \left[\frac{\partial NN}{\partial \mathbf{a}^{(l)}(x_i)} \odot ReLU'(\mathbf{z}^{(l)}(x_i)) \right] \cdot \mathbf{a}^{(l-1).T}(x_i) \\ \frac{\partial MSE(\mathbf{x})}{\partial \mathbf{b}^{(l)}} &= \frac{2}{n} \sum_{i=1}^n [NN(x_i) - target(x_i)] \left[\frac{\partial NN}{\partial \mathbf{a}^{(l)}(x_i)} \odot f'_l(\mathbf{z}^{(l)}(x_i)) \right] \\ \frac{\partial NN}{\partial \mathbf{a}^{(l-1)}(x_i)} &= \mathbf{W}^{(l).T} \cdot \left[\frac{\partial NN}{\partial \mathbf{a}^{(l)}(x_i)} \odot f'_l(\mathbf{z}^{(l)}(x_i)) \right] \quad \forall x_i \in \mathbf{x}.\end{aligned}$$

Finally, update

$$\begin{aligned}\mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial MSE(\mathbf{x})}{\partial \mathbf{W}^{(l)}} \\ \mathbf{b}^{(l)} &\leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial MSE(\mathbf{x})}{\partial \mathbf{b}^{(l)}}\end{aligned}$$

C.2 Transformer

Transformers, whose main component is the attention mechanism, were developed for Natural Language Processing (NLP) whose primary purpose is to understand the meaning of words in their given context. Since the release of OpenAI's ChatGPT-3 Model, the attention mechanism has gained significant attention due to its central role in Large Language Models (LLMs). Although mostly attributed to Vaswani et al. (2017), the attention mechanism was actually developed by Bahdanau et al. (2016). The contribution of Vaswani et al. (2017) is the transformer, which fuels LLMs, which basically removes many bells and whistles and keeps attention as its core along with introducing important extensions such as multi-headed attention.

The attention mechanism's power lies in the ability to assign different levels of importance to different input elements. This dynamic focus facilitates the capture of intricate patterns and long-range dependencies. However, understanding elements in a specific context is not only relevant in natural language processing, which is why the attention mechanism is successful in other areas such as image recognition (Zheng et al. 2022) or Time Series (Wen et al. 2023).

In addition to its capacity to understand context, the attention mechanism has other decisive advantages, namely its ability to vectorise and parallelise processes, thereby circumventing the recurrent nature of conventional RNNs and LSTMs, which are constrained to processing a single input at a time. Consequently, this enhances the efficiency of training, provided adequate computational resources are available. In other words, as RNNs and LSTMs process one input at a time, their runtime is $O(N)$ compared to $O(1)$ for the attention mechanism.

In this Section I will explain how the functioning of the transformer and its implementation with a strong emphasis on the attention mechanism. It is most intuitive to explain attention in terms of NLP. In doing so, I will explain why attention is relevant for financial applications as well.

C.2.1 Attention Mechanism

This Section is a deep-dive into the attention mechanism, the main building block of the transformer. The main exposition is conducted within the NLP-context, while I also address how the attention mechanism relates to finance.

How context matters. Consider the sentence "*Time flies*". For a human, it is obvious that "*flies*" in this case implies *moving quickly* and not literal *flying*. Hence, the verb "*flying*" has both the meaning of literal *flying* and of *moving quickly*. Therefore, a machine learning algorithm needs to learn from context which meaning to apply to which word.

Other small examples would be "*The crane flew over the construction site*", where "*crane*" refers to the bird and not the machine, or "*The bank was crowded with people and ducks*", where "*bank*" refers to a river bank and not the financial institution due to the association with "*duck*". Lastly, "*duck*" refers to the animal and not the verb "*to duck*" because of the word "*with*" behind it.

Of course, in long texts, these dependencies can be far-reaching, so that a word's immediate neighbours may not be the most relevant for determining its contextual meaning. For example, imagine a 100-page novel in which a detective solves a case. If the last words are "*and the murder turned out to be...*", then the language model should be able to identify the culprit based on the entire novel. In contrast to the attention mechanism, RNNs or LSTMs cannot reliably predict the culprit because they can usually only capture dependencies over a few paragraphs.

To make the connection to finance, context is also important in financial applications. For example, when looking at the cross-section of returns, the high-dimensional feature input of a stock will not only depend pairwise on the features of another stock, but also several joint dependencies are likely to play a role and shouldn't be ruled out. Furthermore, these interdependencies need not only be cross-sectional, but may also persist over time (potentially over the long term), so understanding the 'context', i.e. multilateral interdependencies and long-run interdependencies, is crucial. Traditional models such as ARIMA are limited frameworks that struggle with this task. In addition, the attention mechanism learns which features to pay most attention to when trying to predict a stock's return. Although I do not include it, the ability of the attention mechanism to understand words implies that it can process both numerical financial data and textual data to extract sentiment in the same model.

Embedding. Going back to NLP, the attention mechanism is part of a neural network and neural networks use numbers. Thus, words or characters of strings cannot be directly used as an input. Embeddings solve this issue because they represent each word³ of a given dictionary with a vector in a high dimensional space $\mathbb{R}^d$ (embedding space) where d is called the embedding dimension.

Each element of an embedding vector relates to some notion a word can represent. For example, the word "flying" will have loadings onto elements in the embedding space that relate to literal flying and also for quick movements. The word "Apple" will be close to the word "Fruit" and "Technology" in the embedding space. These embedding vectors are randomly initialised and learned by the model through training. For example, adding the embedding vectors $e_{\text{King}} + (e_{\text{Female}} - e_{\text{Male}})$ should output the embedding vector for "Queen", i.e. e_{Queen} , if the model was trained well (compare Mikolov et al. (2013) Section 5).

Although embedding is not too important for financial applications because the input data is mostly numerical, embedding has an application. Firstly, the first simple feed-forward layer of a neural network will have the same interpretation as the embedding vector of words, as the neural network will be trying to make sense of the numerical input data it is presented with, and secondly, this will be important for multi-headed attention, as the number of features of the inputs is important.

How attention works in a nutshell. Consider the text "Bank crowded with ducks". Given the context, "bank" refers to a river bank and not the financial institution. Therefore, the word "bank" should pay most attention to "ducks" and update its meaning (embedding vector) accordingly. Thus, whatever initial embedding the model has for "bank", this embedding should be updated the most by the embedding vector of "duck", e.g. (abbreviations are used henceforth for the subscripts)

$$e_b \leftarrow 0.2e_b + 0.1e_c + 0.1e_w + 0.6e_d$$

where the weights in front of the embedding vectors are the attention scores whose computation remains to be determined.

For example, if e_b loads highly onto dimensions relating to finance in the embedding space $\mathbb{R}^d$, by adding much of e_d , the update vector of e_b should load more onto nature so that the model understands that we are talking about a river bank.

Likewise, if the text is "Bank crowded with people", then e_b should probably pay most attention to itself and not change its meaning if the initial embedding already encoded "bank" as a financial institution.

C.2.2 Single-Head Attention

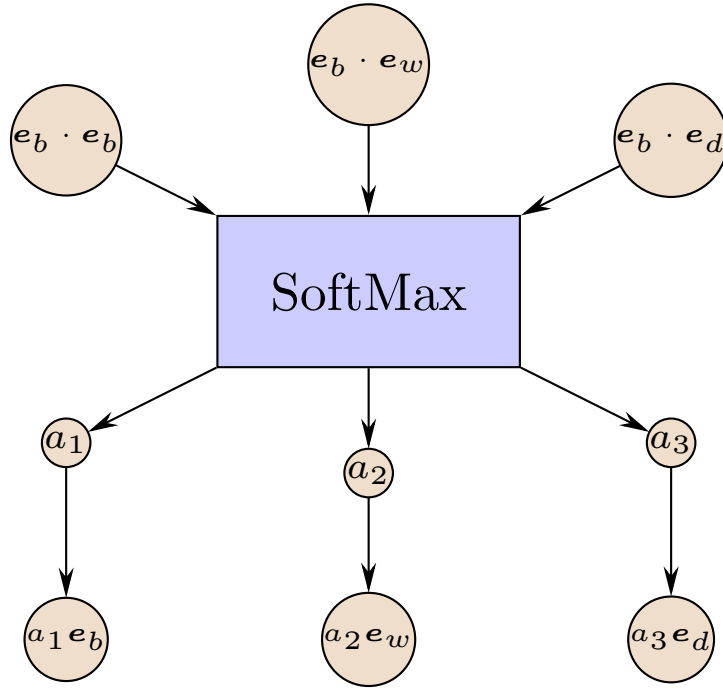
In this Section I motivate and explain how to compute attention scores.

How attention is computed. Figure C.5 illustrates how a naive attention score could be computed. For the sake of simplicity, let's take the text "bank with ducks". Naive attention scores can be computed as follows: to measure the similarity of two words, the dot product of their embedding vectors is computed. So in $e_b \cdot e_d$, the elements corresponding to nature in both embedding vectors will overlap and increase the similarity. The higher the dot product similarity, the more a word should pay attention to the other word.

However, one problem with the raw dot product is that it is not scaled. This problem is solved by applying the SoftMax function, which makes each attention score positive and sum to 1, so that the highest dot product gets the highest attention score. Looking at Figure C.5 immediately shows the computational advantage of attention over RNNs and LSTMs: the operation in Figure C.5 can be vectorised and parallelized because the updating of e_w and e_d is independent of the updating of e_b (parallelization) and the dot products can be computed with matrix algebra (vectorisation). This can also be vectorised if batch sizes greater than one are used, i.e. several different texts. One

³In reality this is more nuanced as LLMs use tokens which are syllables, parts of words or special characters such as "&".

Figure C.5: Naive Attention Mechanism



problem with naive attention scores is that they have no learnable parameters other than the embedding vectors themselves, i.e. the ex-ante meanings of the words. Moreover, dot products are symmetric, so when "bank" looks at "duck" we are looking for the same similarities as when "duck" looks at "bank". This doesn't have to be the case, or at least it's an unnecessary restriction.

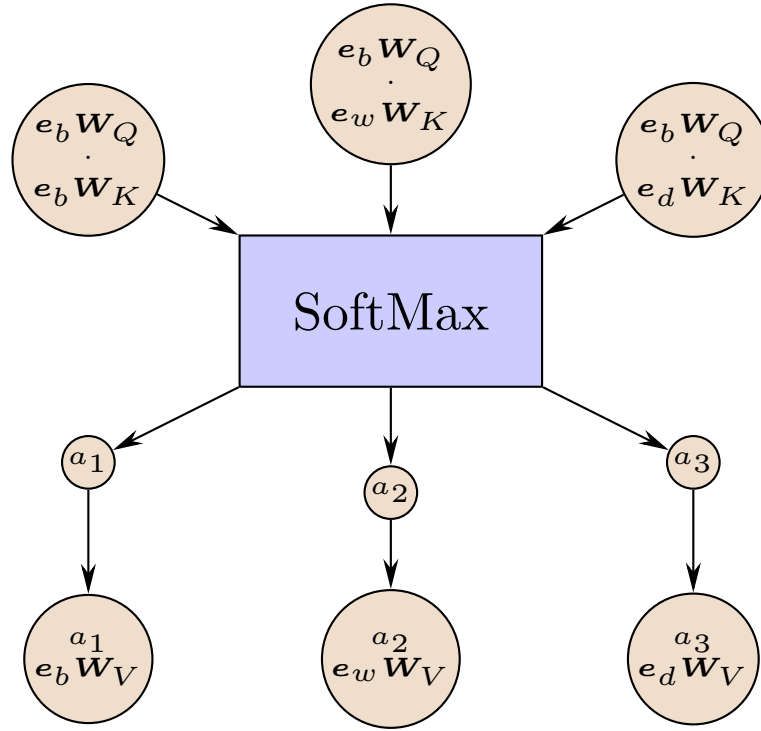
Therefore, to break the symmetry and to include learnable parameters, matrices $\mathbf{W}_Q$, $\mathbf{W}_K$ and $\mathbf{W}_V$ are introduced, which transform the embedding vector into a query (q), key (k) and value (v) vector, e.g. $q_b = e_b \cdot \mathbf{W}_Q$. This is shown in Figure C.6. Each element in the weight matrices $\mathbf{W}_Q$, $\mathbf{W}_K$ and $\mathbf{W}_V$ are learnable parameters. Note that the embedding vectors are row vectors. The reason for this is that when a tensor is fed into a neural network, the tensor's row corresponds to a data point and each column is a feature. The purpose of the Query, Key and Value Vectors is as follows: the Query vector is a transformation of the original embedding vector that asks a question and likewise the key vector is a transformation of the embedding vector of the other word that provides an answer. For example, $\mathbf{W}_Q$ might transform e_b so that it loads highly on nature attributes in the $\mathbb{R}^d$ embedding space. This implies that the word "bank" asks whether the context is related to nature. $\mathbf{W}_K$ then transforms e_d so that it loads highly on nature attributes, thereby providing the answer that the context is nature. The model simultaneously learns what question(s) to ask ($\mathbf{W}_Q$) and what answer(s) to give ($\mathbf{W}_K$).

Notice how this breaks symmetry since e.g. $e_d \mathbf{W}_Q \cdot e_b \mathbf{W}_K$ is not the same as $e_b \mathbf{W}_Q \cdot e_d \mathbf{W}_K$. In other words, "bank" will pay a lot of attention to "duck" as its relevant for its meaning, but "duck" needn't pay any attention to "bank", since "duck" is the animal and not e.g. the action of ducking.

As another example, the query vector of "duck", q_d , could ask whether it is the noun (animal) or the verb (action) by loading highly onto respective features in the embedding space. The key vector of "with", k_w , then provides the answer that it is followed by a noun, thereby establishing that "duck" is indeed the animal which also establishes that "bank" is the river bank.

Of course, these are merely illustrative ways to think about what the attention mechanism could do. In reality, the model might find complicated and non-intuitive ways for a human to establish context, but the underlying mechanism of answering questions and providing answers is the same: if the question(s) of a word are met with the right answer(s) of another word, e.g. $q_b \cdot k_d$ is high, then the model will pay more attention to these word pairs than to other pairs.

Figure C.6: Attention Mechanism



The Value vectors then simply state how the embedding of a word should change if it pays attention to that word, i.e. if a_i is large. For example, "bank" only needs to use the nature attributes of "duck", but not necessarily the "animal" attributes, so that in e_b the loadings onto nature are increased by adding $a_3 v_d$ to e_b .

Since the matrices W_Q , W_K and W_V are linear maps, these are simply edges in a neural network which can be learned by backpropagation in the training process. While there are other tweaks such as positional encoding (words in the proximity are more important than words 100 pages later), for the purposes of my paper Figure C.6 is a sufficient portrayal of the attention mechanism. In the following, I will demonstrate how to vectorise the computation of attention.

Implementing the Attention Mechanism. Leaving the NLP context behind, let's consider a cross-section of stocks. If we have N stocks and F features for each stock, the input matrix is $X \in \mathbb{R}^{N \times F}$. Note that the batch size in this case is 1 since we're only considering one cross-section. To build the Query, Key and Value vectors, a separate simple linear layer of a neural network exists which can be computed in parallel. The Query, Key and Value vectors of each stock is stored in a matrix and given by

$$\underbrace{Q}_{N \times F_h} = XW_Q \quad \underbrace{K}_{N \times F_h} = XW_K \quad \underbrace{V}_{N \times F_h} = XW_V,$$

where W_Q , W_K and W_V are in $\mathbb{R}^{F \times F_h}$ with⁴ $F_h \leq F$. Although F_h can be any number, it is custom to choose $F_h < F$, i.e. a lower dimensional space for the query, key and value vectors. There are a few reasons for this: firstly, it will be pertinent to multi-headed attention. Secondly, introducing sparsity not only reduces the number of parameters, but also improves the runtime⁵⁶. In doing so, the risk of overfitting is reduced and the model ought to focus on compressed relevant attention patterns. Thirdly, reducing the dimension will ensure that the dot products

⁴When introducing the Output Matrix, it will be seen that the Value Matrix could be any dimension $F \times F_v$.

⁵Which is $O(N^2 F_h)$.

⁶In fact, sparse attention mechanisms exist that further reduce the runtime such as Performer, Longformer, Linformer or FlashAttention.

of the query and key vectors do not become too large⁷ which aides stability.

To deepen the understanding, the matrices $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ can be visualised as follows

$$\mathbf{Q} = \begin{bmatrix} \mathbf{q}'_1 \\ \vdots \\ \mathbf{q}'_N \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} \mathbf{k}'_1 \\ \vdots \\ \mathbf{k}'_N \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} \mathbf{v}'_1 \\ \vdots \\ \mathbf{v}'_N \end{bmatrix},$$

so that the i -th row of the matrices correspond to the query, key and value vector of the i -th stock.

The attention scores are computed by the following⁸

$$\mathbf{A} = \text{softmax} \left(\underbrace{\mathbf{Q}\mathbf{K}^T}_{N \times N} \cdot \frac{1}{\sqrt{F_K}} \right) \quad \text{with} \quad \mathbf{Q}\mathbf{K}^T = \begin{bmatrix} \mathbf{q}'_1 \\ \vdots \\ \mathbf{q}'_N \end{bmatrix} [\mathbf{k}_1, \dots, \mathbf{k}_N] = \begin{pmatrix} \mathbf{q}'_1 \mathbf{k}_1 & \dots & \dots & \mathbf{q}'_1 \mathbf{k}_N \\ \vdots & \ddots & & \vdots \\ \mathbf{q}'_N \mathbf{k}_1 & \dots & \dots & \mathbf{q}'_N \mathbf{k}_N \end{pmatrix},$$

where the SoftMax function is applied row-wise. This is the case since the i -th row contains the queries of the i -th stock, the i -th row contains the (unscaled) attention scores for the i -th stock. Compare this with Figure C.6 where the query vector is always the same ($\mathbf{q}_b$ in that case). Moreover, the matrix product $\mathbf{Q}\mathbf{K}^T$ can be efficiently computed with matrix multiplication algorithms and parallelisation (since each row-column dot product can be computed independently from all the rest).

Finally, the rows of the multiplication

$$\text{SoftMax} \left(\mathbf{Q}\mathbf{K}^T \cdot \frac{1}{\sqrt{F_K}} \right) \mathbf{V}$$

yield the result, i.e. the last nodes in Figure C.6. To see this, note that the i -th row of this matrix product is given by

$$\mathbf{z}_i \equiv (a_{i1}, \dots, a_{iN}) \begin{bmatrix} \mathbf{v}'_1 \\ \vdots \\ \mathbf{v}'_N \end{bmatrix} = \sum_{j=1}^N a_{ij} \mathbf{v}'_j.$$

The final issue is that if $F_h < F$, then $\mathbf{z}_i$ is no longer an embedding vector as it is a $1 \times F_h$ vector. Therefore, in the final step an output matrix $\mathbf{W}_O$, consisting of learnable parameters, is used to bring the dimension back to the embedding size.

Summarising everything, the attention scores are computed by

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{SoftMax} \left(\underbrace{\underbrace{\mathbf{X}}_{N \times F} \underbrace{\mathbf{W}_Q}_{F \times F_h} \left(\underbrace{\mathbf{X}}_{N \times F} \underbrace{\mathbf{W}_K}_{F \times F_h} \right)^T}_{N \times N} \cdot \frac{1}{\sqrt{F_K}} \right) \underbrace{\underbrace{\mathbf{X}}_{N \times F} \underbrace{\mathbf{W}_V}_{F \times F_h}}_{N \times F_h}. \quad (\text{C.2.1})$$

To restore the dimension back to $N \times F$, an output matrix $\mathbf{W}_O$, which also consists of learnable parameters, is used. Thus, the updating step is given by

$$\mathbf{X} \leftarrow \underbrace{\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})}_{N \times F_h} \underbrace{\mathbf{W}_O}_{F_h \times F}. \quad (\text{C.2.2})$$

Note that it is possible to implement the output matrix in a different way. For example, the matrices $\mathbf{W}_V$ and $\mathbf{W}_O$

⁷As will be seen, an additional scaling parameter is introduced to mitigate this problem further.

⁸The elements in $\mathbf{Q}\mathbf{K}^T$ are scaled by the overlapping dimension of the matrices to ensure stability. Large values will cause degenerate outputs of the SoftMax function and also lead to vanishing gradients. For any matrix product $\mathbf{AB}$, the resulting variance of the vectors is proportional to the touching dimension. To motivate this, consider two i.i.d random vectors $\mathbf{a}$ and $\mathbf{b}$. Then, $\text{Var}(\mathbf{a} \cdot \mathbf{b}) = \text{Var}(\sum_{i=1}^n a_i b_i) = nV(a_1)V(b_1)$. The scaling factor ensures that larger model dimensions are trainable.

could be represented by a new matrix $\widetilde{\mathbf{W}}$ of dimension $F \times F$. However, if F is large, which it generally is, the matrix $\widetilde{\mathbf{W}}$ would have many more parameters than $\mathbf{W}_Q$ and $\mathbf{W}_K$ together. Therefore, not only is using this low-rank approximation via $\mathbf{W}_V$ and $\mathbf{W}_O$ reducing the number of parameters, it will also be helpful when considering multiple attention heads.

C.2.3 Refinements

The two refinements I discuss here are important for deep neural networks. Although I myself will only train a shallow neural network due to my computational resources, it is useful to mention these refinements as they allow to scale up my approach (given suitable computational resources) and are also nice to implement in shallow networks.

Residual (Skip) Connection. The main idea is that rather than the model trying to predict the embedding vector of the next step in its entirety, the model aims to predict how the embedding vector ought to change. This idea was first introduced by He et al. (2016). Thus, instead of (C.2.1), we use

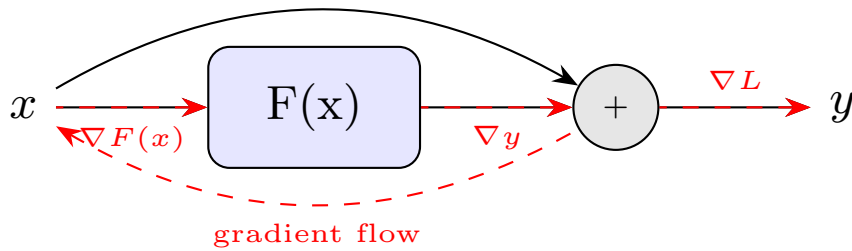
$$\mathbf{X} \leftarrow \mathbf{X} + \underbrace{\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})}_{\Delta \mathbf{X} \mathbf{W}_O}. \quad (\text{C.2.3})$$

This is called a residual or skip connection because the outputs of the neural network are added as a residual. In other words, parts of the input skip the undergoing transformation.

A big advantage of adding a skip connection is that the neural network can learn how many layers out of N of given layers it needs. This works because the model can easily learn to predict $\Delta \mathbf{X} = 0$ by setting all the weights of that layer to zero. This makes it trivial to determine the identity function. Also, it can generally be easier for the neural network to learn the change in $\Delta \mathbf{X}$ than the level, because the level acts as an anchor for the neural network.

Perhaps the most important reason for including residual connections is that the gradient flow is improved. Imagine we're at some layer in the neural network. Let the output of this layer be given by $y = F(x) + x$. Then, the derivative of the loss with respect to x is given by $\frac{dL}{dx} = \frac{dL}{dy} \left(\frac{dF(x)}{dx} + I \right)$ so that all weights before this layer that have contributed to x can flow through to the loss due to the presence of the identity matrix even if $dF(x)/dx$ is small. Figure C.7 illustrates the result.

Figure C.7: Gradient Flow with Residual Connection



Layer Normalisation (LN). Another important concept is layer (Ba et al. 2016) or batch (Ioffe & Szegedy 2015) normalisation. Santurkar et al. (2018) has shown why this helps in the training of neural networks.

Layer normalisation normalises the input vector of a datapoint to mean zero and variance 1 with two additional learnable parameters. Hence, for the input matrix $\mathbf{X}$, each row (datapoint) $\mathbf{x}'_i$ is normalised by

$$\tilde{\mathbf{x}}'_i = \frac{\mathbf{x}'_i - \text{mean}(\mathbf{x}'_i)}{\sqrt{\text{Var}(\mathbf{x}'_i) + \varepsilon}} \gamma + \beta$$

where $\varepsilon > 0$ is a constant to avoid division by zero. γ and β are learnable parameters which are introduced so that if it is optimal to *not* normalise, the model can learn this by undoing the operation by choosing appropriate γ and β .

In contrast, batch normalisation normalises each feature, i.e. the columns of the input matrix $\mathbf{X}$. For LLMs only layer normalisation is applicable as it works with variable input sizes. For example, if an LLM's input size (context window) is 2,000 words, but only a prompt of 50 words is given, then all the remaining 1,950 words must be padded which means that the embedding vector is chosen to be the zero vector. Hence, 1,950 rows of $\mathbf{X}$ are just zero vectors. Normalising the columns (batch normalisation) would imply that all the redundant zeros of the padded inputs come into play and severely tilt the mean and variance to zero of every column for smaller or medium-sized inputs.

For my financial application, *both* batch and layer normalisation are applicable as the input size is always the same.

What is not trivial is when to apply the layer normalisation. The original Vaswani et al. (2017) used what is now called *post-LN*, while modern transformers, including all versions of Chat-GPT, use *pre-LN*. Xiong et al. (2020) showed that when *pre-LN* is used, the gradients of the neural network are well behaved and a warm-up phase to calibrate the learning rate can potentially be skipped. Figure C.8 and Figure C.9 illustrate the differences between the two. In post normalisation, the residual connection is mixed in with the layer normalisation. This prevents the gradient of the residual connection from flowing to the output. In contrast, pre-LN normalisation allows the gradient to flow freely from the residual connection.

As a final illustration for *pre-LN*, assume many sublayers are chained together. Thus, after the first sublayer the output is $x + \Delta x_1$, after the second it is $x + \Delta x_1 + \Delta x_2$ where $x + \Delta x_1$ is the input of the second sublayer. For N sublayers the final output is $x + \sum_{i=1}^N \Delta x_i$. The gradient of the first sub-layer is jumbled up in the other Δx_i for $i \geq 2$ (since these are inputs of the subsequent sublayers), but it is contained (preserved) in Δx_1 and does not vanish by being diluted in Δx_i for $i \geq 2$. In other words Δx_1 has skipped all other sublayers (connections).

Figure C.8: Post-LN

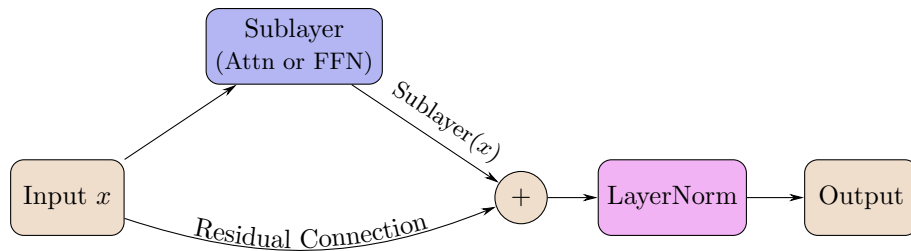
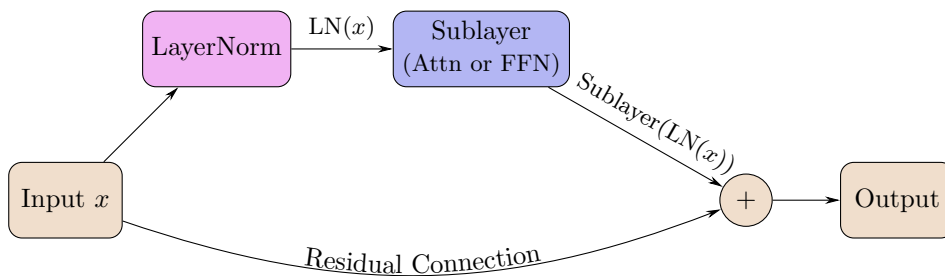


Figure C.9: Pre-LN



C.2.4 Multi-headed Attention

Instead of single-headed attention (C.2.2), multi-headed attention is usually used. The reason for this is that multiple attentions heads can pick up different patterns. This idea is also used in image recognition, where many filters are used to analyse the image in order to detect several different patterns that help the algorithm to classify the image. When it comes to language, for the sentence "my car is in the garage" the model needs both spatial attention "car in the garage" and possession attention "my car". Multiple attention heads are easy to implement.

To illustrate multiple attention heads, Vaswani et al. (2017) used $F = 512$ features (embedding dimension) along with 8 attention heads. Each of the 8 attention heads has its own Query, Key and Value Matrix of size $F \times \frac{F}{8}$. It is thus important that the embedding dimension divided by the number of heads is an integer.

Then, for each head we compute

$$\underbrace{\mathbf{Z}_i}_{N \times \frac{F}{8}} = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) \quad \forall i = 1, \dots, 8,$$

just as in (C.2.1), where each attention head has its separate weight matrices $\mathbf{W}_{Q_i}$, $\mathbf{W}_{K_i}$ and $\mathbf{W}_{V_i}$ consisting of learnable parameters. Notice the power of parallelisation since each attention head can be computed independently from another.

Next, all the $\mathbf{Z}_i$ are concatenated into a matrix

$$\underbrace{\mathbf{Z}}_{N \times F} = \text{concat}(\mathbf{Z}_1, \dots, \mathbf{Z}_8).$$

Lastly, each of the attention heads must communicate with one another. Omitting this would not only entail that the first attention head would only be responsible for updating the first $\frac{F}{8}$ elements in the embedding vector, but also the attention heads couldn't communicate the patterns they found, resulting in poorer model performance. Thus, the final output is

$$\mathbf{X} \leftarrow \mathbf{Z} \underbrace{\mathbf{W}_O}_{F \times F}.$$

One can also think about $\mathbf{W}_O$ as a stacked matrix (row-wise) which stacks each output matrix of the individual attention heads. Likewise, one could also think of big weight matrices $\mathbf{W}_K$, $\mathbf{W}_Q$, $\mathbf{W}_V$ of dimension $F \times F$ which are column-wise split into 8 different parts and used for the individual heads.

C.2.5 Equivariance

We are interested in functions

$$f : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d'},$$

that map one feature matrix (all stocks) to another feature matrix (all stocks, e.g. predicted returns or hidden representations).

Definition C.2.1 (Permutation equivariance). *A function $f : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d'}$ is permutation equivariant (in the token dimension) if*

$$f(PX) = P f(X) \quad \text{for all permutation matrices } P \text{ and all } X.$$

Thus, shuffling the stocks (rows) in the input only shuffles the rows in the output in the same way. This is precisely what you want in a cross-sectional model: it should not matter whether Apple is row 1, row 200, or row 3000. This is important because this entails that in the training window it is not necessary to keep track of the row position of each specific stock.

We will show:

- self-attention without positional encodings is permutation equivariant;
- the standard position-wise feed-forward network does *not* break this equivariance;
- certain normalization layers (including natural variants of batch normalization and layer normalization) also preserve equivariance;
- therefore a transformer stack made of these components is permutation equivariant.

Equivariance of Self-Attention

Consider a single self-attention head without positional encodings. Let $X \in \mathbb{R}^{N \times d}$ be the input. Define

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where W_Q, W_K, W_V are shared weight matrices of appropriate dimensions. Thus $Q, K, V \in \mathbb{R}^{N \times d_k}$ (up to choice of dimensions).

Define the score matrix and attention matrix

$$S = \frac{1}{\sqrt{d_k}} QK^\top \in \mathbb{R}^{N \times N}, \quad A = \text{softmax}_{\text{row}}(S) \in \mathbb{R}^{N \times N},$$

where $\text{softmax}_{\text{row}}$ applies the softmax to each row separately. The output of the head is

$$Y = AV \in \mathbb{R}^{N \times d_v}.$$

Proposition C.2.1. *A single self-attention head (defined as above) is permutation equivariant, i.e. for all permutation matrices P we have*

$$Y' := \text{Attn}(PX) = P \text{Attn}(X).$$

Proof. Let $P \in \mathbb{R}^{N \times N}$ be a permutation matrix and set $X' = PX$. Then

$$Q' = X'W_Q = (PX)W_Q = P(XW_Q) = PQ,$$

and similarly, $K' = PK$ and $V' = PV$.

The score matrix for the permuted input is

$$S' = \frac{1}{\sqrt{d_k}} Q'K'^\top = \frac{1}{\sqrt{d_k}} (PQ)(PK)^\top = \frac{1}{\sqrt{d_k}} PQK^\top P^\top = PSP^\top.$$

Now apply row-wise softmax. For any matrix M and permutation matrix P , we have

$$\text{softmax}_{\text{row}}(PM) = P \text{softmax}_{\text{row}}(M),$$

since permuting the rows of M just permutes the rows of the softmax output. Likewise, for any permutation matrix P ,

$$\text{softmax}(zP^\top) = \text{softmax}(z)P^\top$$

for each row vector z , since permuting the coordinates of z and then taking softmax just permutes the coordinates of the softmax result.

Combining these two facts row-wise, we obtain

$$A' := \text{softmax}_{\text{row}}(S') = \text{softmax}_{\text{row}}(PSP^\top) = P \text{softmax}_{\text{row}}(S) P^\top = PAP^\top.$$

Finally, the output for the permuted input is

$$Y' = A'V' = (PAP^\top)(PV) = PA(P^\top P)V = PAV = PY.$$

Thus, for all P , $\text{Attn}(PX) = P \text{Attn}(X)$, which is permutation equivariance. ■

Multi-head attention is obtained by concatenating or summing several such heads, each defined with its own weight matrices but applied to the same X . A direct sum or concatenation of equivariant maps is again equivariant, so multi-head self-attention is also permutation equivariant.

Equivariance of Position-Wise Feed-Forward Networks The usual transformer feed-forward block is applied *independently* to each token (row) with the *same* parameters. Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ denote the feed-forward function applied to one row, for example

$$\phi(x) = W_2 \sigma(W_1 x + b_1) + b_2,$$

where W_1, W_2 are matrices, b_1, b_2 are bias vectors, and σ is a pointwise nonlinearity (ReLU, GELU, etc.).

Define $F : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d'}$ by

$$F(X)_{i,:} = \phi(x_i) \quad \text{for } i = 1, \dots, N,$$

i.e. apply ϕ to each row independently.

Proposition C.2.2. *If F is defined row-wise with a single shared map ϕ as above, then F is permutation equivariant.*

Proof. Let P be a permutation matrix. Write $\pi : \{1, \dots, N\} \rightarrow \{1, \dots, N\}$ for the corresponding permutation of indices, i.e. $(PX)_{i,:} = x_{\pi(i)}^\top$.

Then

$$F(PX)_{i,:} = \phi((PX)_{i,:}) = \phi(x_{\pi(i)}).$$

On the other hand,

$$(PF(X))_{i,:} = F(X)_{\pi(i),:} = \phi(x_{\pi(i)}).$$

Thus $F(PX)_{i,:} = (PF(X))_{i,:}$ for all i , and therefore $F(PX) = PF(X)$. So F is permutation equivariant. ■

In particular, the standard transformer feed-forward block

$$x \mapsto W_2 \sigma(W_1 x + b_1) + b_2$$

applied row-wise with shared parameters does *not* break permutation equivariance.

Normalization layers are somewhat more subtle. We distinguish two useful cases: (i) layer normalization (LN), and (ii) batch-normalization-type standardization of features, designed carefully.

Layer Normalization (LN) Layer normalization (as used in most transformers) is applied independently to each token across its feature dimension. For each row $x_i \in \mathbb{R}^d$, LN computes

$$\mu_i = \frac{1}{d} \sum_{k=1}^d (x_i)_k, \quad \sigma_i^2 = \frac{1}{d} \sum_{k=1}^d ((x_i)_k - \mu_i)^2,$$

and then

$$\text{LN}(x_i)_k = \gamma_k \frac{(x_i)_k - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_k,$$

where γ_k, β_k are learnable parameters (shared across all rows).

Define $L : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d}$ by applying LN row-wise:

$$L(X)_{i,:} = \text{LN}(x_i).$$

This is again a row-wise map with shared parameters, so by the argument from the previous section it is permutation equivariant.

Proposition C.2.3. *Layer normalization applied independently to each row (with shared parameters across rows) is permutation equivariant.*

Single sample (cross-sectional normalization). Consider a single sample (one time t), so $X \in \mathbb{R}^{N \times d}$. For each feature dimension $j \in \{1, \dots, d\}$ define

$$\mu_j(X) = \frac{1}{N} \sum_{i=1}^N X_{i,j}, \quad \sigma_j^2(X) = \frac{1}{N} \sum_{i=1}^N (X_{i,j} - \mu_j(X))^2.$$

Now define

$$\text{CSN}(X)_{i,j} := \gamma_j \frac{X_{i,j} - \mu_j(X)}{\sqrt{\sigma_j^2(X) + \epsilon}} + \beta_j,$$

where γ_j, β_j are learnable scalars (per feature, shared across stocks).

Proposition C.2.4. *The cross-sectional normalization CSN is permutation equivariant.*

Proof. Let P be a permutation matrix. Consider the permuted input $X' = PX$. Then for any feature j ,

$$\mu_j(X') = \frac{1}{N} \sum_{i=1}^N X'_{i,j} = \frac{1}{N} \sum_{i=1}^N (PX)_{i,j} = \frac{1}{N} \sum_{i=1}^N X_{\pi(i),j} = \frac{1}{N} \sum_{k=1}^N X_{k,j} = \mu_j(X),$$

where π is the permutation of indices corresponding to P . The same argument shows that $\sigma_j^2(X') = \sigma_j^2(X)$, since variance is also a symmetric function of the multiset $\{X_{1,j}, \dots, X_{N,j}\}$.

Therefore, the normalizing statistics (μ_j, σ_j^2) are *unchanged* by permuting the rows of X . Then for each i, j ,

$$\text{CSN}(PX)_{i,j} = \gamma_j \frac{(PX)_{i,j} - \mu_j(X')}{\sqrt{\sigma_j^2(X') + \epsilon}} + \beta_j = \gamma_j \frac{X_{\pi(i),j} - \mu_j(X)}{\sqrt{\sigma_j^2(X) + \epsilon}} + \beta_j = \text{CSN}(X)_{\pi(i),j}.$$

Hence $\text{CSN}(PX)_{i,:} = \text{CSN}(X)_{\pi(i),:}$, which is exactly $(P \text{CSN}(X))_{i,:}$. So $\text{CSN}(PX) = P \text{CSN}(X)$. ■

Thus, if you standardise each feature across stocks (for a given time) in the natural way, that operation respects permutation equivariance.

Mini-batch and batch normalization

In practice you may have a mini-batch of samples. Suppose the input has shape $B \times N \times d$ (batch size B , N stocks, d features). A common batch normalization variant computes for each feature j the mean and variance across *all* (b, i) pairs:

$$\mu_j = \frac{1}{BN} \sum_{b=1}^B \sum_{i=1}^N X_{b,i,j}, \quad \sigma_j^2 = \frac{1}{BN} \sum_{b=1}^B \sum_{i=1}^N (X_{b,i,j} - \mu_j)^2,$$

and then normalizes each $X_{b,i,j}$ with these shared statistics:

$$\text{BN}(X)_{b,i,j} = \gamma_j \frac{X_{b,i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j.$$

For each fixed feature j , μ_j and σ_j^2 are symmetric functions of the multiset $\{X_{b,i,j} : b = 1, \dots, B, i = 1, \dots, N\}$ and hence invariant to any permutation of the (b, i) indices. In particular, if you apply a permutation P to the token index i *within each batch element* b , the set of values does not change, and μ_j, σ_j^2 remain the same.

Thus the same argument as in the cross-sectional case shows that

$$\text{BN}(PX) = P \text{BN}(X),$$

where P acts on the token dimension. Therefore this form of batch normalization is permutation equivariant in the stock dimension.

Caution. If one were to use a variant where the normalization statistics depend on the token index (for example, computing separate means $\mu_{i,j}$ for each token index i across the batch), then a permutation of the token order would *change* which entries contribute to each $\mu_{i,j}$, and such a layer would *not* be permutation equivariant. So to preserve equivariance, the normalization should treat all tokens symmetrically.

Residual Connections and Composition Transformer layers contain residual connections and compositions of the operations described above.

Lemma C.2.1. *If $f, g : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d'}$ are permutation equivariant, then so are:*

- *their sum $h(X) = f(X) + g(X)$ (whenever this is well-defined);*
- *their composition $h(X) = g(f(X))$.*

Proof. Let P be a permutation matrix.

Sum:

$$h(PX) = f(PX) + g(PX) = Pf(X) + Pg(X) = P(f(X) + g(X)) = Ph(X).$$

Composition:

$$h(PX) = g(f(PX)) = g(Pf(X)) = Pg(f(X)) = Ph(X),$$

where we used equivariance of f in the inner step and of g in the outer step. ■

A standard transformer layer (without positional encodings) has the schematic form

$$X \mapsto X + \text{MHA}(\text{Norm}(X)), \quad X \mapsto X + \text{FFN}(\text{Norm}(X)),$$

where:

- MHA is multi-head self-attention (equivariant);
- FFN is a position-wise feed-forward network (equivariant);
- Norm is a suitable normalization layer (LN or an appropriate BN/CSN, as above) that is also equivariant;
- the residual connection adds the input X (identity, which is trivially equivariant).

By the lemma, each sub-layer and each residual block is permutation equivariant. By composing many such layers, the entire transformer encoder

$$T : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d_{\text{out}}}$$

satisfies

$$T(PX) = PT(X) \quad \text{for all permutations } P.$$

Practical remark (not a proof issue). In practice, transformers often use layer normalization rather than batch normalization, because BN's dependence on batch-level statistics can interact unfavorably with sequence modeling and inference. However, from a purely mathematical perspective regarding permutation equivariance, both LN and a carefully designed BN / cross-sectional normalization as above are compatible with the equivariant property.

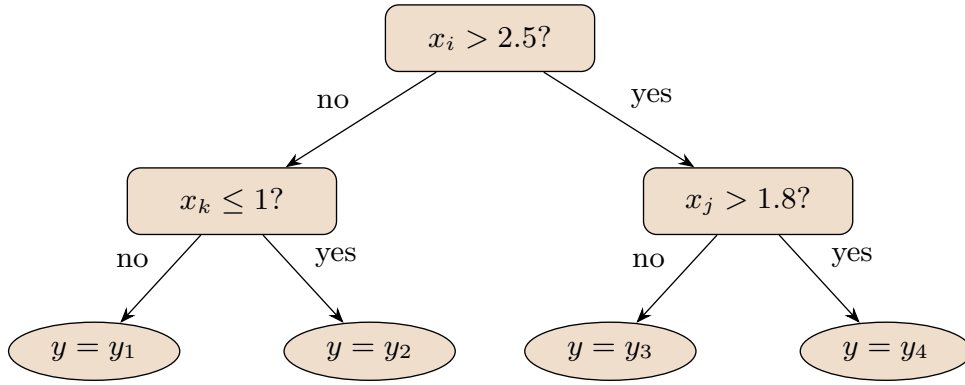
C.3 Trees

Let $\mathcal{X} \subseteq \mathbb{R}^K$ denote the feature space and $\mathcal{Y} \subseteq \mathbb{R}$ the targets space. Throughout, assume the data is stored in a matrix $\mathbf{X} \in \mathbb{R}^{N \times K}$, where each i -th row $\mathbf{x}^{(i)}$ is an observation and each k -th column x_k a feature. High-dimensional settings with $K > N$ are allowed. The target variable $\mathbf{y} \subseteq \mathbb{R}^N$ may be continuous, binary, or categorical.

C.3.1 Decision Tree

A decision tree is the basic building block of all tree-based models. It is a predictive model that represents a sequence of (binary) decisions. For an illustration, consider Figure C.10.

Figure C.10: Decision Tree



The first node is called the root of the tree and asks a question about feature x_i . Each subsequent branch is an outcome of that question and asks a follow up question. Each leaf node (ellipses) is a final prediction of the target variable. These needn't be mutually exclusive and two or more leaves can predict the same variable for the target variable. Moreover, each branch needn't include a question on a new feature, i.e. $i = k = j$ is possible as long as the questions are logically consistent. On top of that, the tree needn't be balanced such that in the left and right sub-tree after the root there needn't be the same number of splits. In short, any node of the tree can be thought of as a bucket that contains a subset $n \leq N$ of the observations in the data set.

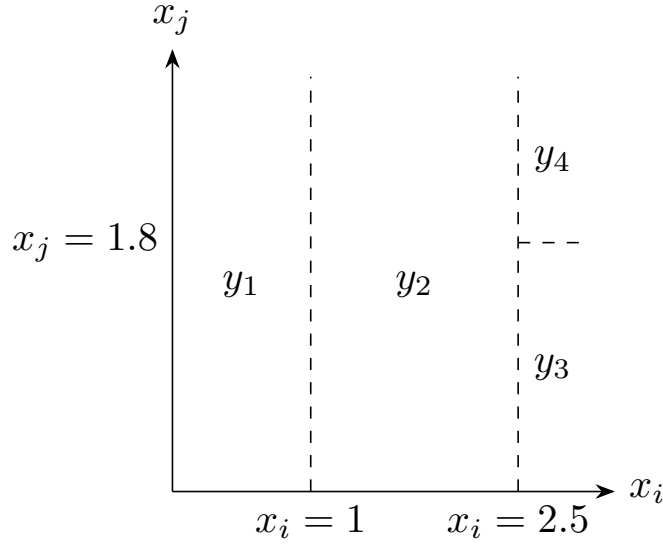
Visually, a decision tree splits the feature space into regions. Assume that for the tree depicted in Figure C.10 that $k = i$, such that the feature space is given by a two-dimensional plane. The regions of the feature space and the corresponding prediction of that region are displayed in Figure C.11.

In short, a decision tree defines a mutually exclusive and exhaustive partition $\{R_1, \dots, R_J\}$ of the feature space and has a prediction for each region. Mathematically, the tree is piecewise constant and can be written as

$$f(\mathbf{x}) = \sum_{j=1}^J c_j \mathbf{1}\{\mathbf{x} \in R_j\}, \quad (\text{C.3.1})$$

where c_j denotes the (constant) prediction assigned to region R_j .

Optimal Splits. The central task is to construct the regions R_j and the associated predictions c_j . In doing so, the goal is to make the observations in each terminal region as homogeneous as possible with respect to the target, so that the impurity (loss) of c_j is as small as possible with respect to the actual targets.

Figure C.11: Regions of Decision Tree

Given a generic loss function $\ell(y, f)$, which is basically a similarity measure of the predictions and the actual targets, the objective of the tree-building procedure is to choose a partition (splits) that minimises the total impurity across all terminal regions (leafs), i.e.

$$\min_{\substack{M \in \mathbb{N}, \{R_m\}_{m=1}^M, \\ \{c_m\}_{m=1}^M}} \sum_{m=1}^M \left(\sum_{y \in R_m} \ell(y, c_m) \right).$$

Solving this is an NP-hard problem, so that no algorithm that scales reasonably with the size of the data exists⁹. To avoid the combinatorial explosion of searching all partitions, the tree is grown recursively and a greedy algorithm called CART (Classification and egression Tree) is considered¹⁰. To start, notice that for a given region, the optimal prediction is obtained by solving

$$c_j^* \in \arg \min_{c \in \mathbb{R}} \sum_{y \in R_j} \ell(y, c).$$

This problem is typically easy to solve. For example, if the MSE is used, i.e. $\ell(y, c) = 0.5/|R_j|(y - c)^2$, the minimiser is the sample mean of region R_j .

Let the impurity of the current parent node in region R be given by

$$I(R) = \min_c \sum_{y \in R} \ell(y, c).$$

At the beginning of the algorithm, i.e. at the root of the tree, R is the entire feature space. This entails that we sum over all N observations in the data set and in the case of the MSE the prediction c would be the sample mean of the target variable.

To find the approximately best split, a greedy algorithm considers iterative binary splits of the form

$$R_1(k, s) = \{x : x_k \leq s\}, \quad \text{and} \quad R_2(k, s) = \{x : x_k > s\},$$

for which it tries all candidate features $k \in 1, \dots, K$ and split points s and chooses the pair (k^*, s^*) that yields

⁹Actually, Hyafil & Rivest (1976) show that this is an NP-complete problem.

¹⁰See Breiman et al. (2017) Chapter 2. Moreover, see van der Linden et al. (2025) for a review of other algorithms.

the greatest reduction in impurity, i.e.

$$(k^*, s^*) \in \arg \min_{\substack{k \in \{1, \dots, K\} \\ s \in \mathbb{R}}} \left[\min_{c_1} \sum_{y \in R_1(k, s)} \ell(y, c_1) + \min_{c_2} \sum_{y \in R_2(k, s)} \ell(y, c_2) \right].$$

To generate split candidates s , the values of feature x_k are sorted, and s is chosen as the midpoints between adjacent sorted values. Having chosen the best split, we replace the current region by the two subregions $R_1(k^*, s^*)$ and $R_2(k^*, s^*)$ and repeat the procedure recursively on each subregion until a stopping criterion is met. The runtime of the greedy algorithm is $O(KN \log(N))$ as at each node for K features N split points must be considered¹¹. Since this is a binary split, at each node the number of observations should roughly halved, so that the total number of iterations must be $\log(N)$. However, this needn't be the case and can significantly worsen the runtime. If the maximum depth of the tree is pruned, then the runtime is $O(KND)$, where D is the depth.

Since a decision tree can perfectly fit the data unless two observations with identical $\mathbf{x}$ have different y , stopping criteria such as a minimum leaf size or a limit on the tree depth are set in order to reduce overfitting. Random Forests are another way to reduce overfitting and are discussed in the next Section.

Missing Values. A great aspect of trees is that they can handle missing data without imputation. At each split $x_k \leq s$ we can test whether shifting the observations with missing values for feature x_k yield the maximum impurity reduction if they are given to R_1 or R_2 . Using such a rule, we can build the tree without having to impute the data.

C.3.2 Random Forests

A single decision tree is intuitive, flexible, and able to capture non-linear interactions among features. However, a single decision tree is prone to overfitting, meaning it can learn the training data, including its noise, too perfectly. This results in a model with very high variance so that small changes in the training data can lead to a completely different tree structures and thus poor out-of-sample performance.

Random Forests, introduced by Breiman (2001), address this problem by averaging many uncorrelated decision trees. The key idea is that although the out-of-sample performance of a single decision tree may be unreliable, the average of many trees can drastically reduce the variance and thus improve predictive performance. When the target variable is categorical, the decision is made by majority vote of the trees. This is called Bagging (Bootstrap Aggregation). In order to train a Random Forest, one firstly draws B bootstrapped samples (with replacement) with full length N from the data original data $\{\mathbf{X}, \mathbf{y}\}$.

For each bootstrapped sample $b \in \{1, \dots, B\}$, a single decision tree f_b with full depth, i.e. perfect in-sample fit can be trained. The Random Forest estimator is then

$$f_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}). \quad (\text{C.3.2})$$

Since the bootstrapped samples are drawn identically and independently from another, the Random Forest stabilises predictions because the average reduces the variance, i.e.

$$\text{Var}(f_{RF}(\mathbf{x})) = \frac{1}{B^2} \sum_{i=1}^B \sum_{j=1}^B \text{Cov}(f_i(\mathbf{x}), f_j(\mathbf{x})).$$

Regularisation. However, decision trees may still be heavily correlated despite bootstrapping. This could occur if a single feature x_k is dominant when building individual decision trees. In other words, if many trees use the same small subset of features and are highly correlated, the variance reduction from bootstrapping is small.

¹¹The computational costs of sorting each feature is not included in this calculation, but the sorting must only be conducted once at the beginning.

Therefore, to reduce the correlation, each bootstrapped sample is only allowed to use a random subset of features $k \leq K$. On top of that, the variance of individual decision trees can be further reduced by pruning the tree, i.e. using a limited depth such that an individual decision tree does not achieve a perfect fit (low bias), but therefore exhibits less variance. Both k and the maximum depth of a Decision Tree are a hyperparameter.

An appealing feature of bagging is the out-of-bag (OOB) error. For each observation i , a subset of decision trees is trained on bootstrapped samples that exclude observation i (at least with probability one if B is large). Let $\mathcal{B}(i)$ denote the set of decision trees for which observation i is OOB. One can then compute

$$f_{OOB}(\mathbf{x}^{(i)}) = \frac{1}{|\mathcal{B}(i)|} \sum_{b \in \mathcal{B}(i)} f_b(\mathbf{x}^{(i)}),$$

which can be used to compute a validation error without an explicit train-test split of the dataset.

C.3.3 Gradient Boosting

Gradient boosting was developed by [Friedman \(2001\)](#). The main idea is rooted in the fact that in a Random Forest each decision tree is trained independently and is thus oblivious to insights gained from the other trees in the forest. Consequently, boosting builds a sequence of trees where each new tree aims to correct the prediction errors made by the previous trees. This entails that every new tree has access to the knowledge gained by the previous trees. To avoid overfitting, boosting algorithms rely on weak learners (simple trees) with a high bias and low variance. The gradient aspect dictates how the boosting is conducted, but has no direct relation to the general ideas of boosting. I follow the approach as in [Hastie et al. \(2009\)](#) Chapter 10.10 onwards, which is identical to [Friedman \(2001\)](#) Equation (16) and (17). Note that boosting is also applicable to a wide class of learners which isn't limited to trees.

Consider a tree $f(\mathbf{x})$. In boosting, the goal is to improve the prediction of $f(\mathbf{x})$ by finding an additional tree such that $f(\mathbf{x}^{(i)}) + h(\mathbf{x}^{(i)}) = y_i \forall i = 1, \dots, N$. Thus, a naive boosting approach would be to fit the new tree $h(\mathbf{x})$ on the residuals $y - f(\mathbf{x})$.

There are many ways to combine individual trees to a final output and the typical choice is a simple linear combination. Therefore, let $\mathcal{F}$ be the set of trees and consider the linear span

$$\text{lin}(\mathcal{F}) = \left\{ \sum_{m=1}^M \beta_m f_m : f_m \in \mathcal{F}, \beta_m \in \mathbb{R} \right\}.$$

Thus, boosting fits an additive expansion of elementary basis functions (learners).

Since we are considering trees, without loss of generality we can set the expansion coefficients $\beta_m = 1 \forall m$. This is due to (C.3.1) so that β_m can be absorbed into the constant $c_{j,m}$ in the respective leaf, i.e. $f_m = \sum_R \tilde{c} \mathbf{1}(R)$ with $\tilde{c} = \beta c$.

When training the algorithm, we want to find an $F \in \text{lin}(\mathcal{F})$ that minimises the following cost *functional*¹²

$$\mathcal{C}(F) := \sum_{i=1}^N \ell(y_i, F(\mathbf{x}^{(i)})).$$

However, tackling this problem head-on is computationally challenging since optimising over a sequence of trees that depend on each other is computationally difficult. Therefore, an iterative and greedy approach is used where one tree is added after another sequentially.

Initialising the first tree is simple since the first tree f_0 is assumed to be constant across the entire feature space. For example, f_0 could output the empirical average of the target variable regardless of the input $\mathbf{x}$. I now proceed inductively. Suppose that after $m - 1$ iterations a model $F_{m-1} \in \text{lin}(\mathcal{F})$ has been constructed. When adding the

¹²This is simply the error in the prediction and could be referred to as the loss. To avoid confusion with the loss function, I call this the cost functional.

next tree $f_m \in \mathcal{F}$, the aim is to solve

$$f_m = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)}) + f(\mathbf{x}^{(i)})),$$

so that the updated model $F_m = F_{m-1} + f_m$ achieves the largest decrease in loss.¹³

However, even this minimisation is generally intractable because decision trees are not parametrised in a form amenable to direct optimisation. Instead, a first-order Taylor approximation of the cost functional is considered. Thus, for a small $\varepsilon \in \mathbb{R}$,¹⁴ consider

$$\begin{aligned} \mathcal{C}(F_{m-1} + \varepsilon f_m) &= \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)}) + \varepsilon f_m(\mathbf{x}^{(i)})) \\ &\approx \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)})) + \varepsilon \sum_{i=1}^N \frac{\partial \ell(y_i, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \bigg|_{F=F_{m-1}} f_m(\mathbf{x}^{(i)}) \\ \mathcal{C}_{new} &\stackrel{\varepsilon=1}{\approx} \mathcal{C}_{old} - \sum_{i=1}^N r_{m-1}(\mathbf{x}^{(i)}) f_m(\mathbf{x}^{(i)}). \end{aligned}$$

We are considering $\varepsilon = 1$ as we are interested in $\mathcal{C}_{new} = \mathcal{C}(F_{m-1} + f_m)$.

The *pseudo-residual* (the negative functional gradient) is denoted by

$$r_{m-1}(\mathbf{x}^{(i)}) \equiv - \frac{\partial \ell(y_i, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \bigg|_{F=F_{m-1}}.$$

This is called a pseudo-residual because it can be used for any loss function ℓ . If the MSE is considered, then $r_{m-1} = (y - F_{m-1})$ which is the actual residual. In this case, it would be optimal to set $f_m = r_{m-1}$ since then $F_{m-1} + f_m = y$. In general, it is optimal to set $f_m = \eta f_{m-1}$ with $0 < \eta \leq 1$ denoting the step-size. Therefore, with the tree we aim to find the *steepest descent* of the loss function, given by the functional derivative (pseudo-residual), which is why the method is called *Gradient Boosting*.

We will deal with the optimal step-size later and firstly focus on achieving $f_m = r_{m-1}$. To do so, the following objective is chosen

$$f_m := \arg \min_{f \in \mathcal{F}} \sum_{i=1}^N \left(r_{m-1}(\mathbf{x}^{(i)}) - f(\mathbf{x}^{(i)}) \right)^2. \quad (\text{C.3.3})$$

The MSE is specifically used because in practice a tree will not be able to satisfy $f_m(\mathbf{x}) = r_{m-1}(\mathbf{x}) \forall \mathbf{x} \in \mathcal{X}$, which would be the optimal choice as is done in gradient descent with neural networks¹⁵. The tree cannot be set to the pseudo-residual because the weak learner we are using doesn't have arbitrary flexibility, i.e. it is unreasonable to assume that the pseudo-residual is piecewise constant.

Since we can't perfectly set the tree to be equal to the pseudo-residual, the next best thing to achieve is the following

$$\max_{f_m} \langle r_{m-1}, f_m \rangle,$$

since this objective is maximised if the tree points in the same direction as the pseudo-residual, i.e. f_m is *maximally*

¹³Boosting is also applicable to non-tree learners. For instance, one can consider $(\beta_m, a_m) = \arg \min_{\beta, a} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \beta h(x_i; \mathbf{a}))$. The vector $\mathbf{a}$ in our case defines the regions (splits) of the tree and the values in the leaves, but there is no reason why $h(x, \mathbf{a})$ couldn't be, for example, a Chebyshev Polynomial where $\mathbf{a}$ are the respective parameters. See Algorithm 1 in [Friedman \(2001\)](#) for how this more general problem is solved.

¹⁴The role of ε is to compute the functional Gateaux derivative. Technically, this has to be done as the cost functional takes a function and not a vector as an input. However, as we only have discrete data points, one can conceptualise $F_{m-1}(\mathbf{x}^{(i)})$ as a vector in $\mathbb{R}^N$ and apply the standard notion of the derivative.

¹⁵In this instance, the magnitude is additionally scaled down by some learning rate $\alpha \in (0, 1)$.

aligned with the pseudo-residual. This is due to $\langle r_{m-1}, f_m \rangle = \|r_{m-1}\| \cdot \|f_m\| \cos(\theta)$ which is maximised for $\theta = 0$. Note that maximising an inner product only makes sense if the norms are bounded as otherwise the inner product can be made infinitely large. Because any tree has bounded complexity which occurs either through regularisation or the inherent structure of the tree (finite number of regions and/or bounded leaf values), this is guaranteed.

Minimising the MSE is equivalent to maximising the inner product which is immediately evident from expanding the objective function, i.e.

$$\sum_{i=1}^N \left(r_{m-1}(\mathbf{x}^{(i)}) - f_m(\mathbf{x}^{(i)}) \right)^2 = \|r_{m-1}\|^2 + \|f_m\|^2 - 2\langle r_{m-1}, f_m \rangle,$$

which explains the fixed choice of the MSE.

Next we determine the optimal step-size of the steepest ascent since the first-order Taylor expansion is only locally valid. In other words, if the step in the direction of the steepest descent is too large, we may end up increasing the loss¹⁶.

Therefore, the solution of (C.3.3) solely yields the regions $\{R_{j,m}\}_{j=1}^J$ of the tree f_m . Given these regions, the final tree is obtained by solving the following problem

$$\mathbf{c}_m = \arg \min_{\{c_{1,m}, \dots, c_{J,m}\}} \sum_{i=1}^N \ell \left(y_i, F_{m-1}(\mathbf{x}^{(i)}) + c_{j,m} \mathbf{1}(\mathbf{x}^{(i)} \in R_{j,m}) \right).$$

Since the regions are mutually exclusive and exhaustive, each $c_{j,m}$ can be optimised independently. Notice that performance can never decrease, i.e. the loss can never increase, as we could set $\mathbf{c} = \mathbf{0}$.

Thus, the final tree is given by

$$f_m(\mathbf{x}) \leftarrow \sum_{\mathbf{x} \in R_{j,m}} c_{j,m} \mathbf{1}(\mathbf{x} \in R_{j,m}).$$

Notice that due to how $\mathbf{c}_m$ was obtained, there is an individual step-size in the gradient descent for each leaf. Alternatively, one could use f_m from (C.3.3) and find one global step-size γ by line-search that minimises $\sum_i \ell \left(y_i, F_{m-1}(\mathbf{x}^{(i)}) + \gamma f_m(\mathbf{x}^{(i)}) \right)$. However, having an individual leaf-specific step-size gives us more degrees of freedom and will therefore, in the worst case, yield the same performance as the global line-search.

Putting everything together, the new predictor is given by

$$F_m(\mathbf{x}^{(i)}) = F_{m-1}(\mathbf{x}^{(i)}) + \sum_{(\mathbf{x}^{(i)}) \in R_{j,m}} c_{j,m} \mathbf{1}\{(\mathbf{x}^{(i)}) \in R_{j,m}\}.$$

Compare this with Algorithm 10.3 in [Hastie et al. \(2009\)](#).

Regularisation. [Hastie et al. \(2009\)](#) state that the depth of each of the weak learners f_m should be limited to $2 \leq J \leq 8$, where many times the maximum depth J needn't necessarily exceed the value of 6. The optimal number of trees M is, as is standard in Machine Learning, determined by optimal stopping by evaluating the training error on a valuation set. Moreover, a learning rate $0 < \alpha < 1$ is advised such that

$$F_M = F_{M-1} + \alpha f_m,$$

where $\alpha < 0.1$ is recommended.

¹⁶Geometrically, while the negative gradient points downhill, the loss landscape is curved. Moving too far in this linear direction can result in overshooting the valley floor and ascending back upwards the loss landscape.

Algorithm 1: Gradient Boosting Tree Algorithm

Input : Training set $\{(\mathbf{x}^{(i)}, y_i)\}_{i=1}^N$, a differentiable loss function $\ell(y, F(\mathbf{x}))$, number of trees M , and learning rate α .

Output: The final model $F_M(\mathbf{x})$.

```

/* 1. Initialise the model with a constant value: */
1:  $F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^N \ell(y_i, \gamma)$ ;
2: for  $m = 1$  to  $M$  do
    /* 2.a Compute pseudo-residuals: */
    3: for  $i = 1$  to  $N$  do
    4:  $r_{m-1}(\mathbf{x}^{(i)}) = - \left[ \frac{\partial \ell(y_i, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \right]_{F=F_{m-1}}$ ;
    /* 2.b Fit a weak learner (tree) to pseudo-residuals: */
    5: Fit a regression tree to  $r_{m-1}(\mathbf{x}^{(i)})$  giving terminal regions  $R_{j,m}$  for  $j = 1, \dots, J_m$ ;
    /* 2.c Compute optimal leaf values: */
    6: for  $j = 1$  to  $J_m$  do
    7:  $c_{j,m} = \arg \min_c \sum_{\mathbf{x}^{(i)} \in R_{j,m}} \ell(y_i, F_{m-1}(\mathbf{x}^{(i)}) + c)$ ;
    /* 2.d Update the model: */
    8:  $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \alpha \sum_{j=1}^{J_m} c_{j,m} \mathbf{1}(\mathbf{x} \in R_{j,m})$ ;
9: return  $F_M(\mathbf{x})$ 

```

C.3.4 XGBoost

XGBoost (Extreme Gradient Boosting), see [Chen & Guestrin \(2016\)](#), is an optimised version of the above described Gradient Boosting algorithm. It contains many added features such as explicit L_1 and L_2 regularisation of the values in the leaves and many system optimisations such as efficient Cache memory allocation, GPU support, heuristic splitting rule for s and many more which are listed in the paper.

One of the key differences I want to emphasise is that XGBoost does not rely solely on the first-order Taylor expansion of the loss, but instead uses the second-order Taylor approximation. This yields an explicit formula for the optimal leaf weights which yields speed gains.

With L_2 regularisation¹⁷, the cost functional is given by

$$\mathcal{C}(F_M) = \sum_{i=1}^N \ell(y_i, F_M(\mathbf{x}^{(i)})) + \sum_{m=1}^M \Omega(f_m),$$

where for a tree with leaves $\mathbf{c} = (c_1, \dots, c_{J_m})$ the penalty Ω is defined as

$$\Omega(f_m) = \gamma J_m + \frac{\lambda}{2} \sum_{j=1}^{J_m} c_{j,m}^2,$$

with hyperparameter $\gamma \geq 0$ penalising the number of leaves of tree f_m and hyperparameter $\lambda \geq 0$ imposing an L^2 penalty on the leaf values.

Once again, the trees are built iteratively. At iteration m , consider the cost functional

$$\mathcal{C}(F_{m-1} + f_m) = \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)}) + f_m(\mathbf{x}^{(i)})) + \sum_{t=1}^{m-1} \Omega(f_t) + \Omega(f_m).$$

¹⁷I omit the L_1 penalty. With a soft-threshold, this penalty can be included into the closed-form expression derived in the end as well.

XGBoost approximates the first term using the functional derivative as was done with gradient boosting. Hence, we approximate

$$\sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)}) + \varepsilon f_m(\mathbf{x}^{(i)})) \approx \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)})) + \varepsilon \sum_{i=1}^N g_i f_m(\mathbf{x}^{(i)}) + \frac{\varepsilon^2}{2} \sum_{i=1}^N h_i f_m(\mathbf{x}^{(i)}),$$

where the functional derivatives are given by

$$g_i := \left. \frac{\partial \ell(y_i, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \right|_{F=F_{m-1}}, \quad h_i := \left. \frac{\partial^2 \ell(y_i, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})^2} \right|_{F=F_{m-1}}.$$

Using this, the cost functional is approximately

$$\begin{aligned} \mathcal{C}(F_{m-1} + f_m) &\approx \sum_{i=1}^N \ell(y_i, F_{m-1}(\mathbf{x}^{(i)})) + \sum_{t=1}^{m-1} \Omega(f_t) \\ &\quad + \sum_{i=1}^N g_i f_m(\mathbf{x}^{(i)}) + \frac{1}{2} \sum_{i=1}^N h_i f_m(\mathbf{x}^{(i)}) + \Omega(f_m), \end{aligned}$$

where the first line is simply $\mathcal{C}(F_{m-1})$. At iteration m this value can no longer be changed and is therefore irrelevant when it comes to finding the optimal tree f_m .

In sum, XGBoost also uses the (negative) pseudo-residual. On top of that, it makes use of the second derivative h_i , which supplies curvature information. Thus, in order to (approximately) minimise $\mathcal{C}(F_{m-1} + f_m)$, we must achieve the following objective

$$\text{Obj} = \min_{f_m \in \mathcal{F}} \sum_{i=1}^N \left(g_i f_m(\mathbf{x}^{(i)}) + \frac{1}{2} h_i f_m(\mathbf{x}^{(i)})^2 \right) + \gamma J_m + \frac{\lambda}{2} \sum_{i=1}^N \sum_{j=1}^{J_m} c_{j,m}^2 \mathbf{1}\{\mathbf{x}^{(i)} \in R_{j,m}\}.$$

As before, for a given region of the tree, the objective simplifies to

$$\text{Obj}_j = \min_{c_{j,m}} \sum_{i \in R_{j,m}} \left(g_i c_{j,m} + \frac{1}{2} h_i c_{j,m}^2 \right) + \frac{\lambda}{2} c_{j,m}^2 + \gamma,$$

where $\sum_{j=1}^{J_m} \text{Obj}_j = \text{Obj}$. Notice that Obj_j is quadratic in $c_{j,m}$ and can be computed in closed-form.

By taking the derivative of $c_{j,m}$ and setting it to zero, the (approximate) optimal prediction in each region is given by

$$c_{j,m}^* = -\frac{G_j}{H_j + \lambda} \quad \text{with} \quad G_j := \sum_{i \in R_{j,m}} g_i \text{ and } H_j := \sum_{i \in R_{j,m}} h_i,$$

and the respective minimum value of the objective is

$$\text{Obj}_j^* = -0.5 \frac{G_j^2}{H_j + \lambda} + \gamma \quad \Rightarrow \quad \text{Obj} = -0.5 \sum_{j=1}^{J_m} \frac{G_j^2}{H_j + \lambda} + \gamma J_m.$$

Notice the role of the second-order term. If H_j is large (high curvature), then the first derivative (gradient) G_j changes rapidly. In other words, the second-derivative reflects uncertainty about the loss landscape so that the algorithm takes smaller steps by reducing the magnitude of the leaf change. Notice that when using the MSE for the loss, the second-order term is always the same constant and thus there is no benefit in this case. On top of that, it is very important that $H_j > 0$ as otherwise we are computing a maximum and are increasing the loss. For the MSE, this is always guaranteed.

Using this quadratic approximation, at each node XGBoost evaluates candidate splits

$$R_R(k, s) = \{x : x_k \leq s\}, \quad \text{and} \quad R_L(k, s) = \{x : x_k > s\}$$

by their *gain* (reduction in loss) just as with regular Gradient Boosting (greedy algorithm).

At any node of the tree, this is given by

$$\begin{aligned}
 \text{Gain}_{node} &= \text{Obj}_{L+R} - [\text{Obj}_L + \text{Obj}_R] \\
 &= -0.5 \frac{G_{L+R}^2}{H_{L+R} + \lambda} + \gamma - \left[-0.5 \frac{G_L^2}{H_L + \lambda} + \gamma - 0.5 \frac{G_R^2}{H_R + \lambda} + \gamma \right] \\
 &= 0.5 \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_{L+R}^2}{H_{L+R} + \lambda} \right] - \gamma,
 \end{aligned}$$

where $L + R$ is the parent node. The split with the maximum gain is chosen. However, only splits with positive gains are accepted. Else-wise the branch is not expanded, i.e. no further split is conducted.

XGBoost can be viewed as a second-order, regularised generalisation of classical Gradient Boosting, with efficient closed-form solutions for leaf weights and a well-defined split quality criterion.

C.4 Factor Models

C.4.1 Static

Consider the following static factor model

$$\mathbf{r}_t = \mathbf{\Gamma} \mathbf{f}_t + \mathbf{e}_t \quad t \in \{1, \dots, T\}, \quad (\text{C.4.1})$$

where $\mathbf{r}_t$ is an $N \times 1$ vector of asset returns at time t , $\mathbf{\Gamma}$ is an $N \times K$ matrix of factor loadings, $\mathbf{f}_t$ is a $K \times 1$ vector of common factors, and $\mathbf{e}_t$ is an $N \times 1$ vector capturing the idiosyncratic errors. Hence, the k -th column of $\mathbf{\Gamma}$, γ_k , contains the factor loadings onto the k -th factor and the n -th row of $\mathbf{\Gamma}$ contain the loadings of the n -th stock onto all K factors.

We can also consider the stacked model. Define the following matrices

$$\underbrace{\mathbf{R}}_{T \times N} \equiv \begin{pmatrix} \mathbf{r}_1^\top \\ \mathbf{r}_2^\top \\ \vdots \\ \mathbf{r}_T^\top \end{pmatrix}, \quad \underbrace{\mathbf{F}}_{T \times K} \equiv \begin{pmatrix} \mathbf{f}_1^\top \\ \mathbf{f}_2^\top \\ \vdots \\ \mathbf{f}_T^\top \end{pmatrix}, \quad \underbrace{\mathbf{E}}_{T \times N} \equiv \begin{pmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \\ \vdots \\ \mathbf{e}_T^\top \end{pmatrix},$$

where each row corresponds to one time period t (observation) and each column corresponds to one asset (feature). With this notation, the static factor model (C.4.1) can be written compactly in matrix form as

$$\mathbf{R} = \mathbf{F} \mathbf{\Gamma}^\top + \mathbf{E}.$$

Relation to Fama-French Factor Pricing Models. In a sense, factor models provide a unifying framework for standard linear regression and Fama-French-type factor regressions. In the standard linear regression framework, the matrix $\mathbf{\Gamma}$ is fixed as the observed data matrix $\mathbf{X}_t$ (explanatory variables), which now varies over time, while the (regression) coefficient vector $\mathbf{f}_t \equiv \beta$ is fixed in time and estimated conditional on $\mathbf{X}_t$. By contrast, in Fama-French-style regressions the factors $\mathbf{f}_t$ are observed, such as those in the Fama-French three-factor model, and the factor loadings $\mathbf{\Gamma}$ are estimated, typically via time-series regressions performed separately for each asset, so that the n -th row of $\mathbf{\Gamma}$ is the loading vector of the n -th stock.

In a general factor model, however, *both* the factor loadings $\mathbf{\Gamma}$ and the factors $\mathbf{f}_t$ are unknown and constitute parameters to be jointly estimated. The objective is therefore not merely to estimate exposures to pre-specified factors, but to extract latent common components from the panel of asset returns. Thus, factor models seek to decompose the return data into a small number of pervasive signals that account for co-movement across assets.

Therefore, Fama-French-style empirical asset pricing represents a complementary, but conceptually distinct, approach. They construct factors a priori from characteristic-sorted long-short portfolios, which gives them an immediate economic interpretation, and empirical analysis focuses on testing whether these pre-specified factors are priced or explain the cross-section of expected returns. By contrast, a general factor model abstracts from such a pre-imposed structure and instead recovers latent factors directly from the data, which have to be interpreted *ex post*.

Identification. Factor models are subject to a fundamental indeterminacy. For any nonsingular $K \times K$ matrix $\mathbf{H}$, the representation

$$\mathbf{r}_t = \mathbf{\Gamma} \mathbf{f}_t + \mathbf{e}_t = (\mathbf{\Gamma} \mathbf{H})(\mathbf{H}^{-1} \mathbf{f}_t) + \mathbf{e}_t \equiv \tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_t + \mathbf{e}_t$$

is observationally equivalent, so that neither the scale nor the orientation of the factors is identified without further restrictions. To resolve this issue, it is customary to impose normalisations such as $\mathbf{\Gamma}' \mathbf{\Gamma} = \mathbf{I}_K$.¹⁸ These restrictions

¹⁸Note that in this case the factors $\mathbf{f}_t$ are only identified up to an orthogonal rotation $\mathbf{Q}$, i.e. $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, so that the restriction admits $\tilde{\mathbf{\Gamma}} := \mathbf{\Gamma} \mathbf{Q}$ and $\tilde{\mathbf{f}}_t := \mathbf{Q}^\top \mathbf{f}_t$. This can be used later on to ease interpretability of the factors by e.g. using the VariMax rotation. Moreover, the sign (direction) of any factor $f_{k,t}$ is not identified either.

do not carry economic content and simply select a particular orthogonal basis for the K -dimensional space spanned by the common components and ensure that different factors capture distinct, uncorrelated sources of systematic variation.

As in standard linear regression models, the error terms are assumed to be uncorrelated. This is a basic identifying restriction of the factor model since it is assumed that the factors drive the co-movement of stocks.

MLE Estimation. To obtain a likelihood-based estimator, additional distributional assumptions are imposed on the static factor model. A common assumption is that the latent factors and idiosyncratic errors are jointly Gaussian, mutually independent, and serially uncorrelated, with

$$\mathbf{f}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{\Omega}) \quad \text{and} \quad \mathbf{e}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{\Psi}),$$

where both $\mathbf{\Omega}$ and $\mathbf{\Psi}$ are diagonal matrices. The diagonal structure of $\mathbf{\Omega}$ ensures that factors represent distinct, uncorrelated sources of systematic variation, while $\mathbf{\Psi}$ captures asset-specific idiosyncratic risk.

Under these assumptions, returns are Gaussian with zero mean¹⁹ and covariance matrix

$$\mathbf{V}(\mathbf{r}) \equiv \mathbf{\Sigma}_r = \mathbf{\Gamma} \mathbf{\Omega} \mathbf{\Gamma}^\top + \mathbf{\Psi},$$

which decomposes the return covariance into a low-rank common component and a diagonal idiosyncratic component. Estimation by maximum likelihood proceeds by choosing $(\mathbf{\Gamma}, \mathbf{\Omega}, \mathbf{\Psi})$ to maximise the Gaussian log-likelihood of the observed returns $\{\mathbf{r}_t\}_{t=1}^T$.

The resulting maximum likelihood estimator jointly recovers the factor loadings and the variance parameters governing common and idiosyncratic risk. Conditional on the estimated parameters, the latent factors $\mathbf{f}_t$ can be obtained as the posterior mean under the Gaussian model, which coincides with the generalised least squares projection of returns onto the estimated factor loadings. In the special case where $\mathbf{\Psi} = \sigma^2 \mathbf{I}_N$, the model reduces to probabilistic principal component analysis (PPCA), and the MLE of $\mathbf{\Gamma}$ spans the same space as the leading principal components of the sample covariance matrix.

MSE Minimisation. While maximum likelihood estimation provides a fully specified statistical framework, it relies on distributional assumptions that are not essential for recovering the latent factor space. Computationally simpler, one can instead estimate the factor model by minimising the mean squared error, which is equivalent to maximising the explained variance of $\mathbf{\Gamma} \mathbf{f}_t$. Thus, the objective is to solve the following

$$\min_{\mathbf{\Gamma}, \mathbf{F}} = \frac{1}{T} \sum_{t=1}^T (\mathbf{r}_t - \mathbf{\Gamma} \mathbf{f}_t)^\top (\mathbf{r}_t - \mathbf{\Gamma} \mathbf{f}_t).$$

As in OLS, taking the derivative with respect to $\mathbf{f}_t$ yields

$$\mathbf{f}_t = (\mathbf{\Gamma}^\top \mathbf{\Gamma})^{-1} \mathbf{\Gamma}^\top \mathbf{r}_t,$$

which is the OLS estimator of regressing cross-sectional returns on the columns of $\mathbf{\Gamma}$, i.e. $r_{n,t} = \gamma_{n,1} f_{1,t} + \dots \gamma_{n,K} f_{K,t} + \varepsilon_{n,t} \forall n$.

When using $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$ as the identifying restriction, this simplifies to $\mathbf{f}_t = \mathbf{\Gamma}^\top \mathbf{r}_t$. Substituting this in and also taking the trace operator, which can be done since the trace of a scalar is simply the scalar, yields

$$\min_{\mathbf{\Gamma}} \frac{1}{T} \sum_{t=1}^T \text{Tr} \left[(\mathbf{r}_t - \mathbf{\Gamma} \mathbf{\Gamma}^\top \mathbf{r}_t) (\mathbf{r}_t - \mathbf{\Gamma} \mathbf{\Gamma}^\top \mathbf{r}_t)^\top \right].$$

¹⁹By including a constant in (C.4.1) the mean can be made non-zero immediately. Alternatively, one could impose $f_{1,t} = 1 \forall t$ such that the first factor is the constant.

Multiplying out the product and exploiting the trace property $\text{Tr}(AB) = \text{Tr}(BA)$ alongside the restriction $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$, the problem reduces to the following

$$\max_{\mathbf{\Gamma}} \text{Tr}(\mathbf{\Gamma}^\top \widehat{\mathbf{\Sigma}}_r \mathbf{\Gamma}), \quad \text{subject to } \mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K, \quad \text{with } \widehat{\mathbf{\Sigma}}_r \equiv \left[\frac{1}{T} \sum_{t=1}^T \mathbf{r}_t \mathbf{r}_t^\top \right]. \quad (\text{C.4.2})$$

This objective is the same as in Principle Component Analysis (PCA). The aim of PCA is to find a linear combination of the variables (here: stock returns) that maximises the variance. The j -th principal component vector γ_j aims to solve $\max \text{Var}(\gamma_j^\top \mathbf{r}) = \max \gamma_j^\top \widehat{\mathbf{\Sigma}}_r \gamma_j$ subject to $\gamma_j^\top \gamma_j = 1$ and being uncorrelated with the other principal component vectors, i.e. $\gamma_j^\top \gamma_{-j} = 0$. This is the same as the identifying restriction $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$. Since

$$\text{Tr}(\mathbf{\Gamma}^\top \widehat{\mathbf{\Sigma}}_r \mathbf{\Gamma}) = \sum_{k=1}^K \gamma_k^\top \widehat{\mathbf{\Sigma}}_r \gamma_k,$$

this shows that minimising the MSE of (C.4.1) results in finding linear combinations of the returns that maximises the variance, which is exactly the same as in PCA.

Solution. Therefore, the solution of $\mathbf{\Gamma}$ is given by standard PCA theory (see, e.g. Jolliffe (2004)). Thus, loading vector γ_k , which is associated with the k -th factor, corresponds to the eigenvector associated with the k -th largest eigenvalue of the sample covariance matrix $\widehat{\mathbf{\Sigma}}_r$.

Let $\widehat{\mathbf{\Sigma}}_r = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$ denote the spectral decomposition of $\widehat{\mathbf{\Sigma}}_r$, where the eigenvalues in $\mathbf{\Lambda}$ are ordered in decreasing magnitude and $\mathbf{V}$ contains the corresponding orthogonal eigenvectors. Under the identifying restriction $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$, the loading matrix must consist of K orthogonal vectors. Since the eigenvectors of $\widehat{\mathbf{\Sigma}}_r$ form an orthogonal basis, this restriction directly implies that $\mathbf{\Gamma} = \mathbf{V}_K$, where $\mathbf{V}_K$ collects the first K eigenvectors.

With this normalisation, the estimated factors are $\mathbf{f}_t = \mathbf{V}_K^\top \mathbf{r}_t$. Note that this implies that the covariance matrix of factors is diagonal since

$$\begin{aligned} \text{Var}(\mathbf{f}_t) &= \text{Var}(\mathbf{V}_K^\top \mathbf{r}_t) = \mathbf{V}_K^\top \text{Var}(\mathbf{r}_t) \mathbf{V}_K \\ &= \mathbf{V}_K^\top (\mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top) \mathbf{V}_K = (\mathbf{V}_K^\top \mathbf{V}) \mathbf{\Lambda} (\mathbf{V}^\top \mathbf{V}_K) \\ &= [\mathbf{I}_K \mathbf{0}_{K \times (N-K)}] \mathbf{\Lambda} [\mathbf{I}_K \mathbf{0}_{K \times (N-K)}]^\top = \mathbf{\Lambda}_K. \end{aligned}$$

Crucially, imposing $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$ and $\text{Var}(\mathbf{f}_t) = \mathbf{\Lambda}_K$ in the order of descending eigenvalues is a *unique* solution up to the sign of $f_{k,t}$ as no longer can an (orthogonal) rotation be applied²⁰. To see this, if we instead used $\tilde{\mathbf{\Gamma}} \equiv \mathbf{\Gamma} \mathbf{Q}$ and $\tilde{\mathbf{f}}_t \equiv \mathbf{Q}^\top \mathbf{f}_t$, where $\mathbf{Q}$ is an orthogonal matrix, then indeed $\tilde{\mathbf{\Gamma}}^\top \tilde{\mathbf{\Gamma}} = \mathbf{I}_K$ still holds, but $\text{Var}(\tilde{\mathbf{f}}_t) = \mathbf{Q}^\top \mathbf{\Lambda}_K \mathbf{Q}$ which is no longer diagonal. However, we can flip the sign of any set of elements in $\mathbf{f}_t$ if we flip the sign of the corresponding loadings so that a fully unique solution is only obtained if the sign of $\mathbf{f}_t$ is fixed.

Equivalently, one may adopt an alternative normalisation $\widehat{\mathbf{\Gamma}} = \mathbf{V}_K \mathbf{\Lambda}_K^{1/2}$, such that $\widehat{\mathbf{\Gamma}}^\top \widehat{\mathbf{\Gamma}}$ is diagonal, in which case the factors satisfy $\text{Var}(\widehat{\mathbf{f}}_t) = \mathbf{I}_K$ and the scaling is absorbed into the loadings. Both normalisations recover the same factor space and differ only in how the variance of the common component is allocated between factor loadings and factor variances.

Note that, in general, factor analysis and PCA do not coincide and in this case is an artefact of using minimising the MSE which is equivalent to maximising explained variance.

VariMax. This method only imposes the identifying restriction $\mathbf{\Gamma}^\top \mathbf{\Gamma} = \mathbf{I}_K$, but is unconcerned with the variance structure of factors and instead finds an orthogonal rotation $\mathbf{Q}$ in order to ease the interpretation of the factors. In doing so, VariMax maximises the variance of squared entries of the loading vectors, i.e. the γ_k . Formally, VariMax

²⁰Recall that any orthogonal matrix $\mathbf{Q}$ satisfies $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_K$.

chooses $\mathbf{Q}$ to maximise

$$\sum_{k=1}^K \left[\frac{1}{N} \sum_{n=1}^N \tilde{\gamma}_{nk}^4 - \left(\frac{1}{N} \sum_{n=1}^N \tilde{\gamma}_{nk}^2 \right)^2 \right],$$

where $\tilde{\gamma}_{nk}$ denotes the (n, k) -th element of $\tilde{\mathbf{\Gamma}}$. The squared entries are considered as only the magnitude is important and the direction (sign) is irrelevant. Each column in $\tilde{\mathbf{\Gamma}}$ ought to have only either large or small values.²¹

C.4.2 IPCA

One of the main drawbacks of the static factor model for empirical asset pricing is that without further refinements the static factor model requires a balanced panel and that the loadings onto the factors are the same in each period, i.e. $\mathbf{\Gamma}$ does not depend on time.

IPCA (Kelly et al. 2019) addresses these issues by using stock characteristics as proxies for dynamic factor loadings. Therefore, the IPCA model is given by

$$\begin{aligned} r_{i,t+1} &= \alpha_{i,t} + \beta_{i,t}^\top \mathbf{f}_{t+1} + \epsilon_{i,t+1} \\ \text{with} \\ \beta_{i,t}^\top &= \mathbf{z}_{i,t}^\top \mathbf{\Gamma}_\beta + \nu_{\beta,i,t}^\top \\ \alpha_{i,t} &= \mathbf{z}_{i,t}^\top \mathbf{\Gamma}_\alpha + \nu_{\alpha,i,t} \end{aligned}$$

where $\mathbf{f}_{t+1}$ is a $K \times 1$ vector of latent factors, $\mathbf{z}_{i,t}$ is an $L \times 1$ vector of stock characteristics, $\mathbf{\Gamma}_\beta$ is an $L \times K$ matrix and $\mathbf{\Gamma}_\alpha$ is an $L \times 1$ matrix. Typically, $K \ll L$ should be chosen since financial data is noisy so that $\mathbf{\Gamma}$ -matrices are intended to find the best K linear combinations of these characteristics (dimensionality reduction).

From the static factor model it is known that minimising the mean squared error (MSE) yields a loading matrix ($\mathbf{\Gamma}_\beta$) that captures the principal component vectors. IPCA extends this logic by "anchoring" these principal components to fundamental stock characteristics. By leveraging the cross-section of characteristics rather than relying solely on the return history, IPCA incorporates a significantly broader information set into the factor estimation process

The IPCA model can be written more compactly by inserting $\beta_{i,t}$ and $\alpha_{i,t}$, such that

$$r_{i,t+1} = \mathbf{z}_{i,t}^\top \tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_{t+1} + \epsilon_{i,t+1}^*$$

where $\tilde{\mathbf{\Gamma}} \equiv [\mathbf{\Gamma}_\alpha, \mathbf{\Gamma}_\beta]$ and $\tilde{\mathbf{f}}_{t+1} \equiv [1, \mathbf{f}_{t+1}^\top]^\top$. In other words, $\tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_{t+1} = \mathbf{\Gamma}_\alpha + \mathbf{\Gamma}_\beta \mathbf{f}_{t+1}$ and the new error term is simply a composite error containing the leftover terms.

Vector Form. Let's write the IPCA model in vector form which will ease notation. Let $\mathbf{r}_{t+1} = [r_{1,t+1}, \dots, r_{N_t,t+1}]^\top$ be the $N_t \times 1$ vector of returns and $\mathbf{Z}_t = [\mathbf{z}_{1,t}, \dots, \mathbf{z}_{N_t,t}]^\top$ be the $N_t \times L$ matrix of stacked characteristics. The vector form (cross-section) is then given by

$$\mathbf{r}_{t+1} = \mathbf{Z}_t \tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_{t+1} + \boldsymbol{\epsilon}_{t+1}^*.$$

Objective. The primary objective of IPCA is to minimise the Mean Squared Error (MSE), defined as

$$\min_{\tilde{\mathbf{\Gamma}}, \{\mathbf{f}_t\}_{t=1}^T} \sum_{t=1}^T \omega_t \left(\mathbf{r}_{t+1} - \mathbf{Z}_t \tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_{t+1} \right)^\top \left(\mathbf{r}_{t+1} - \mathbf{Z}_t \tilde{\mathbf{\Gamma}} \tilde{\mathbf{f}}_{t+1} \right), \quad (\text{C.4.3})$$

where ω_t represents the period-specific weight. In the standard case, $\omega_t = \frac{1}{\sum_{t=1}^T N_t} \forall t$. In a weighted least squares settings, $\omega_t = \frac{1}{N_t}$ can be chosen. This choice ensures that the estimation considers the average squared error within

²¹VariMax can also be applied to the principal component vectors in PCA

each cross-section, preventing periods with a larger number of stocks from exerting a disproportionate influence on the objective function simply due to the higher number of terms in the summation.

Optimal Factors. Finding the FOC for $\mathbf{f}_{t+1}$ requires disentangling it from $\tilde{\mathbf{f}}_{t+1}$. Therefore, using $\tilde{\mathbf{\Gamma}}\tilde{\mathbf{f}}_{t+1} = \mathbf{\Gamma}_\alpha + \mathbf{\Gamma}_\beta\mathbf{f}_{t+1}$, consider the re-formulated IPCA model given by

$$(\mathbf{r}_{t+1} - \mathbf{Z}_t\mathbf{\Gamma}_\alpha) = (\mathbf{Z}_t\mathbf{\Gamma}_\beta)\mathbf{f}_{t+1} + \boldsymbol{\epsilon}_{t+1}^*.$$

Let $\mathbf{y}_{t+1} \equiv \mathbf{r}_{t+1} - \mathbf{Z}_t\mathbf{\Gamma}_\alpha$ be the "intercept-adjusted" returns and $\mathbf{X}_t \equiv \mathbf{Z}_t\mathbf{\Gamma}_\beta$ be the regressor matrix (the "data matrix"). To find the optimal $\mathbf{f}_{t+1}$, the relevant MSE objective is $\omega_t(\mathbf{y}_{t+1} - \mathbf{X}_t\mathbf{f}_{t+1})^\top(\mathbf{y}_{t+1} - \mathbf{X}_t\mathbf{f}_{t+1})$. Minimising this with respect to $\mathbf{f}_{t+1}$ is nothing else than the OLS-estimator. Substituting the expression for $\mathbf{y}_{t+1}$ and $\mathbf{X}_t$ back in then yields

$$\mathbf{f}_{t+1} = (\mathbf{\Gamma}_\beta^\top \mathbf{Z}_t^\top \mathbf{Z}_t \mathbf{\Gamma}_\beta)^{-1} \mathbf{\Gamma}_\beta^\top \mathbf{Z}_t^\top (\mathbf{r}_{t+1} - \mathbf{Z}_t \mathbf{\Gamma}_\alpha). \quad (\text{C.4.4})$$

Notice that the weight ω_t disappears when differentiating and setting the derivative to zero as it is merely a multiplicative scalar.

Optimal Loadings. To find the optimal $\tilde{\mathbf{\Gamma}}$, rewrite the model using the vectorisation operator since it is easier to take the derivative with respect to a vector. Note that the vector $\mathbf{Z}_t\tilde{\mathbf{\Gamma}}\tilde{\mathbf{f}}_{t+1}$ can be expressed as²²

$$\mathbf{Z}_t\tilde{\mathbf{\Gamma}}\tilde{\mathbf{f}}_{t+1} = \text{vec}(\mathbf{Z}_t\tilde{\mathbf{\Gamma}}\tilde{\mathbf{f}}_{t+1}) = (\tilde{\mathbf{f}}_{t+1}^\top \otimes \mathbf{Z}_t)\text{vec}(\tilde{\mathbf{\Gamma}}).$$

Plugging this into (C.4.3) and taking the derivative with respect to $\text{vec}(\tilde{\mathbf{\Gamma}})$ is once again as in standard OLS. Therefore, the FOC is given by

$$\sum_{t=1}^T \omega_t (\tilde{\mathbf{f}}_{t+1}^\top \otimes \mathbf{Z}_t)^\top (\mathbf{r}_{t+1} - (\tilde{\mathbf{f}}_{t+1}^\top \otimes \mathbf{Z}_t)\text{vec}(\tilde{\mathbf{\Gamma}})) = 0$$

Rearranging to solve for $\text{vec}(\tilde{\mathbf{\Gamma}})$ yields

$$\text{vec}(\tilde{\mathbf{\Gamma}}) = \left(\sum_{t=1}^T \omega_t (\tilde{\mathbf{f}}_{t+1} \tilde{\mathbf{f}}_{t+1}^\top) \otimes (\mathbf{Z}_t^\top \mathbf{Z}_t) \right)^{-1} \left(\sum_{t=1}^T \omega_t (\tilde{\mathbf{f}}_{t+1} \otimes \mathbf{Z}_t^\top) \mathbf{r}_{t+1} \right).$$

Identification. While the FOCs provide a path to a solution, as with the static factor model, IPCA is subject to rotational indeterminacy. To ensure a unique solution, Kelly et al. (2019) impose the following:

- $\mathbf{\Gamma}_\beta^\top \mathbf{\Gamma}_\beta = \mathbf{I}_K$ (Orthogonality of loadings).
- $\mathbb{E}[\mathbf{f}_t \mathbf{f}_t^\top]$ is diagonal with descending elements (Ordering of factors).
- $\mathbf{\Gamma}_\alpha^\top \mathbf{\Gamma}_\beta = \mathbf{0}_{1 \times K}$ (Orthogonality of intercept and loadings).
- Factor mean $\mathbb{E}[\mathbf{f}_t] > \mathbf{0}$ (Sign).

In the following it will be shown how to impose these restrictions and how they ensure a unique solution.

Assume we have a solution for $\mathbf{\Gamma}_\beta$ that satisfies the FOC. Naturally, this doesn't automatically guarantee that it is an orthogonal matrix. To guarantee this, simply use the Singular Value Decomposition such that $\mathbf{\Gamma}_\beta = \mathbf{U}\mathbf{S}\mathbf{V}^\top$, where $\mathbf{U}$ is an $L \times K$ semi-orthogonal matrix ($\mathbf{U}^\top \mathbf{U} = \mathbf{I}_K$), $\mathbf{S}$ is a $K \times K$ diagonal matrix and $\mathbf{V}$ is a $K \times K$ orthogonal matrix. We then use $(\mathbf{S}\mathbf{V}^\top)^{-1}$ as the rotation such that we set $\mathbf{\Gamma}_\beta^{new} := \mathbf{\Gamma}_\beta(\mathbf{S}\mathbf{V}^\top)^{-1}$ and $\mathbf{f}_t^{new} = (\mathbf{S}\mathbf{V}^\top)\mathbf{f}_t$. Thus, we can always achieve that $\mathbf{\Gamma}_\beta$ is orthogonal.

Next, assume we have a solution for $\mathbf{\Gamma}_\beta$ satisfying $\mathbf{\Gamma}_\beta^\top \mathbf{\Gamma}_\beta = \mathbf{I}_K$. Naturally, the $\mathbf{f}_t$ satisfying their FOC will not automatically have a diagonal covariance matrix. However, using the Spectral Decomposition, such that $\mathbf{V}(\mathbf{f}) =$

²²Recall that $\text{vec}(ABC) = (\mathbf{C}^\top \otimes \mathbf{A})\text{vec}(\mathbf{B})$

$\Sigma_f = V\Lambda V^\top$, simply use $\Gamma_\beta^{new} = \Gamma_\beta V$ and $f_t^{new} = V^\top f_t$. Since V is an orthogonal matrix, one can instantly verify that $\Gamma_\beta^{new\top} \Gamma_\beta^{new} = I_K$ still holds. Likewise, we now have $V(f_t^{new}) = V^\top V\Lambda V^\top V = \Lambda$ so that the second restriction can always be ensured.

Next, to see the relevance of the third restriction, it is always possible to re-write the IPCA model as $r_t = Z_t(\Gamma_\alpha - \Gamma_\beta \xi) + Z_t \Gamma_\beta (f_{t+1} + \xi) + \varepsilon_{t+1}$ for some vector ξ which gives rise to indeterminacy for Γ_α . Enforcing $\Gamma_\alpha^\top \Gamma_\beta = 0$ removes the possibility of adding such a vector $\xi \neq 0$ and this condition can always be enforced by regressing Γ_α on Γ_β and setting Γ_α to the orthogonal residual. In other words, the vector Γ_α is the vector of dependent variables and Γ_β is "data matrix". If we let the regression coefficients be denoted by ξ , then we have by OLS $\xi = (\Gamma_\beta^\top \Gamma_\beta)^{-1} \Gamma_\beta^\top \Gamma_\alpha$. In this case, $\Gamma_\alpha^{new} = \Gamma_\alpha - \Gamma_\beta \xi$ is the solution since this is the residual of the OLS regression which is orthogonal to the data matrix Γ_β .

Lastly, ensuring the sign is easy. For each element f_k for which the mean over time was less than zero, multiply the entry by -1 and likewise multiply the k -th column of Γ_β by -1 .

Numerical Procedure. Since both FOCs are coupled, Kelly et al. (2019) opt for an alternating algorithm which first solves for f_{t+1} given $\tilde{\Gamma}$ and then given this f_{t+1} solves for $\tilde{\Gamma}$. This process is repeated until the objects converge up to a desired tolerance level.

To initiate the algorithm, start by assuming $\Gamma_\alpha = 0$. Next, an initial guess for Γ_β will be derived.

Plug the FOC for f_{t+1} (C.4.4) into the objective function (C.4.3). This yields

$$\min_{\Gamma_\beta} \sum_{t=1}^T \omega_t \left(r_{t+1} - Z_t \Gamma_\beta f_{t+1} \right)^\top \left(r_{t+1} - Z_t \Gamma_\beta f_{t+1} \right). \quad (C.4.5)$$

Expanding the terms yields

$$\begin{aligned} & \min_{\Gamma_\beta} \sum_{t=1}^T \omega_t (r_{t+1}^\top r_{t+1} - 2r_{t+1}^\top Z_t \Gamma_\beta f_{t+1} + f_{t+1}^\top \Gamma_\beta^\top Z_t^\top Z_t \Gamma_\beta f_{t+1}) \\ & \min_{\Gamma_\beta} = \sum_{t=1}^T \omega_t (r_{t+1}^\top r_{t+1} - r_{t+1}^\top Z_t \Gamma_\beta (\Gamma_\beta^\top Z_t^\top Z_t \Gamma_\beta)^{-1} \Gamma_\beta^\top Z_t^\top r_{t+1}) \\ & \iff \\ & \max_{\Gamma_\beta} \sum_{t=1}^T \omega_t (r_{t+1}^\top Z_t \Gamma_\beta (\Gamma_\beta^\top Z_t^\top Z_t \Gamma_\beta)^{-1} \Gamma_\beta^\top Z_t^\top r_{t+1}). \end{aligned}$$

The last line follows since r does not depend on Γ_β and the sign is simply flipped which flips the operator from a minimiser to a maximiser.

Since the above is a scalar, applying the trace operator does not change the objective. Then, using the property of the trace, $\text{tr}(ABCD) = \text{tr}(BCDA)$ yields

$$\max_{\Gamma_\beta} \sum_{t=1}^T \text{Tr} \left(\omega_t (\Gamma_\beta^\top Z_t^\top Z_t \Gamma_\beta)^{-1} \Gamma_\beta^\top Z_t^\top r_{t+1} r_{t+1}^\top Z_t \Gamma_\beta \right).$$

Notice that this is similar to (C.4.2) and indeed the above will be simplified with further assumptions such that Γ_β is initialised as the PCA solution.

To proceed, Kelly et al. (2019) define the following intermediary objects (characteristics managed portfolio)

$$x_{t+1} = \frac{1}{N_t} Z_t^\top r_{t+1} \quad \text{and} \quad \frac{Z_t^\top Z_t}{N_t} \approx W,$$

with $W = \frac{1}{T} \sum_{t=1}^T \frac{Z_t^\top Z_t}{N_t}$ being the full sample covariance matrix of characteristics²³. Expanding $Z_t^\top Z_t$ with

²³Recall that $Z_t^\top Z_t = \sum_{i=1}^{N_t} z_{i,t} z_{i,t}^\top$

N_t and substituting the above quantities in yields

$$\max_{\mathbf{\Gamma}_\beta} \sum_{t=1}^T \text{Tr} \left(\omega_t \frac{1}{N_t} (\mathbf{\Gamma}_\beta^\top \mathbf{W} \mathbf{\Gamma}_\beta)^{-1} N_t^2 \mathbf{\Gamma}_\beta^\top \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top \mathbf{\Gamma}_\beta \right)$$

Assuming weighted least squares, this yields

$$\max_{\mathbf{\Gamma}_\beta} \text{tr} \left((\mathbf{\Gamma}_\beta^\top \mathbf{W} \mathbf{\Gamma}_\beta)^{-1} \mathbf{\Gamma}_\beta^\top \left[\sum_{t=1}^T \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top \right] \mathbf{\Gamma}_\beta \right)$$

This is almost a standard PCA problem. To fully achieve it, notice that we can consider a rotation of $\mathbf{\Gamma}_\beta$ to get there. Write $\mathbf{W} = \mathbf{W}^{0.5} \mathbf{W}^{0.5}$, which always works as $\mathbf{W}$ is a symmetric positive definite matrix. Then, define $\mathbf{\Phi}_\beta = \mathbf{W}^{0.5} \mathbf{\Gamma}_\beta$ to get

$$\max_{\mathbf{\Phi}_\beta} \text{Tr} \left((\mathbf{\Phi}_\beta^\top \mathbf{\Phi}_\beta)^{-1} \mathbf{\Phi}_\beta^\top \mathbf{\Omega} \mathbf{\Phi}_\beta \right)$$

with $\mathbf{\Omega} \equiv \mathbf{W}^{-0.5} \left[\sum_{t=1}^T \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top \right] \mathbf{W}^{-0.5}$ and imposing $\mathbf{\Phi}_\beta^\top \mathbf{\Phi}_\beta = \mathbf{I}$. In practice, however, due to data pre-processing, such as cross-sectional rank-standardisation or Z-scores, it is somewhat reasonable to assume $\mathbf{W} \approx \mathbf{I}$ which is also what [Kelly et al. \(2019\)](#) do. This is computationally much cheaper, since $\mathbf{\Gamma}_\beta$ is now simply given by the K eigenvectors with the K largest eigenvalues of $\sum_{t=1}^T \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top$ and the initial guess is only an approximation anyway.

Algorithm 2: IPCA Alternating Weighted Least Squares**1. Initialisation:**

- Set $\mathbf{\Gamma}_\alpha^{(0)} = \mathbf{0}_{L \times 1}$.
- Compute the managed portfolios $\mathbf{x}_{t+1} = \frac{\mathbf{Z}_t^\top \mathbf{r}_{t+1}}{\mathbf{N}_t}$ and $\mathbf{W}_t = \frac{\mathbf{Z}_t^\top \mathbf{Z}_t}{\mathbf{N}_t}$ for all t .
- Initialise $\mathbf{\Gamma}_\beta^{(0)}$ as the K eigenvectors corresponding to the K largest eigenvalues of $\sum_{t=1}^T \mathbf{x}_{t+1} \mathbf{x}_{t+1}^\top$.

2. Iterative Step: For $j = 0, 1, 2, \dots$ until convergence.

- **Update Factors:** Given $\tilde{\mathbf{\Gamma}}^{(j)}$, calculate the latent factors for each t :

$$\mathbf{f}_{t+1}^{(j+1)} = \left(\mathbf{\Gamma}_\beta^{(j)\top} \mathbf{W}_t \mathbf{\Gamma}_\beta^{(j)} \right)^{-1} \mathbf{\Gamma}_\beta^{(j)\top} \left(\mathbf{x}_{t+1} - \mathbf{W}_t \mathbf{\Gamma}_\alpha^{(j)} \right).$$

- **Update Loadings:** Given $\tilde{\mathbf{f}}_{t+1}^{(j+1)} = [1, \mathbf{f}_{t+1}^{(j+1)\top}]^\top$, update the mapping matrix:

$$\text{vec}(\tilde{\mathbf{\Gamma}}^{(j+1)}) = \left(\sum_{t=1}^T \left(\tilde{\mathbf{f}}_{t+1}^{(j+1)} (\tilde{\mathbf{f}}_{t+1}^{(j+1)})^\top \right) \otimes \mathbf{W}_t \right)^{-1} \left(\sum_{t=1}^T \tilde{\mathbf{f}}_{t+1}^{(j+1)} \otimes \mathbf{x}_{t+1} \right).$$

Unstack the $(K+1)L \times 1$ vector $\text{vec}(\tilde{\mathbf{\Gamma}}^{(j+1)}) = [\gamma_0^\top, \gamma_1^\top, \dots, \gamma_K^\top]^\top$ to

$\tilde{\mathbf{\Gamma}}^{(j+1)} = [\gamma_0, \gamma_1, \dots, \gamma_K]$ where each γ_k is an $L \times 1$ vector.

Then, $\mathbf{\Gamma}_\alpha = \gamma_0$ and $\mathbf{\Gamma}_\beta = [\gamma_1, \dots, \gamma_K]$.

- **Identification:** Apply the identification constraints:

- (a) Orthogonalise $\mathbf{\Gamma}_\beta$. Compute its SVD $\mathbf{\Gamma}_\beta = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ and set

$$\mathbf{\Gamma}_\beta \leftarrow \mathbf{U} \quad \text{and} \quad \mathbf{f}_{t+1} \leftarrow \mathbf{S} \mathbf{V}^\top \mathbf{f}_{t+1} \quad \forall t.$$

- (b) Diagonalise $\mathbf{\Sigma}_f$. Compute its Spectral Decomposition, $\mathbf{\Sigma}_f = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$, and set

$$\mathbf{f}_{t+1} \leftarrow \mathbf{V}^\top \mathbf{f}_{t+1} \quad \forall t \quad \text{and} \quad \mathbf{\Gamma}_\beta \leftarrow \mathbf{\Gamma}_\beta \mathbf{V}.$$

- (c) Orthogonalise $\mathbf{\Gamma}_\alpha$. Compute $\mathbf{\xi} = (\mathbf{\Gamma}_\beta^\top \mathbf{\Gamma}_\beta)^{-1} \mathbf{\Gamma}_\beta^\top \mathbf{\Gamma}_\alpha$ and set

$$\mathbf{\Gamma}_\alpha \leftarrow \mathbf{\Gamma}_\alpha - \mathbf{\Gamma}_\beta \mathbf{\xi} \quad \text{and} \quad \mathbf{f}_{t+1} \leftarrow \mathbf{f}_{t+1} + \mathbf{\xi} \quad \forall t.$$

- (d) Sign Restriction. Compute $\bar{\mathbf{f}} = \frac{1}{T} \sum_{t=1}^T \mathbf{f}_{t+1}$. For each $\bar{f}_k < 0$, multiply the k -th element of $\mathbf{f}_{t+1}$ by -1 for all t and multiply the respective k -th column of $\mathbf{\Gamma}_\beta$ by -1 .

- **Convergence:** Check if the projection converges. Compute

$$\mathbf{P}^{(j+1)} = \mathbf{\Gamma}_\alpha^{(j+1)} \mathbf{1}_T^\top + \mathbf{\Gamma}_\beta^{(j+1)} [\tilde{\mathbf{f}}_1^{(j+1)}, \dots, \tilde{\mathbf{f}}_T^{(j+1)}]$$

and stop if $\|\mathbf{P}^{(j+1)} - \mathbf{P}^{(j)}\|_\infty < \varepsilon$.